

人工智能中的编程

从 CUDA 到深度学习系统

理解一次计算如何被表达、求导、编译与执行

VON

课程参考：2025 秋人工智能中的编程

授课教师：王鹏帅、楚梦渝、梁一韬

目录

从哪里开始	v
第 1 章 一次训练步骤里，究竟发生了什么	1
1.1 把训练拆成可检查的计算	1
1.2 先做一份可信的串行参考	1
1.3 同一计算有多种表示	2
第 2 章 CUDA 从零开始：让一个线程知道自己该做什么	4
2.1 Host 与 Device：两边各做什么	4
2.2 逐行读懂第一个核函数	4
2.3 尖括号里的四个位置	5
2.4 从主机数组到设备结果	7
2.5 编译、验证与最先排查的错误	8
2.6 异步执行：启动返回不等于计算完成	8
2.7 二维线程、展平索引与张量形状	8
2.8 网格跨步循环：每条线程可以做不止一个元素	9
第 3 章 线程如何协作：执行、内存与同步	11
3.1 Block、Warp 与 SM 不是同一个东西	11
3.2 内存名称首先描述可见范围和访问方式	12
3.3 合并访存：看一组线程访问了哪些地址	13
3.4 先写后读：屏障到底解决什么	14
3.5 原子操作：对同一位置的修改怎样合并	15
3.6 从通信模式判断反向计算	16
3.7 流、事件与测量：让先后关系可见	16
第 4 章 并行基本算法：把依赖关系改写成可协作的步骤	19
4.1 工作量与关键路径要分开看	19
4.2 归约：把很多元素变成一个结果	19
4.3 直方图：把热点写入先分散	22
4.4 扫描：不仅求总和，还求每个位置前面的和	23
4.5 筛选、分段与递推为什么都能用扫描	25
4.6 排序网络与基数排序	26
4.7 转置：用共享内存改变访存方向	28
第 5 章 矩阵乘法：复用数据比多写线程更重要	29
5.1 先统一形状、布局和工作量	29
5.2 分块：一批输入支持一批输出	30
5.3 用数据流量估计复用收益	31
5.4 Roofline：算得慢，还是搬得慢	32
5.5 cuBLAS：接口中的布局也是数学的一部分	32
5.6 稀疏矩阵：少做乘法之后，瓶颈去了哪里	34
5.7 矩阵乘法的反向为什么又是矩阵乘法	35

第 6 章 神经网络算子：从前向索引追到反向累积	36
6.1 卷积的形状与下标约定	36
6.2 直接实现：一条线程负责一个输出	37
6.3 im2col：把局部窗口排成矩阵	37
6.4 卷积的反向：共享的位置都要累加	38
6.5 最大池化：保存选择，反向再分配梯度	40
6.6 Softmax：两次归约与数值范围	41
6.7 交叉熵：把稳定性延伸到损失计算	43
第 7 章 连接 Python 与底层算子：边界上的约定	44
7.1 模块名、函数名与编译产物	44
7.2 NumPy 数组：形状之外还有步长	44
7.3 构建：让依赖于明确的目标	46
7.4 接入 PyTorch：前向函数还缺少哪些信息	46
7.5 测试要对准边界，而不只跑一个随机数组	47
第 8 章 自动微分：把链式法则变成可执行的程序	49
8.1 先区分三种求导方式	49
8.2 把一个函数拆成图上的节点	49
8.3 前向模式：把一个输入方向沿图推进	50
8.4 反向模式：从输出的重要性回传到输入	51
8.5 JVP 与 VJP：通常不需要完整 Jacobian	52
8.6 张量的局部反向：形状变化如何逆向处理	52
8.7 实现一个小型反向引擎	53
8.8 保存值、原地修改与高阶导数	56
8.9 控制流与不可微操作的边界	56
第 9 章 计算图与训练框架：把算子组织成一个系统	57
9.1 数据、算子与参数承担不同责任	57
9.2 用已有算子训练一个小网络	57
9.3 图表示的不只是函数调用顺序	59
9.4 即时执行与先建图执行	59
9.5 图捕获的核心问题：这次记录能否用于下一次	59
9.6 前向图与反向图可以一起优化	60
第 10 章 编译器前端：看见整个计算，才能跨算子优化	61
10.1 IR 是一套明确的程序约定	61
10.2 常量折叠与推理时的 BatchNorm 合并	61
10.3 公共子表达式与死代码消除	62
10.4 算子融合：省掉哪些实际工作	64
10.5 内存规划：复用已经结束的生命期	65
10.6 布局变换与执行分派	66
第 11 章 优化器与数值精度：梯度算出来以后怎样更新	67
11.1 梯度下降：方向正确，步子仍可能太大	67
11.2 动量：把过去的方向保留下来	68
11.3 二阶信息：校正不同方向的曲率	68
11.4 自适应缩放：每个坐标使用不同尺度	69
11.5 Adam 与 AdamW：状态、偏差修正与权重衰减	69

11.6 训练策略仍要与模型和数据配合	70
11.7 混合精度：表示范围与表示精细程度	71
11.8 多个目标与约束：先明确优化的对象	73
第 12 章 编译器后端：把一个算子变成硬件上的工作	74
12.1 计算定义与执行安排分开	74
12.2 循环变换必须尊重依赖	74
12.3 分块、展开与向量化分别改变什么	75
12.4 存储层次与双缓冲	75
12.5 自动调优：搜索的是有限且受约束的实现空间	76
12.6 减少提交开销，不等于把整个网络变成一个 kernel	76
12.7 训练内存：参数只是其中一部分	76
12.8 激活重算：用额外计算换内存	77
第 13 章 数据管线与异构系统：让计算单元持续拿到数据	79
13.1 Dataset 与 DataLoader 分别负责什么	79
13.2 多工作进程：提速之前先避免重复读取	80
13.3 预取与流水线：把等待搬到前面	80
13.4 存储格式与检查点	81
13.5 异构硬件：专用单元需要相应的数据组织	82
13.6 节点互联：设备位置也会改变成本	83
第 14 章 分布式训练：把局部梯度合成同一次更新	84
14.1 进程、设备与 rank	84
14.2 数据并行究竟平均什么	84
14.3 集合通信：按语义选择操作	85
14.4 环形归约：每一步只与邻居交换一块	86
14.5 把同步 SGD 写成一个可运行步骤	88
14.6 通信与反向重叠	89
14.7 参数服务器、异步更新与扩展效率	89
第 15 章 模型、张量与流水线并行：切开以后怎样接回来	90
15.1 先区分三种拆分对象	90
15.2 列并行：拆输出特征，结果需要拼接	90
15.3 行并行：拆归约维，结果需要相加	90
15.4 两层前馈网络怎样避免中间聚集	92
15.5 流水线：把不同微批次送进不同阶段	92
15.6 状态分片：不让每个副本保存所有优化器状态	94
15.7 组合并行：通信应发生在哪个组里	94
15.8 选择划分时，把重新分布成本算进去	95
第 16 章 量化：用更少的位数表达计算	96
16.1 整数并不知道自己代表多大的实数	96
16.2 矩阵乘法怎样转成整数累加	97
16.3 量化粒度：一个 scale 还是很多个	98
16.4 校准与量化感知训练	99
16.5 码本与剪枝：压缩表示不等于计算一定更快	99
第 17 章 大模型推理与服务：每生成一个 token，要做哪些工作	100

17.1 Prefill 与 Decode 的工作不同	100
17.2 为什么缓存 K、V，而不必每次重算全部历史	100
17.3 从固定批次到连续批处理	101
17.4 分页 KV cache：逻辑连续，物理可以分散	102
17.5 FlashAttention：不保存完整注意力矩阵	103
17.6 推测解码：先提出候选，再由目标模型核验	104
17.7 把服务优化放回完整请求路径	105
第 18 章 联邦学习：数据留在各处以后怎样共同训练	106
18.1 系统差异与数据差异同时存在	106
18.2 FedAvg：多走几步，再交换结果	106
18.3 FedProx：限制本地模型离公共起点太远	107
18.4 一个共同模型，还是每个客户保留自己的模型	108
18.5 不上传原始样本，并不直接给出隐私保证	108
18.6 失败、偏差与状态管理	109
附录 A 完整起点与代码运行	110
A.1 CUDA 的最小完整程序	110
A.2 设备内存辅助类	111
A.3 在 Linux 与 NVIDIA GPU 环境运行	112
A.4 Python 参考与绑定工程	112
附录 B 一阶自动微分引擎的完整定义	114
B.1 节点、局部规则与反向遍历	114
B.2 从最小图开始核对	118
附录 C 贯通几个难点的自检	119
C.1 线程和归约	119
C.2 广播和梯度	119
C.3 并行拆分与归一化	119
C.4 在线注意力为什么要重缩放旧状态	119
参考文献	120

从哪里开始

写出一层神经网络并不难。真正的挑战是解释它为什么算对了，梯度从哪里来，以及数据和线程怎样配合，才能在合理时间内得到结果。本讲义沿着这条路线，把算子、自动微分、编译与分布式执行连在一起，课程主线依据北京大学 2025 年秋季课件。^[1]

读者应熟悉基本的 C++、Python、线性代数和微积分。CUDA 则从零开始：第一次出现的关键语法会解释清楚，随后再把它用于归约、矩阵乘法和神经网络算子。理解代码时，应始终追踪三个问题：谁在执行、它访问哪一段数据、访问之间有什么先后约束。

CUDA 示例以 C++17 为基线。Python 数组示例统一用 `import numpy as np` 导入 NumPy；PyTorch 示例会使用相应框架接口。短片段解释局部步骤，独立程序提供输入、调用和结果检查，基础辅助类的完整定义收入附录。

数学推导中的数组形状是接口的一部分。代码中的索引、步长与同步，也是算法的一部分。我们会先用小例子建立直觉，再说明一般形式及适用前提。性能分析会写清假设；某一种 GPU 上的具体数字不能替代通用结论。

第 1 章 一次训练步骤里，究竟发生了什么

在 Python 中，训练可能只需要几行代码。但每一行都依赖一套具体机制：张量保存数据，算子读取并写入数组，自动微分生成梯度，优化器修改参数，运行时把工作安排到处理器上。先把这条路径看清楚，后面的 CUDA、编译器和分布式计算才有共同的落点。

1.1 把训练拆成可检查的计算

考虑一个批次的线性分类器。设输入 $X \in \mathbb{R}^{B \times D}$ ，参数 $W \in \mathbb{R}^{D \times C}$ ，偏置 $b \in \mathbb{R}^C$ 。 B 是批大小， D 是输入维数， C 是类别数。前向计算为

$$S = XW + \mathbf{1}b^\top, \quad P_{ic} = \frac{\exp S_{ic}}{\sum_{j=1}^C \exp S_{ij}}, \quad L = -\frac{1}{B} \sum_{i=1}^B \log P_{i,y_i}.$$

S 的每一行给一个样本的各类别打分。Softmax 把一行分数变为和为 1 的概率，损失再衡量正确类别得到的概率是否足够大。此时还没有任何“训练魔法”，只有矩阵乘法、逐元素操作、归约与索引。

令 Y 是标签的独热矩阵。反向传播会得到

$$G = \frac{\partial L}{\partial S} = \frac{P - Y}{B}, \quad \frac{\partial L}{\partial W} = X^\top G, \quad \frac{\partial L}{\partial b} = \sum_{i=1}^B G_{i,:}.$$

这些式子的推导将在自动微分与损失函数部分展开。现在先检查形状： $X^\top$ 是 $D \times B$ ，与 $B \times C$ 的 G 相乘，恰好得到与 W 相同的 $D \times C$ 梯度。

原理剖析 梯度必须回答“对谁求导”

一个梯度数组的每个位置，都对应原变量的一个位置。形状对不上，往往不是需要随手补一个转置，而是输入、输出和求导对象的约定没有说清楚。

1.2 先做一份可信的串行参考

配套程序用 NumPy 写出上面的前向和梯度。为避免指数溢出，先减去每行最大值。这不改变 Softmax，却能显著改善数值范围。

代码示例 显式计算一个训练步骤

```
import numpy as np

def loss_and_grad(x, labels, w, b):
    logits = x @ w + b
    # 每行减去自己的最大值，Softmax 不变而数值范围更安全。
```

```

shifted = logits - logits.max(axis=1, keepdims=True)
exp_logits = np.exp(shifted)
probs = exp_logits / exp_logits.sum(axis=1, keepdims=True)
loss = np.mean(np.log(exp_logits.sum(axis=1))
               - shifted[np.arange(len(x)), labels])
ds = probs.copy()
# 正确类别处减1，得到概率减独热标签。
ds[np.arange(len(x)), labels] -= 1
ds /= len(x)
# 矩阵梯度沿批次求和，偏置梯度也合并所有样本贡献。
return loss, x.T @ ds, ds.sum(axis=0)

```

这里的 `keepdims=True` 保留被归约的维度，使一个形状为 $B \times 1$ 的数组能沿类别维广播。`ds` 最初是概率矩阵；正确类别处减 1，再除以批大小，便得到对 logits 的梯度。最后两步分别完成矩阵乘法的反向和广播偏置的反向。

梯度下降更新为

$$W \leftarrow W - \eta \frac{\partial L}{\partial W}, \quad b \leftarrow b - \eta \frac{\partial L}{\partial b}.$$

负号表示沿着局部下降方向移动。学习率 η 决定移动距离，过大时不能期待损失持续下降。这个程序用四个可分样本检查梯度并训练，损失预期由约 0.693 降至 0.011。它不是用来比较模型效果，而是建立一个可以逐步核对的参考结果。

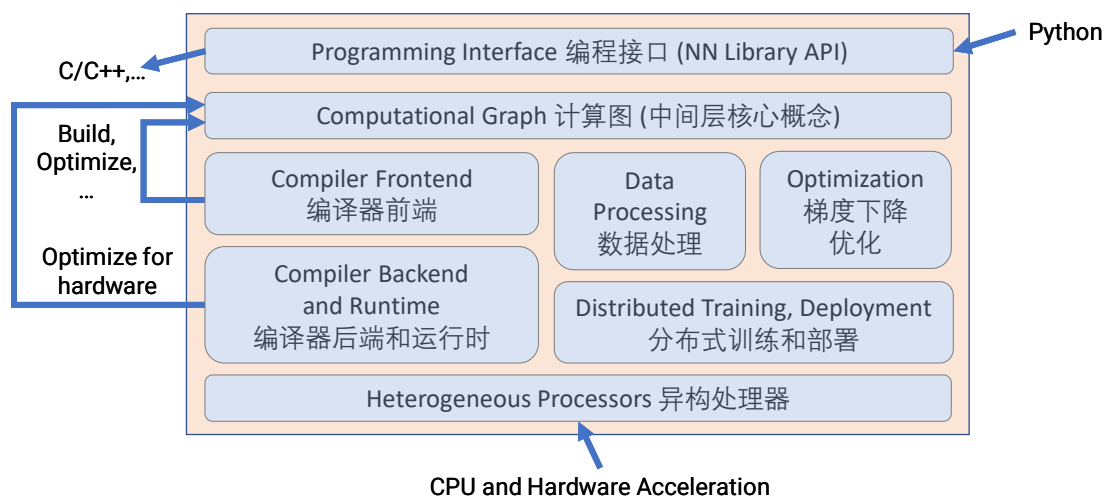


图 1.1 编程接口、计算图、编译与硬件之间的关系。后续章节将逐步解释这些层次如何协作。（课程第 09 讲，第 12 页）

1.3 同一计算有多种表示

从数学到硬件，需要经过几层表示，但它们必须描述一致的计算。

例如，数学上很短的 XW ，实现时可能要先搬运输入、切分矩阵块、安排几千个线程，再把结果交给下一层。反过来，某个实现虽然运行很快，若修改了归约语义、广播规则或数值精度，也不能直接当作原计算的等价替换。

层次	关心什么	本例中的问题
数学表达	函数、形状和导数	XW 的梯度为什么含有转置
计算图	数据依赖与可复用结果	Softmax 输出是否需要保存供反向使用
算子实现	循环、索引和归约	一条线程计算一个元素还是一小块矩阵
运行时	内存、设备与执行顺序	参数在哪张卡上，何时可以读取结果
系统调度	数据与通信的重叠	下一批数据是否赶得上 GPU

表 1.1 不同层次不是相互替代的解释，而是同一计算的不同视角。

工程实践 每次只把一个未知因素引入程序

先确认串行参考的结果，再实现 GPU 算子；先检查单个算子的梯度，再组合计算图；先让单设备训练正确，再引入多设备通信。这样遇到差异时，能够定位是哪一层的约定发生了变化。

第 2 章 CUDA 从零开始：让一个线程知道自己该做什么

CUDA 是围绕 GPU 执行模型扩展的编程接口。[2] 本章不假设读者见过 CUDA 语法。我们从一个 ReLU 算子开始，把声明、启动、索引、内存复制和错误检查逐一拆开。目标是让每个符号都对应一个实际动作，而不是记住一段能运行的模板。

2.1 Host 与 Device：两边各做什么

通常把 CPU 一侧称为 host，把 GPU 一侧称为 device。CPU 负责准备数据、管理资源和提交工作；GPU 按提交的配置启动许多线程，执行核函数。主机代码与设备代码可以写在同一个 .cu 文件里，但它们并不因此共享所有执行能力。

先使用最明确的内存模型：主机数组位于主机内存，设备数组由 CUDA 在设备上分配。CPU 变量 `d_x` 保存一个设备地址，但 CPU 不应像访问普通数组那样直接解引用它。指针值在哪里保存，与它指向的数据在哪里，是两个问题。

核心概念 核函数描述一个线程的工作

核函数 (kernel) 是一段设备代码。一次启动会让许多线程执行这段代码，每个线程有自己的索引和局部状态。程序员不必为每条线程写一份函数，而是用索引让同一份代码处理不同的数据。

对于 ReLU,

$$y_i = \max(x_i, 0).$$

每个输出只依赖一个输入，因此不同 i 可以独立计算。这是很适合第一次尝试的并行任务：还不需线程交换中间结果，也没有多个线程争写同一个输出。

2.2 逐行读懂第一个核函数

代码示例 每条线程处理一个数组元素

```
__global__ void relu(const float* x, float* y, int n) {  
    // 把块编号和块内线程编号合成全局元素编号。  
    int i = blockIdx.x * blockDim.x + threadIdx.x;  
    // 最后一块可能多启动线程，只允许合法位置写出。  
    if (i < n) y[i] = x[i] > 0.0f ? x[i] : 0.0f;  
}
```

先读函数头。`__global__` 是 CUDA 的函数限定符，表示这是可通过核函数启动语法调用的设备函数。这个例子从 CPU 启动它；基础接口使用 `void` 返回类型，结果通过 `y` 指向的设备数组写回。

`const float* x` 表示此函数通过 `x` 只读输入。它没有声明数据放在哪里；本例在调用前已经分配设备内存，并把设备地址作为参数传入。`float* y` 则允许写入输出。`int n` 是元素个数，以参数值传给每个线程。

函数体中唯一陌生的算式是

$$i = \text{blockIdx.x} \cdot \text{blockDim.x} + \text{threadIdx.x}.$$

它把两级编号合成一个全局编号：前面有多少个块，每块有多少线程，再加上自己在当前块中的位置。三个名字是 CUDA 提供的内建变量，不是需要在代码里自行赋值的全局变量。

名字	含义	一维启动中的取值
<code>threadIdx.x</code>	当前线程在块内的编号	$0, \dots, T - 1$
<code>blockIdx.x</code>	当前块在网格中的编号	$0, \dots, K - 1$
<code>blockDim.x</code>	每块的线程数量	T
<code>gridDim.x</code>	网格中的块数量	K

表 2.1 编号从 0 开始，数量不是最后一个编号。

最后的 `if (i < n)` 用来排除多启动的线程。线程存在，不代表它一定有一个有效数据元素可以处理。边界检查属于算法正确性的一部分。

为什么输入有 `const`，输出没有

`const` 帮助表达读写契约，但不会自动解决并行冲突。如果两个线程仍写入同一个 `y[i]`，输出指针的类型不会阻止它。我们需要进一步说明：每个合法 `i` 恰好由一条线程得到，因此每个输出恰好有一个写入者。

2.3 尖括号里的四个位置

启动核函数时使用

```
relu<<<blocks, threads>>>(d_x, d_y, n);
```

它与普通函数调用的区别在于，函数名后面先出现一组启动配置，再出现函数的实际参数。这里的 `blocks` 是网格中的块数，`threads` 是每块线程数。它们不是核函数参数，不能在函数体中按原来的主机变量名直接使用。

更完整的启动形式是：

```
kernel<<<grid, block, shared_bytes, stream>>>(arguments...);
```

前两个位置规定线程组织；第三个位置规定**每个块**额外需要多少字节的动态共享内存；第四个位置指定把任务提交到哪个流。省略后两项时，动态共享内存大小默认为 0，任务提交到默认流。流的顺序规则将在后面解释。

运行追踪 用八条线程检查索引

若启动 `<<<2,4>>>`，第 0 块的线程编号为 0,1,2,3，全局编号也为 0,1,2,3。第 1 块仍有块内编号 0,1,2,3，但全局编号变成 4,5,6,7。`threadIdx.x` 会在新块中从 0 重新计数，不能单独用它处理整个大数组。

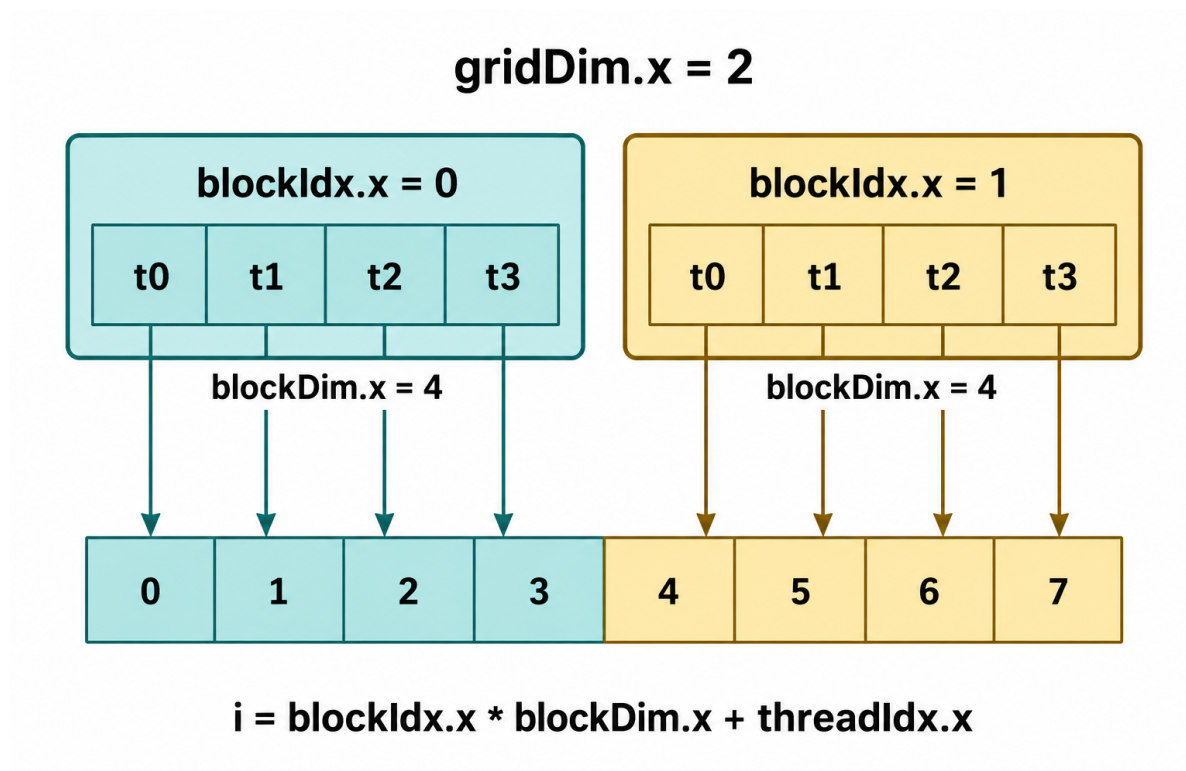


图 2.1 用每块四条线程的小例子理解全局编号。这里的块大小仅用于展示索引关系，不表示实际 GPU 的 warp 大小。

线程数与数据量不必相等

假设有 $n = 1003$ 个元素，每块启动 $T = 256$ 条线程。块数应取

$$K = \left\lceil \frac{n}{T} \right\rceil = \frac{n + T - 1}{T} \quad (\text{右侧采用非负整数除法}).$$

于是启动 4 块，共 1024 条线程。前 1003 条处理数据，其余 21 条通过边界检查跳过写入。不能向下取整，否则最后一个不完整块中的元素会遗漏。

这段加法形式要求 $n + T - 1$ 不溢出。处理极大数组时应使用适当宽度的类型，也可以写成 $n / T + (n \% T != 0)$ 。当 $n = 0$ 时，主机端直接返回，不启动零个块的 kernel。

易错点 不要把任意长度都塞进一个块

`<<<1,n>>>` 只适合 n 不超过设备及核函数约束的情形。许多设备允许一个块至多 1024 条线程，但程序应查询设备限制，不能把该数字当成所有资源限制的全部。即使线程数合法，寄存器和共享内存需求也可能限制启动或并发程度。

2.4 从主机数组到设备结果

完整的数据路径有五步：在两边准备存储，把输入复制到设备，启动 kernel，确认结果已经可用，把结果复制回来。配套程序先生成 1003 个包含正负数的输入，然后执行：

代码示例 主机端的内存与执行流程

```
float* d_x = nullptr;
float* d_y = nullptr;
// 把新设备地址写入 d_x; bytes 是字节数，不是元素个数。
CUDA_CHECK(cudaMalloc(&d_x, bytes));
CUDA_CHECK(cudaMalloc(&d_y, bytes));
CUDA_CHECK(cudaMemcpy(d_x, h_x.data(), bytes, cudaMemcpyHostToDevice));
const int threads = 256;
const int blocks = (n + threads - 1) / threads;
// 尖括号是启动配置；圆括号里的参数才进入核函数。
relu<<<blocks, threads>>>(d_x, d_y, n);
CUDA_CHECK(cudaGetLastError());
// 等设备工作完成后再继续检查与复制结果。
CUDA_CHECK(cudaDeviceSynchronize());
CUDA_CHECK(cudaMemcpy(h_y.data(), d_y, bytes, cudaMemcpyDeviceToHost));
CUDA_CHECK(cudaFree(d_y));
CUDA_CHECK(cudaFree(d_x));
```

cudaMalloc 为什么接收指针的地址

变量 `d_x` 一开始为空。调用 `cudaMalloc(&d_x, bytes)` 后，运行时要把新分配区域的地址写进 `d_x`，因此需要传入这个指针变量的地址。这里分配的是 `bytes` 个字节，而不是 `bytes` 个浮点数：

$$\text{bytes} = n \cdot \text{sizeof}(\text{float}).$$

CUDA C++ 提供相应重载支持这种指针调用。不要把“二级指针”理解成 GPU 上还需要再分配一个指针数组。

cudaMemcpy 的方向不能靠变量名猜

`cudaMemcpy(dst, src, bytes, direction)` 的第一个参数是目标，第二个参数是来源。主机到设备使用 `cudaMemcpyHostToDevice`，设备到主机使用 `cudaMemcpyDeviceToHost`。拷贝量仍然以字节计算。`h_` 与 `d_` 只是帮助阅读的命名习惯，没有语言层面的保护作用。

设备输入和输出在逻辑上是不同数组。虽然某些逐元素算子可以原地运行，但不能据此认为任何算子的输入输出都能别名。邻域操作和矩阵乘法尤其需要重新分析。

CUDA_CHECK 是普通的错误检查封装

示例用一个小封装检查运行时调用的返回值：`cudaError_t` 是否等于 `cudaSuccess`，失败时输出错误并结束示例。这个名字不是 CUDA 关键字，具体定义在配套的 `common.cuh` 中。

```
// 等待设备，并读取可能在异步执行期间发生的错误。
cudaError_t error = cudaDeviceSynchronize();
```

```
if (error != cudaSuccess) {  
    std::fprintf(stderr, "%s\n", cudaGetErrorString(error));  
    std::exit(EXIT_FAILURE);  
}
```

实际库通常把错误转成异常或状态，并按拥有关系释放资源。后续用 `DeviceBuffer` 管理设备数组，避免每个案例重复分配与释放代码。

2.5 编译、验证与最先排查的错误

在支持 CUDA 的环境中，使用 `nvcc` 编译 `.cu` 文件。它协调主机代码和设备代码的编译，不能简单用一个普通 C++ 编译命令替代。

```
nvcc -std=c++17 -O2 examples/cuda/01_relu.cu -o relu  
./relu
```

示例会把每个输出与 CPU 参考逐项比较。测试用 1003 而不是 1024 个元素，是为了主动覆盖尾部不足一块的情形。更完整的测试还应包括一个元素、整块长度、跨块长度，以及输入输出的接口约束。

遇到错误时，可以按依赖关系向前检查：设备指针是否有效，字节数是否正确，启动配置是否合法，索引是否越界，CPU 是否过早使用尚未完成的结果。设备端的 `printf` 有时能帮助观察执行，但它会扰动性能与调度，输出顺序也不能用来证明正常执行时的线程顺序。

2.6 异步执行：启动返回不等于计算完成

CPU 提交 kernel 后，通常可以继续执行主机代码。此时 GPU 可能还没有开始，也可能正在运行。因此，刚返回的启动语句不能作为“所有结果已经算完”的证明。

`cudaGetLastError()` 检查并清除当前线程保存的 CUDA 错误状态。把它紧跟在启动后面，可以及时发现非法启动配置等问题。它通常不能保证一个尚未结束的 kernel 内部没有出错。

`cudaDeviceSynchronize()` 则等待此前提交到当前设备的相关工作完成，并可能报告执行期间发生的错误。教学代码把启动检查与同步检查分开，可以更容易定位问题。性能敏感的程序通常会把同步放到真正需要结果或诊断的边界，而不是每个 kernel 后都强制同步。

原理剖析 正确性依赖明确的依赖关系

如果 B 要读 A 写出的数据，就必须建立 A 完成写入之后 B 才能读取的顺序。它可以由同一流行的顺序、事件依赖或合适的同步操作提供，不能由“两个调用挨得很近”提供。

本章的同步复制路径足以让复制回来的主机数组可供 CPU 读取，但不要把所有名字含有 `Memcpy` 的接口都理解成相同的阻塞行为。同步、异步接口以及主机内存是否锁页，都会影响实际行为。

2.7 二维线程、展平索引与张量形状

矩阵天然有行列。可以用 `dim3` 表示二维启动配置：

```
dim3 block(16, 16); // 每块16乘16，共256条线程
```

```
dim3 grid((width + block.x - 1) / block.x,
          (height + block.y - 1) / block.y);
kernel<<<grid, block>>>(...);
```

dim3 没有指定的维度取 1。这个配置的每块线程数是 $16 \times 16 = 256$ ，不是 16。核函数中可计算

// x 方向对应列，y 方向对应行；边界由实际矩阵尺寸决定。

```
int col = blockIdx.x * blockDim.x + threadIdx.x;
int row = blockIdx.y * blockDim.y + threadIdx.y;
if (row < height && col < width) {
    int offset = row * width + col;
    // access row-major element (row, col)
}
```

为什么通常把 x 方向对应列？对行优先矩阵，列编号加 1 会访问相邻元素。让相邻线程访问相邻地址，有利于合并访存。它是一种实现选择，不是二维线程必须遵守的矩阵语义。

运行追踪 形状为 3×5 的行优先矩阵

元素 (2,3) 前面有两整行，共 $2 \times 5 = 10$ 个元素，再向右移动 3 个位置，因此偏移为 13。线程组织只负责产生 (2,3)；真正的数组地址仍由布局和步长决定。

对一般张量，元素偏移应按步长计算：

$$\text{offset}(i_0, \dots, i_{d-1}) = o + \sum_{k=0}^{d-1} i_k s_k.$$

o 为存储偏移， s_k 为以元素计的步长。形状为 (H, W) 的连续行优先矩阵步长是 $(W, 1)$ ；转置视图可以改成形状 (W, H) 、步长 $(1, W)$ ，而不立即搬动数据。形状相同不意味着布局相同，得到一个转置视图也不意味着存储已经转置。

2.8 网格跨步循环：每条线程可以做不止一个元素

一次启动的线程总数不必覆盖全部数据。令总线程数为 S ，初始全局编号为 i_0 ，一条线程可以处理

$$i_0, \quad i_0 + S, \quad i_0 + 2S, \quad \dots$$

直到超出数组范围。这就是 grid-stride loop。

代码示例 把索引生成与数据规模分开

```
// 普通设备函数只在当前线程内调用，不会再启动一组线程。
__device__ float positive(float x) { return x > 0.0f ? x : 0.0f; }
__global__ void relu_stride(const float* x, float* y, size_t n) {
    // 先转为足够宽的整数，再计算乘积，避免窄类型先溢出。
    size_t first = size_t(blockIdx.x) * blockDim.x + threadIdx.x;
    size_t stride = size_t(blockDim.x) * gridDim.x;
    // 每次跨过整个网格的线程数，使线程反复处理互不重叠的位置。
    for (size_t i = first; i < n; i += stride)
        y[i] = positive(x[i]);
}
```

`__device__` 声明普通的设备函数。这里每条 GPU 线程通过普通函数调用语法调用 `positive`；它不会额外启动一组线程。`__host__` 则用于主机函数，普通未加限定的函数通常是主机函数。需要同一函数分别编译成两种可调用版本时，可以使用 `__host__ __device__`，但函数体必须分别满足目标执行环境的限制。

索引计算先把 `blockIdx.x` 转成 `size_t`，再做乘法，避免先在较窄整数类型中乘完才转换。任意合法位置 i 都能唯一写成 $i = qS + r$ ，其中 $0 \leq r < S$ 。编号为 r 的线程会在第 q 轮处理它，因此没有遗漏和重复。

第 3 章 线程如何协作：执行、内存与同步

ReLU 中每个输出只有一个写入者，线程之间几乎不需要合作。归约、卷积和矩阵乘法则不同：一个输出可能依赖许多输入，许多线程也可能复用同一批数据。此时，只知道全局线程编号已经不够，还要说明中间结果存在哪里，以及何时允许其他线程读取。

3.1 Block、Warp 与 SM 不是同一个东西

block 是程序员选择的合作范围，SM (streaming multiprocessor) 是 GPU 上执行线程的硬件单元，warp 则是硬件组织线程执行的基本分组。它们属于不同层次。

在本课讨论的常规 CUDA 执行模型中，一个 block 被安排到一个 SM 上，不能把同一 block 的线程任意拆到不同 SM。一个 SM 可以同时驻留多个 block，前提是寄存器、共享内存和线程等资源足够。一个网格的 block 往往多于设备一次能驻留的数量，因此需要分批安排。图中的算术核心只用于示意执行位置；线程并非始终独占一个固定核心。

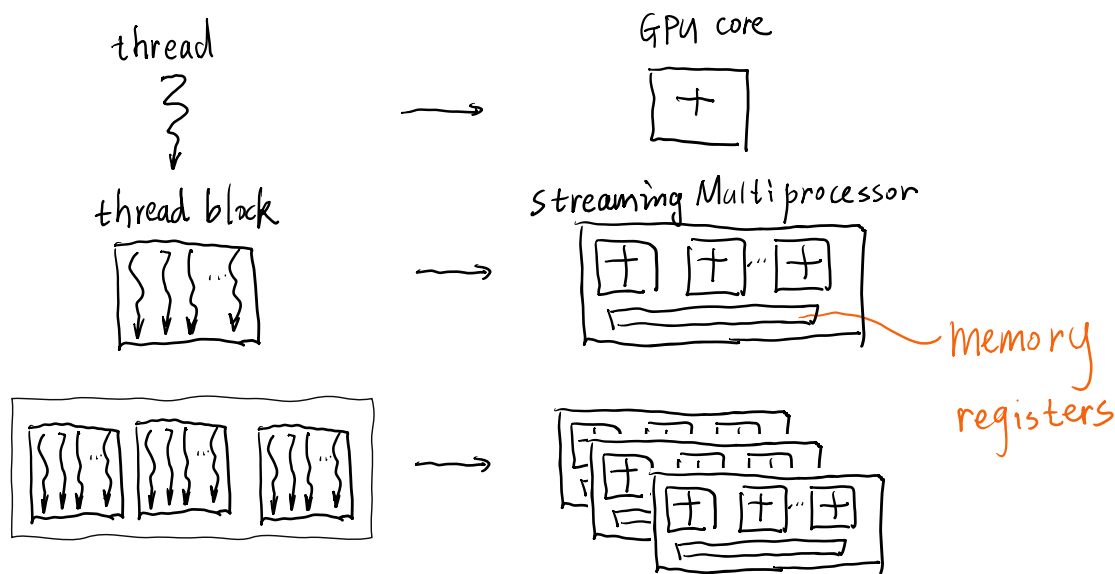


图 3.1 程序定义块，硬件把块安排到 SM。图是概念对应关系，不规定固定调度顺序或实际核心数量。
(课程第 03 讲，第 5 页)

同时存在，不表示每一步同时执行

一个 block 被安排到 SM，不意味着其中所有线程在每个时钟周期同时执行指令。线程按 warp 组织；本课所涉及的 NVIDIA GPU 中，一个 warp 包含 32 个线程位置。这种程序按单条线程描述、硬件按线程组组织执行的方式通常称为 SIMT (单指令、多线程)。SM 在可执行的 warp 之间调度，以隐藏访存等延迟。

二维、三维块也会先按线性编号组织 warp，x 维变化最快。例如线性编号为

$$t = \text{threadIdx.x} + \text{blockDim.x}(\text{threadIdx.y} + \text{blockDim.y} \cdot \text{threadIdx.z}).$$

若块形状为 (16,16)，一个 warp 的前 16 个位置来自第一行，后 16 个位置来自第二行。不能仅凭代码里有二维坐标，就认为每行恰好是一个 warp。

分支发散改变有效工作的比例

一个 warp 中的线程若走不同分支，硬件需要分别处理相应执行路径，并使不属于该路径的线程暂时不参与这些指令。若同一个 warp 中只有少数线程需要很长的工作，其余线程位置可能大部分时间都没有做有效计算。

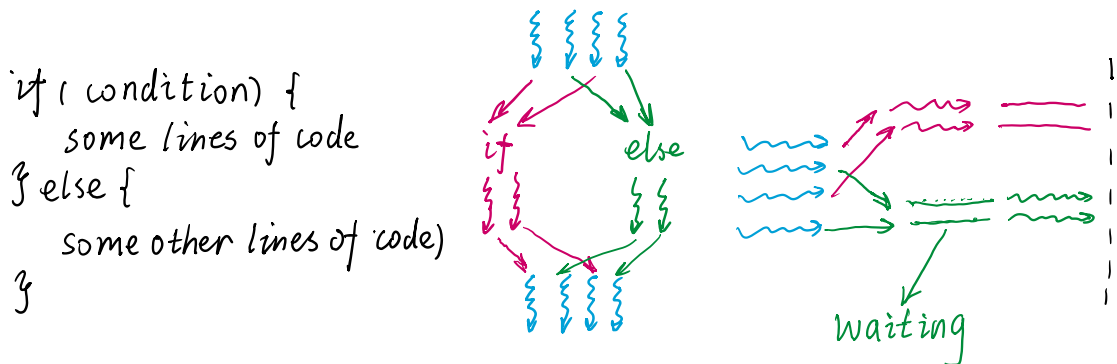


图 3.2 发散发生在同一 warp 的不同执行路径之间。图中各色路径只表达分组执行的直觉，不是线程之间的同步保证。（课程第 03 讲，第 11 页）

分支并非越少越好。边界检查不能为了避免发散而删掉。要比较的是正确实现之间的代价：发散发生的频率、各路径的工作量，以及是否值得为重新分组额外搬动数据。现代 GPU 的独立线程调度也不允许我们把“同属一个 warp”当成无需同步的数据交换保证。

3.2 内存名称首先描述可见范围和访问方式

存储	通常由谁访问	容易误解的地方
寄存器	单个线程	数量有限，需求太多会影响驻留或导致溢出
local memory	单个线程的地址空间	名叫 local，不表示一定在片上或访问很快
shared memory	同一块的线程	生命周期通常随块结束，需显式处理协作顺序
global memory	合法访问它的设备线程	地址空间可见不表示不同线程已建立执行顺序
constant memory	设备线程只读访问	缓存收益与访问模式有关，不是任意只读数据都同样快

表 3.1 逻辑作用域与物理存放位置应分别理解。

核函数中的普通局部变量如 `float sum`，通常尽量放入寄存器。但编译器可能因寄存器压力、地址获取或数组访问等原因，把部分线程私有数据放入 local memory。它在逻辑上是线程私有的，却可能依赖设备内存层次。因此，不能看到 C++ 局部变量就认定其读写一定没有设备内存流量。

共享内存由整个块共同使用

静态共享数组可写成：

```
__shared__ float tile[128];
```

一个块拥有一份 `tile`，不是每条线程各有一份。不同块拥有各自的实例，即使代码和数组名字相同，也不是同一段存储。

若大小在启动时确定，可以写：

```
extern __shared__ float tile[];
kernel<<<blocks, threads, shared_bytes>>>(...);
```

第三个启动参数给出每块所需的字节数。这里的 `extern` 是 CUDA 动态共享内存声明的一部分，并不表示从另一份普通 C++ 源文件寻找一个全局数组。若一块内需要多段动态共享数组，通常从同一段区域切分，并处理偏移与对齐。

3.3 合并访存：看一组线程访问了哪些地址

GPU 不会把每条线程的一次访问都简单变成一次独立的完整内存事务。相邻线程访问相邻且适当对齐的地址时，硬件能用较少事务满足更多请求。这就是合并访存（coalescing）的核心直觉。

- The CUDA program is efficient when threads read / write contiguous memory locations

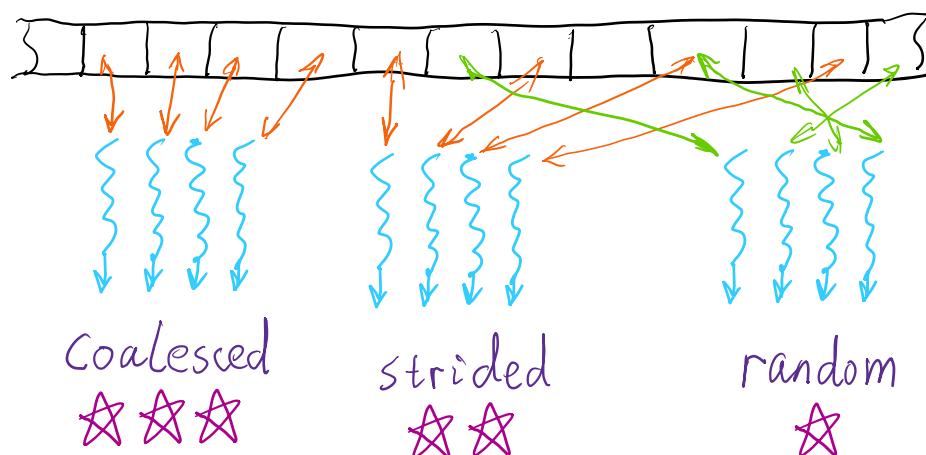


图 3.3 比较连续、跨步和分散访问。应考察同一条访存指令中一组线程的地址集合，而不只是单个线程自己的循环顺序。（课程第 02 讲，第 31 页）

例如，一个 warp 的线程分别读 `x[t]`，地址连续；分别读 `x[32*t]` 时，地址分散到许多位置。二者都只写了一次下标访问，但对内存系统提出的请求很不一样。

`x[t+1]` 仍然是连续访问，只是对齐边界可能导致更多事务。不能把它与随机访问等同。具体事务大小与规则应结合目标架构、访问宽度和对齐分析。

共享内存也可能产生冲突

共享内存通常组织为多个 bank。同一条指令中，多个线程若请求同一 bank 的不同地址，可能需要分批服务，形成 bank conflict。多个线程读取同一个地址则可能由广播处理，不应笼统判断为同样的冲突。

对常见的 32 位元素与 32-bank 分析，读取一行 `tile[row][t]` 通常分散到不同 bank；读取 `tile[t][col]` 时，行宽若正好是 32，多个地址可能落在同一 bank。把共享 tile 声明成 `tile[32][33]`，常能打破这种步长关系。这个加一技巧依赖布局 and 元素宽度，不能脱离地址计算机械使用。

3.4 先写后读：屏障到底解决什么

考虑

$$y_i = x_i + x_{i+1},$$

最末端超出输入的值按 0 处理。如果直接原地更新 `x[i]`，相邻线程就可能一边读旧值，一边写新值。即使每个位置只有一个写入者，也仍然可能出现读写冲突。

最清楚的实现先使用不同的输入、输出数组。下面把每块需要的输入放入共享内存，再计算相邻和。块末线程还需要下一块开头的的一个输入，因此多加载一个 halo 元素。

代码示例 每块多存一个邻居元素

```
__global__ void shift_sum(const float* x, float* y, int n) {
    // 这段共享数组由启动配置的第三项指定每块字节数。
    extern __shared__ float tile[];
    int t = threadIdx.x;
    int i = blockIdx.x * blockDim.x + t;
    // 无效尾部填0，但这些线程仍要参加后面的屏障。
    tile[t] = i < n ? x[i] : 0.0f;
    if (t == 0) {
        // 块末输出还需要右邻居，所以额外装入一个 halo 元素。
        int halo = (blockIdx.x + 1) * blockDim.x;
        tile[blockDim.x] = halo < n ? x[halo] : 0.0f;
    }
    // 等待本块完成所有加载，才读取邻居线程写出的共享位置。
    __syncthreads();
    if (i < n) y[i] = tile[t] + tile[t + 1];
}
```

启动时必须给每块的动态共享数组分配 $T + 1$ 个 float 的空间。沿用上一章的设备指针，可以写：

```
int threads = 128;
int blocks = (n+threads-1)/threads;
// 每块多申请一个halo位置；所有指针都指向已分配的设备数组。
shift_sum<<<blocks, threads, (threads+1)*sizeof(float)>>>>(d_x, d_y, n);
```

其中多出的一个位置就是 halo。第三项写的是每块字节数，并不是整个网格的总字节数。

这里的步骤不能随意调换。每条线程先写自己负责的 `tile[t]`；第 0 条线程额外加载 `tile[blockDim.x]`；`__syncthreads()` 等待块内协作线程完成这个阶段，并提供相应的内存顺序保证。随后才能读取相邻线程写出的 `tile[t+1]`。

运行追踪 用块大小 4 看一次边界

若某块处理输入位置 4, 5, 6, 7，共享数组需存放 x_4, x_5, x_6, x_7, x_8 。块内线程 3 计算 y_7 时读取共享位置 3 与 4。它依赖的是加载进来的 x_8 ，不需要等待下一个块去计算 y_8 。这就是 halo 复制把跨块依赖变成块内读取的意义。

对于尾部块，即使某条线程没有合法输出，也仍负责将其共享位置填入安全值并到达屏障。不要在屏障前简单写 `if (i >= n) return`；再假设剩余代码对任何协作算法都成立。更容易验证的组织方式是：**加载时做边界处理，协作线程一致经过屏障，最后再决定是否写出。**

易错点 屏障不是给任意竞争“补一行”

把屏障放在读写冲突已经发生之后，不能撤销错误。只让部分线程进入含屏障的条件分支，也不能形成正确的全块协作。先标清每个共享位置的生产者、消费者和阶段，再决定屏障应放在哪里。

块内屏障没有替你同步整个网格

`__syncthreads()` 的范围是当前块。不同块不能靠它互相等待。让当前驻留的块自旋等待尚未得到调度的块，还可能让程序无法前进。

普通算法可把跨块依赖拆成多个 kernel，并放入同一流：第一个 kernel 写完所有部分结果，后一个 kernel 再消费它们。专门的协作启动和更高级同步机制有额外前提，不是给任意启动补一个“全网格屏障”就能成立。

3.5 原子操作：对同一位置的修改怎样合并

普通的

```
counter[index] += 1;
```

在逻辑上包含读取、相加和写回。两个线程都读到 5，然后各自写入 6，便丢失了一次增量。对不同位置独立更新不需要因此使用原子操作；真正的问题是多个更新是否可能重合。

`atomicAdd(&counter[index], 1)` 把对这个位置的读改写作为原子操作处理。其他合法的原子更新不会插进这次操作的中间。但这不意味着整个函数成为原子事务，也不表示不相关数据的发布与读取已经同步。

原子性、顺序性与可重复性不同

整数计数在不溢出等前提下，可以得到确定的最终总数。浮点求和则还受到加法顺序影响。例如 $(a + b) + c$ 与 $a + (b + c)$ 可能因舍入不同而不相等。因此，正确使用浮点原子加可以避免丢失更新，却不一定得到每次逐位相同的结果。

若所有线程都争用同一个计数器，原子操作可能形成严重串行瓶颈。常见改进是先在块内合并局部结果，再用较少的全局更新完成汇总。下一章的归约与直方图会展示这种组织方法。

3.6 从通信模式判断反向计算

很多算子可以用“从哪里读、向哪里写”理解。map 中一个输出只读取对应输入；gather 根据索引收集输入；scatter 把结果分发到指定位置；stencil 从固定邻域读取；transpose 则重新排列位置。

Gather: Many-to-One

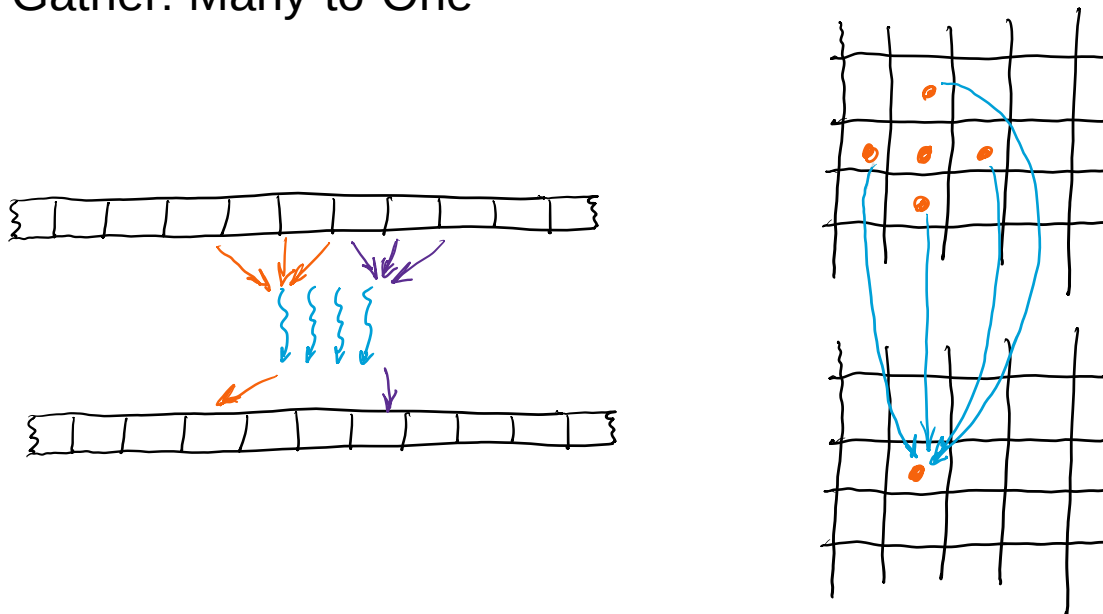


图 3.4 Gather 按指定关系收集数据。一个输入可能被多次读取；只读复用本身不构成写冲突。（课程第 03 讲，第 29 页）

例如 $y_j = x_{a_j}$ ，给定输出梯度 $\bar{y}_j$ 后，

$$\bar{x}_i = \sum_{j:a_j=i} \bar{y}_j.$$

索引重复时，反向必须把多条路径的贡献相加。这一条式子同时解释了自动微分中的梯度累积，以及 CUDA 实现为什么可能需要原子加、分段归约或重新组织写入者。

3.7 流、事件与测量：让先后关系可见

流（stream）是一条有序提交队列。一个流中的后续依赖操作按相应语义接在先前操作之后；不同流之间不能仅凭主机提交顺序假定已建立依赖。它们可能并发，也可能因为硬件资源等原因仍然串行。

```
cudaEventRecord(ready, producer_stream);
cudaStreamWaitEvent(consumer_stream, ready, 0);
// 后续消费核函数提交到 consumer_stream，就会先等 ready 事件。
```

事件把两个流之间需要的先后关系表达出来。默认流存在传统默认流与每线程默认流等模式，隐式同步行为有所不同。学习和排查依赖时，显式命名流与事件通常更清楚。

Scatter: One-to-Many

- Scatter is the reverse of Gather

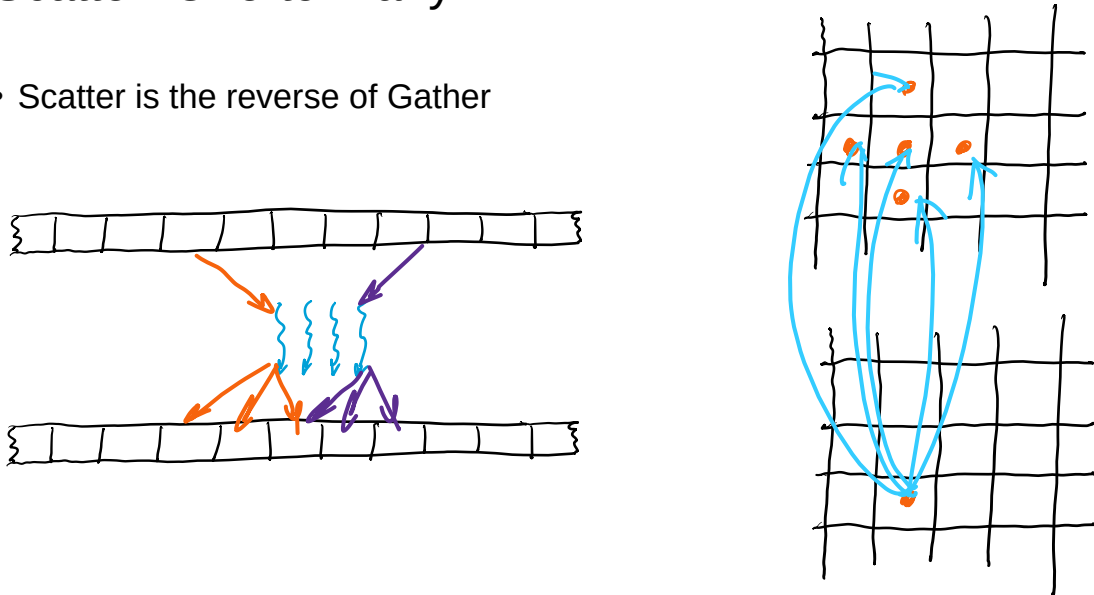


图 3.5 把收集关系反过来时，多个贡献可能落到同一位置。反向传播中常需要累加，而不能直接覆盖。
(课程第 03 讲，第 30 页)

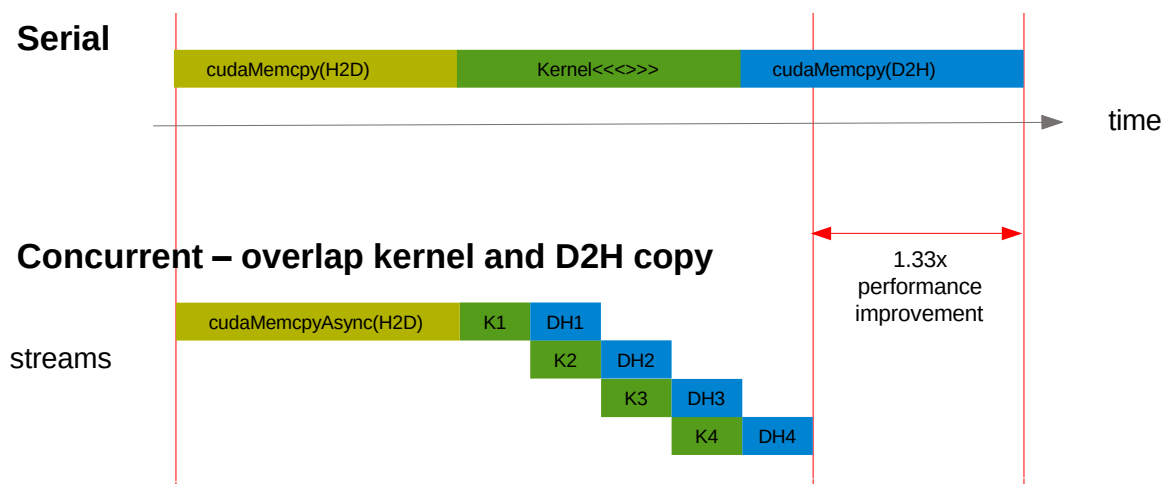


图 3.6 满足资源与依赖条件时，可以把数据复制与计算重叠。图中的速度比只是该示意例的结果，不是使用多流的固定收益。(课程第 05 讲，第 32 页)

异步主机到设备复制若要可靠地获得重叠机会，通常需要合适的锁页主机内存，并要求硬件支持以及缓冲区寿命正确。`cudaMemcpyAsync` 返回后，不能立刻修改还在传输的输入缓冲区，或提前释放尚未使用完的数组。

GPU 时间应该测量哪个区间

```
// 在同一流上记录区间两端，等待结束事件后读取耗时。
cudaEvent_t start, stop;
CUDA_CHECK(cudaEventCreate(&start));
CUDA_CHECK(cudaEventCreate(&stop));
CUDA_CHECK(cudaEventRecord(start, stream));
kernel<<<grid, block, 0, stream>>>(...);
CUDA_CHECK(cudaEventRecord(stop, stream));
CUDA_CHECK(cudaEventSynchronize(stop));
float ms = 0;
CUDA_CHECK(cudaEventElapsedTime(&ms, start, stop));
CUDA_CHECK(cudaEventDestroy(stop));
CUDA_CHECK(cudaEventDestroy(start));
```

把 CPU 计时器仅包在启动语句外面，常常主要测到提交成本。若关心 GPU 队列中的计算区间，可以使用同一流上的事件；若关心用户等待的端到端耗时，则应在主机计时中纳入必要同步、数据准备和传输。两种指标回答不同问题。

测量前先预热，进行多次重复，并把误差和输入规模一起记录。多个流并行时，一对只覆盖某一流的事件，也未必代表整个系统完成所有任务的时间。

第 4 章 并行基本算法：把依赖关系改写成可协作的步骤

把一个串行循环的迭代分给不同线程，并不一定得到正确或高效的并行算法。求和、前缀和和排序都有跨元素依赖。需要改变的是计算的组织方式：哪些结果能同时得到，哪些阶段必须等待，以及怎样控制总工作量。

本章把归约、扫描与排序连起来。它们不仅是独立算法，还会出现在归一化、稀疏计算、数据筛选、注意力与分布式通信中。

4.1 工作量与关键路径要分开看

设 W 为总工作量， S 为最长依赖链上的步骤数，也称 span。串行求和每读一个数都更新同一个累加器，因此 $W = O(n)$ 、 $S = O(n)$ 。树形求和仍需大约 $n - 1$ 次加法，但每一层可以并行，因而 $S = O(\log n)$ 。

在有 p 个理想执行单元的简化模型里，

$$T_p \geq \max(W/p, S).$$

它给出两个不同限制：即使没有长依赖链，也要花时间完成所有工作；即使处理器很多，也不能跳过前后相依的步骤。真实 GPU 还要支付访存、同步、启动和资源调度等成本。

原理剖析 并行度不是线程数

W/S 可用来衡量算法中平均可利用的并行性。只增加线程数量，既不会自动缩短依赖链，也不会保证内存系统能更快供给数据。好的并行算法应同时分析工作量、关键路径和数据移动。

4.2 归约：把很多元素变成一个结果

对求和归约，将数组分成小组，各组先计算部分和，再归约这些部分和。若每组大小为 B ，每轮输出长度大约缩小为原来的 $1/B$ 。

最容易验证的版本让每个块处理至多 B 个元素，并使用共享内存保存中间结果。这里要求 B 为 2 的幂，启动时每块恰好有 B 条线程。

代码示例 一个块产生一个部分和

```
template<int B>
__global__ void reduce_blocks(const float* x, float* partial, int n) {
    __shared__ float s[B];
    int t = threadIdx.x;
    int i = blockIdx.x * B + t;
    // 补入加法单位元0，使尾部也能使用同一棵归约树。
```

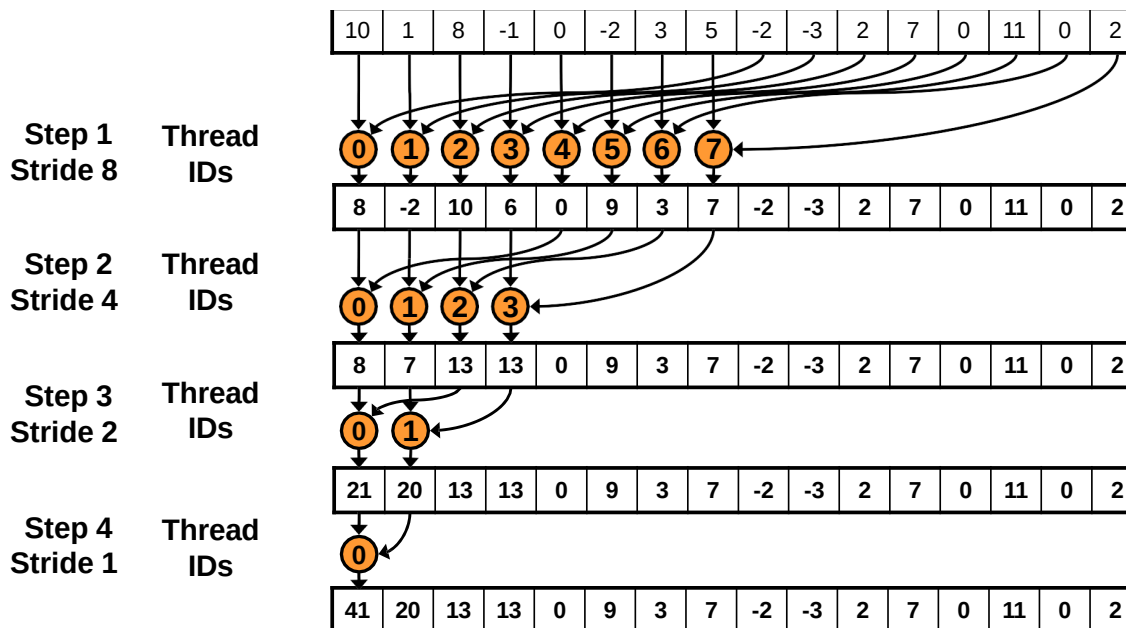


图 4.1 每一阶段让仍参与工作的线程合并两个部分结果，逐层缩小活动范围。示例采用顺序编号的活跃线程。（课程第 04 讲，第 9 页）

```

s[t] = i < n ? x[i] : 0.0f;
__syncthreads();
// 步长每轮减半，活动位置逐步缩到 s[0]。
for (int stride = B / 2; stride > 0; stride /= 2) {
    if (t < stride) s[t] += s[t + stride];
    __syncthreads();
}
// 每块只写出一份部分和，后续 kernel 再处理跨块合并。
if (t == 0) partial[blockIdx.x] = s[0];
}

```

加载后的第一次屏障保证所有 $s[t]$ 都已写好。接着令步长依次为 $B/2, B/4, \dots, 1$ 。某个阶段开始时，所需部分和已经由上一个阶段建立；本阶段结束时，屏障再保证它们可供下一阶段使用。

运行追踪 四个数怎样缩成一个数

初始共享数组为 $[2, 5, 1, 4]$ 。步长为 2 时，线程 0 得到 $2 + 1 = 3$ ，线程 1 得到 $5 + 4 = 9$ 。屏障后，步长为 1，线程 0 得到 $3 + 9 = 12$ 。其余线程虽然不再做加法，仍按同一阶段结构到达屏障。

尾部不足 B 个元素时，以加法单位元 0 填充。若改成最大值归约，应换成相应单位元或明确的边界处理，不能继续无条件填 0。例如所有有效输入都为负数时，填 0 会改变最大值。

跨块归约为什么通常再启动一次 kernel

第一轮每个块写一个独立的部分和，不需要争用单个全局结果。第二轮再读这些部分和。把两轮放入同一流，就能用 kernel 边界建立所需顺序。DeviceBuffer<float> 是本书的小型设备内

存拥有者，构造时分配，析构时释放，`data()` 返回设备指针。`upload` 和 `download` 包装两种复制方向，完整定义在附录。它没有替归约执行额外算法，只把重复的资源管理步骤集中起来。

代码示例 两份暂存区轮换，保持输入只读

```
float reduce_sum(const float* x, int n) {
    if (n == 0) return 0.0f;
    constexpr int B = 256; // power of two, launched with exactly B threads
    int capacity = (n + B - 1) / B;
    DeviceBuffer<float> a(capacity), b(capacity);
    const float* current = x;
    bool use_a = true;
    while (n > 1) {
        int blocks = (n + B - 1) / B;
        // 当前输入与本轮输出使用不同暂存区，防止覆盖尚未读完的数据。
        float* next = use_a ? a.data() : b.data();
        reduce_blocks<B><<<blocks, B>>>(current, next, n);
        CUDA_CHECK(cudaGetLastError());
        current = next;
        n = blocks;
        use_a = !use_a;
    }
    float result;
    CUDA_CHECK(cudaMemcpy(&result, current, sizeof(float),
                          cudaMemcpyDeviceToHost));
    return result;
}
```

`current` 总是指向上一轮已经产生的数据，`next` 指向另一份输出区。输入输出不能在缺乏证明时重叠，否则某个块写入新部分和时，可能覆盖其他块尚未读取的输入。这里还单独处理了空数组和一个元素：空和为 0，一个元素不需要执行合并轮次。

结合性与数值计算的边界

树形重组通常依赖操作的结合性。整数的数学加法满足它，固定宽度整数还要检查溢出规则；浮点加法则通常不严格结合。因此，并行求和与串行参考可能有小的舍入差异。

配套归约测试用可精确表示且不会超过相关精度范围的小整数值，方便检查索引与覆盖。对一般浮点输入，应使用合适精度的参考与误差判据，不能把“与某种串行顺序逐位相同”作为所有正确并行实现的定义。

warp 内的 shuffle：在线程之间交换寄存器值

某些归约可以在 warp 内通过 shuffle 交换各线程的寄存器值，减少共享内存访问。例如：

```
__global__ void warp_sums(const float* x, float* partial, int n) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    int lane = threadIdx.x % 32;
    // 所有完整 warp 的线程都参与；没有数据的线程用 0 补齐。
    float value = i < n ? x[i] : 0.0f;
```

```

for(int offset=16;offset>0;offset/=2)
    // 取更高 lane 的寄存器值，按16、8、4、2、1的距离归约。
    value+=__shfl_down_sync(0xffffffffu,value,offset);
// 只有每个 warp 的第0条线程持有完整和并写出结果。
if(lane==0)partial[i/32]=value;
}

```

这个版本要求块大小是 32 的倍数，所有线程都执行每一次 shuffle；尾部无数据的线程以 0 参与。常量 0xffffffffu 的 32 个置位比特表示完整 warp 的参与者。shuffle 调用的形式是：

```
__shfl_down_sync(mask, value, offset); // 从相应更高 lane 取得寄存器值
```

最后由 lane 0 写出本 warp 的和。[3]

mask 必须与真实参与线程一致。若提前让部分线程退出，就不能仍无条件使用完整 mask；读取未参与线程的值也不能获得有效结果。shuffle 的同步语义不等同于任意共享内存访问的屏障，需要时应另行使用符合参与范围的 __syncwarp(mask) 或块级同步。

4.3 直方图：把热点写入先分散

直方可表示为

$$h_k = \sum_{i=0}^{n-1} \mathbf{1}[x_i = k].$$

每条线程知道自己应增加哪个 bin，但很多线程可能选中同一个 bin。最直接的全局原子加实现正确性较容易，却可能让热点位置承受大量竞争。

一个折中方案是每块先建立局部直方图，然后将每个 bin 的局部计数合并到全局：

代码示例 先在共享内存计数，再合并

```

constexpr int K = 16; // 本例有16个bin，输入值必须位于0到15
__global__ void histogram(const int* x, int* bins, int n) {
    __shared__ int local[K];
    int t=threadIdx.x;
    // 合作清零本块直方图，各线程负责不同的 bin。
    for(int k=t;k<K;k+=blockDim.x) local[k]=0;
    __syncthreads();
    for(int i=blockIdx.x*blockDim.x+t;i<n;i+=blockDim.x*gridDim.x)
        // 多个输入可能落在同一 bin，局部计数也必须安全累加。
        atomicAdd(&local[x[i]],1); // input contract: 0 <= x[i] < K
    __syncthreads();
    for(int k=t;k<K;k+=blockDim.x)
        // 块内统计完成后，再用少量全局原子操作合并结果。
        atomicAdd(&bins[k],local[k]);
}

```

启动前必须把全局 bins 清零。第一次屏障保证局部计数器初始化完成，第二次屏障保证局部统计结束。最后每块只需至多 K 次全局合并，而不是每个输入元素都访问全局计数器。

局部原子操作仍然可能竞争，尤其当输入几乎全部落在同一 bin 时。优化是否值得，还要比较 bin 数量、共享内存需求、数据分布以及合并成本。测试应同时包含均匀分布和极端偏斜分布。

4.4 扫描：不仅求总和，还求每个位置前面的和

含当前位置的前缀和称为 inclusive scan：

$$s_i = \sum_{j=0}^i x_j.$$

不含当前位置的版本称为 exclusive scan：

$$e_i = \sum_{j=0}^{i-1} x_j, \quad e_0 = 0.$$

输入 $[3, 1, 4, 2]$ 的两种结果分别为 $[3, 4, 8, 10]$ 与 $[0, 3, 4, 8]$ 。差别只有一个位置，却决定了后面如何把 scan 用作输出地址分配。

Hillis–Steele：每一轮把已知范围翻倍

开始时，每个位置只知道自己。第一轮再纳入左边距离 1 的部分结果，第二轮纳入左边距离 2 的结果，随后是 4、8 等距离。第 k 轮后，一个位置已经合并了至多 2^{k+1} 个连续输入。

代码示例 用快照表达每一轮的并行语义

```
def inclusive_scan(x):
    out = list(x)
    stride = 1
    while stride < len(out):
        # 所有本轮更新读同一份上一轮快照，模拟并行阶段。
        old = out.copy()
        for i in range(stride, len(out)):
            out[i] = old[i - stride] + old[i]
        stride *= 2
    return out
```

这里 old 很重要。所有本轮计算都应读取上一轮的结果。若按从左到右的顺序原地写入并继续读取，就会把某些本轮结果提前混进去，变成另一个算法。

运行追踪 四个元素的两轮计算

距离为 1 时，由原始快照得到 $[1, 3, 5, 7]$ 。距离为 2 时，仍读取完整的上一轮：第三个位置变成 $5 + 1 = 6$ ，第四个位置变成 $7 + 3 = 10$ 。最终结果是 $[1, 3, 6, 10]$ 。

每一轮有 $O(n)$ 个位置参与，共 $O(\log n)$ 轮，所以工作量为 $O(n \log n)$ 。它缩短了依赖链，但付出了额外工作。CUDA 中可以使用两份共享数组轮换：

代码示例 双缓冲避免同一轮相互覆盖

```
constexpr int B = 256; // 与启动时每块线程数一致，且为2的幂
__global__ void scan_blocks(const int* x, int* y, int* totals, int n) {
    __shared__ int buffer[2][B];
    int t = threadIdx.x, i = blockIdx.x * B + t;
    // 两份共享数组分别保存上一轮和本轮，不能边覆盖边读取旧值。
```

```

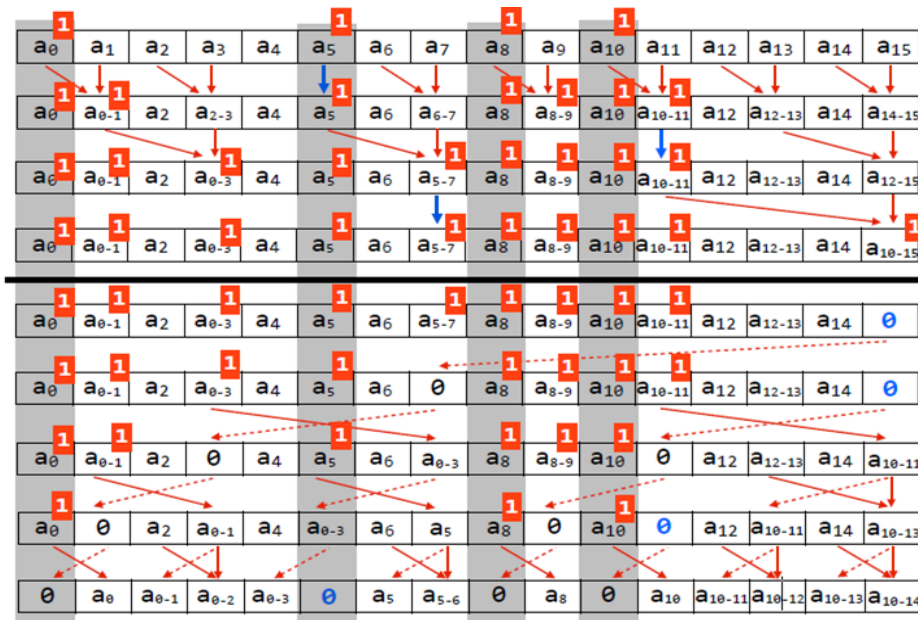
int* read = buffer[0];
int* write = buffer[1];
read[t] = i < n ? x[i] : 0;
// 初始加载完成后，才能开始第一轮邻居读取。
__syncthreads();
for (int offset = 1; offset < B; offset *= 2) {
    // 本轮所有线程只读取上一轮；没有左邻居时只保留自身前缀。
    write[t] = read[t] + (t >= offset ? read[t - offset] : 0);
    __syncthreads();
    // 本轮写入都已完成，再交换读写角色。
    int* old = read; read = write; write = old;
}
if (i < n) y[i] = read[t];
if (t == B-1) totals[blockIdx.x] = read[t];
}

```

每轮屏障放在写入之后，保证所有线程完成本轮读写，再共同切换读写角色。配套完整程序先求块内扫描和块总和，再扫描块总和，最后给第 b 块加上前 b 块的总量。块内结果与跨块偏移缺一不可。

Blelloch：为什么能把工作量降到线性

Blelloch 扫描分为上扫和下扫。上扫像归约树，计算各子树的总和；下扫把“本子树之前一共有多少”传给子树。根节点的前缀为 0，左子树继承父前缀，右子树得到父前缀加上左子树总和。



(CMU 15-418,
Spring 2012)

图 4.2 上扫保存区间总和，下扫传播进入每个区间前的前缀。图中根部从总和改为单位元，开始构造 exclusive scan。（课程第 04 讲，第 40 页）

一棵二叉树只有 $O(n)$ 个节点，每个节点在上下两遍中各做常数工作，故 $W = O(n)$ ；树高为 $O(\log n)$ ，故 $S = O(\log n)$ 。

代码示例 可运行的上扫与下扫参考

```
def exclusive_scan(x):
    if not x:
        return []
    n = 1 << (len(x) - 1).bit_length()
    a = list(x) + [0] * (n - len(x))
    stride = 1
    while stride < n:                                # up-sweep
        for end in range(2*stride-1, n, 2*stride):
            a[end] += a[end-stride]
        stride *= 2
    # 把根的总和替换为单位元，开始传播各子树之前的前缀。
    a[-1] = 0                                         # identity at root
    stride = n // 2
    while stride:
        for end in range(2*stride-1, n, 2*stride):
            # 先保存将被覆盖的左子树总和，再更新左右前缀。
            left_sum = a[end-stride]
            a[end-stride] = a[end]
            a[end] += left_sum
        stride //= 2
    return a[:len(x)]
```

代码把长度补到 2 的幂，并填入加法单位元。下扫中必须先保存 `left_sum`：它既要被父前缀覆盖，又要用于计算右子树前缀。这是另一个“先想清楚读写依赖”的具体例子。

4.5 筛选、分段与递推为什么都能用扫描

筛选：前面保留了多少元素，就是输出位置

给定保留标记 $f_i \in \{0, 1\}$ ，先做 exclusive scan：

$$p_i = \sum_{j < i} f_j.$$

若 $f_i = 1$ ，把 x_i 写到输出位置 p_i 。不同保留元素拥有严格递增的位置，因此不会争写；原始相对顺序也得以保留。

代码示例 用扫描生成互不重叠的输出位置

```
def compact(x, keep):
    flags = [int(v) for v in keep]
    # 某元素前面保留了多少个元素，就是它的输出下标。
    positions = exclusive_scan(flags)
    out = [None] * sum(flags)
    for i, flag in enumerate(flags):
        if flag:
            out[positions[i]] = x[i]
```

3	1	8	4	6	5	2	7
T	F	T	F	T	F	T	F
3	-	8	-	6	-	2	-
0	1	1	2	2	3	3	4
3	8	6	2				

图 4.3 标记、前缀地址与写出之间的关系。先解决“写到哪里”，再进行并行搬运。（课程第 04 讲，第 35 页）

```
return out
```

对于非空输入，输出长度可由 $p_{n-1} + f_{n-1}$ 得到；空输入单独返回空输出。这个边界比“总能读取最后一个元素”更准确。

分段扫描：让段首重置累计

若数据包含多个连续段，可以为每个位置附一个段首标记 h_i 。合并两个按顺序排列的聚合结果时，若右侧包含新的段首，就丢弃来自更早段的累计。对加法，可使用

$$(a, h) \diamond (b, k) = (k ? b : a + b, h \vee k).$$

其中标记表明该聚合区间是否出现过段首。这个二元操作可形成适合扫描的结合结构。段首位置的 exclusive 结果仍要按单位元重置，不能简单把跨段 inclusive 结果整体右移而不处理边界。

线性递推：扫描的对象可以是变换

考虑 $h_t = A_t h_{t-1} + b_t$ 。表面上它必须逐步计算，因为每一步都依赖上一步。换个表示：每步是一个仿射变换 (A_t, b_t) ，两步复合为

$$(A_2, b_2) \circ (A_1, b_1) = (A_2 A_1, A_2 b_1 + b_2).$$

函数复合满足结合律，因此可以对这些变换做前缀复合，再作用到初值 h_0 。复合通常不满足交换律，**可以改变括号，但不能改变时间顺序**。

这解释了某些状态空间模型为何能使用并行 scan。收益还取决于 A_t 的结构；若每次复合都需要昂贵的稠密矩阵乘法，就不能只看到较浅的依赖链而忽略增加的工作。

4.6 排序网络与基数排序

并行排序的关键是让比较或分配尽量按可预测的结构执行。奇偶交换排序容易并行，但工作量仍可能达到 $O(n^2)$ ；归并排序减少总工作，却要认真设计不同规模的并行归并。

排序网络把比较位置固定下来

比较器把一对输入输出为较小值与较大值。网络事先确定哪些位置在每层比较，不根据数据重新选择连接方式。同层互不冲突的比较器可以并行。

- A bitonic sorter is composed of several **half-cleaners**
- A **half-cleaner**: If the input is a bitonic sequence of 0's and 1's, then the output satisfies: both the top half and the bottom half are bitonic and at least one half is clean

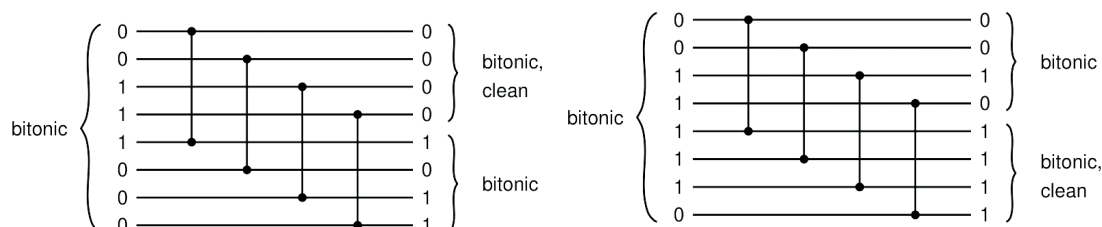


图 4.4 双调合并利用固定的比较连接，把已经具有结构的输入逐层整理。（课程第 05 讲，第 20 页）

双调序列先单调上升再下降，或经循环移动后具有这个形式。把两个有序序列中的一个反向，再连接起来，可以构造双调序列。对长度 $n = 2^k$ 的双调输入，递归的半清理与合并需要 $O(\log n)$ 层；从任意输入开始完成整个双调排序，则要叠加多种规模的合并，总深度为 $O(\log^2 n)$ ，比较工作量为 $O(n \log^2 n)$ 。不能把合并深度直接当成完整排序深度。

为什么零一输入有助于证明网络正确？阈值函数与最小值、最大值相容。如果某个一般输入产生逆序对，选一个位于逆序两值之间的阈值，就会得到一个对应的零一逆序输出。因此，能正确处理全部零一输入的比较网络，也能处理一般有序域中的输入。这是零一原理的核心论证。

基数排序把比较换成稳定分组

对固定宽度的非负整数，可以从低位到高位逐位稳定分组：当前位为 0 的放前面，为 1 的放后面；两组内部保持此前顺序。扫描与 compact 正好能生成这些稳定位置。

代码示例 低位优先的二进制基数排序

```
def radix_sort_unsigned(x, bits):
    out = list(x)
    if any(v < 0 or v >= 2**bits for v in out):
        raise ValueError("key outside declared unsigned width")
    for bit in range(bits):
        zero = [(v >> bit) & 1 == 0 for v in out]
        # 按当前位稳定分组，组内保留较低位已经确定的顺序。
        out = compact(out, zero) + compact(out, [not z for z in zero])
    return out
```

稳定性保证：处理更高一位时，较低各位已经建立的次序在同组内部仍然保留。这里明确限定无符号范围。负整数、浮点数以及 NaN 的排序，需要额外定义键变换和排序契约，不能直接套用这个位操作程序。

高位优先也能形成正确的基数排序，但需要正确递归处理各分组；不能把高位优先一概判断为错误。实际选用哪种算法，应结合键类型、稳定性、数据规模、暂存空间和成熟库实现。

4.7 转置：用共享内存改变访存方向

朴素转置容易让读取连续、写入跨步，或恰好相反。可以先按连续方向把一个 tile 读到共享内存，经过屏障后交换线程访问坐标，再按输出的连续方向写回。

代码示例 支持非方形矩阵与不完整边缘块

```
constexpr int T = 16; // tile 的边长，同时决定二维块形状
__global__ void transpose(const float* x, float* y, int rows, int cols) {
    // 多留一行改变共享数组行步长；冲突程度仍取决于线程布局。
    __shared__ float tile[T][T+1];
    int col=blockIdx.x*T+threadIdx.x;
    int row=blockIdx.y*T+threadIdx.y;
    // 按输入布局读入 tile，边缘元素先做范围检查。
    if(row<rows && col<cols) tile[threadIdx.y][threadIdx.x]=x[row*cols+col];
    __syncthreads();
    // 交换块坐标与块内坐标，让输出列方向的线程写连续位置。
    int out_col=blockIdx.y*T+threadIdx.x;
    int out_row=blockIdx.x*T+threadIdx.y;
    if(out_row<cols && out_col<rows)
        y[out_row*rows+out_col]=tile[threadIdx.x][threadIdx.y];
}
```

输入为 $\text{rows} \times \text{cols}$ ，输出为 $\text{cols} \times \text{rows}$ ，所以输出行步长是 rows 。两个边界条件看起来不同，实际描述的是同一个有效元素的两个坐标系。

例如输出检查通过时，所读共享位置对应的输入行就是 out_col ，输入列就是 out_row ，因此其加载检查也必然通过。这说明为何未加载的边缘共享位置不会被合法输出读取。仅仅“两个地方都写了 if”还不够，必须证明这两个条件确实匹配。

第 5 章 矩阵乘法：复用数据比多写线程更重要

矩阵乘法是深度学习系统中最值得反复理解的算子。全连接层直接调用它，卷积可以转成类似的运算，注意力也包含多个矩阵乘法。它同时展示了三个问题：怎样把数学下标映射到线程，怎样减少重复读取，以及怎样判断性能受什么限制。

5.1 先统一形状、布局和工作量

设

$$A \in \mathbb{R}^{M \times K}, \quad B \in \mathbb{R}^{K \times N}, \quad C = AB \in \mathbb{R}^{M \times N}.$$

单个元素为

$$C_{ij} = \sum_{q=0}^{K-1} A_{iq} B_{qj}.$$

i, j 决定输出位置， q 是归约维。这三个角色不能在“调整访存”时任意交换。对紧凑行优先存储，三种地址分别为

$$A_{iq} \leftrightarrow iK + q, \quad B_{qj} \leftrightarrow qN + j, \quad C_{ij} \leftrightarrow iN + j.$$

一个输出需要约 K 次乘法和 K 次加法，全部输出约需 $2MNK$ 次浮点运算。这里按一次乘法与一次加法分别计数，即使硬件以融合乘加指令执行，常见 FLOP 统计仍按 2 次操作计。

代码示例 最直接的 GPU 映射

```
__global__ void matmul_naive(const float* a, const float* b, float* c,
                             int m, int n, int k) {
    // 相邻 x 线程负责相邻输出列，匹配行优先的连续方向。
    int col = blockIdx.x * blockDim.x + threadIdx.x;
    int row = blockIdx.y * blockDim.y + threadIdx.y;
    if (row < m && col < n) {
        float sum = 0;
        for (int q = 0; q < k; ++q) sum += a[row * k + q] * b[q * n + col];
        c[row * n + col] = sum;
    }
}
```

相邻的 `threadIdx.x` 对应相邻输出列，因此写入 C 连续；对固定 q ，读取 B_{qj} 也连续。每条线程仍沿归约维串行累计，因此这个版本只并行化了输出元素之间的独立工作。

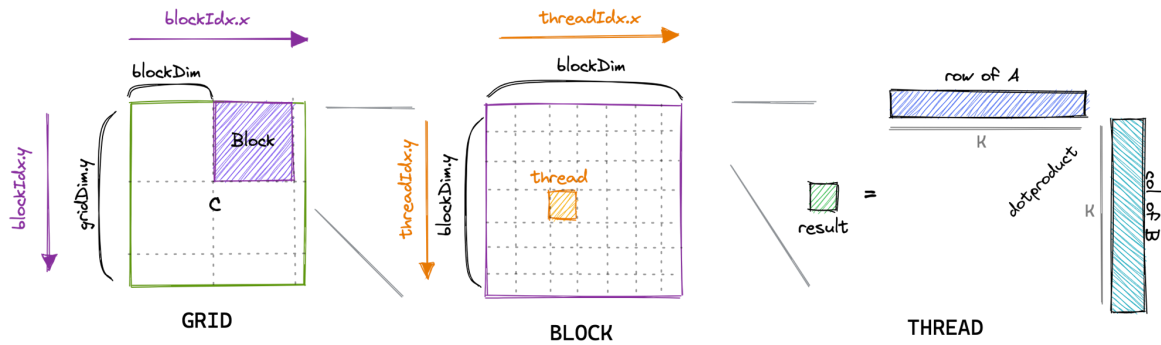


图 5.1 把输出矩阵划成块，再把单个输出交给线程。图展示映射层次；具体行列方向必须与程序的地址公式保持一致。（课程第 06 讲，第 6 页）

5.2 分块：一批输入支持一批输出

如果相邻线程分别计算 $C_{i0}, C_{i1}, \dots$ ，它们都会读取 A 的同一行。若每条线程独立重复读取，许多数据会被重复请求。共享内存分块让整个 block 合作加载一份数据，再共同使用。

令输出 tile 为 $T \times T$ 。沿 K 维每前进 T 个位置，加载一个 A 的 $T \times T$ 小块和一个 B 的 $T \times T$ 小块。每条线程从前者的一行和后者的一列取数据，更新自己负责的输出。

代码示例 逐块累计矩阵乘积

```
template<int T>
__global__ void matmul_tiled(const float* a, const float* b, float* c,
                             int m, int n, int k) {
    __shared__ float as[T][T], bs[T][T];
    int tx=threadIdx.x, ty=threadIdx.y;
    int row=blockIdx.y*T+ty, col=blockIdx.x*T+tx;
    float sum=0;
    for(int base=0; base<k; base+=T) {
        // 全块合作加载两份 tile; 越过归约维或边界的输入补0。
        as[ty][tx]=(row<m && base+tx<k)?a[row*k+base+tx]:0.0f;
        bs[ty][tx]=(base+ty<k && col<n)?b[(base+ty)*n+col]:0.0f;
        __syncthreads();
        // 每条线程复用一行 A 与一列 B，累计自己的输出。
        for(int q=0; q<T; ++q) sum+=as[ty][q]*bs[q][tx];
        __syncthreads();
    }
    // 所有线程完成合作以后，只有合法输出位置写回。
    if(row<m && col<n) c[row*n+col]=sum;
}
```

为这个模板选择 $T = 16$ ，并让块内确实有 16×16 条线程：

```
constexpr int T = 16;
dim3 block(T, T), grid((n+T-1)/T, (m+T-1)/T);
matmul_tiled<T><<<grid, block>>>(d_a, d_b, d_c, m, n, k); // 模板尺寸与块形状一致
```

函数名后的 `<T>` 是 C++ 模板实参，决定共享数组等编译期尺寸；后面的三重尖括号是 CUDA 启动配置。两者承担不同职责，不能漏掉其中一组。

先读地址，再读算式

块内线程坐标为 (ty, tx) 。它加载的 A 元素坐标是 $(row, base + tx)$ ，加载的 B 元素坐标是 $(base + ty, col)$ 。前者让一行线程共同加载 A 的一行，后者让一行线程共同加载 B 的一行。

计算时却读取 $as[ty][q]$ 与 $bs[q][tx]$ ，也就是共享 tile 的行与列。加载分工与计算分工不必一样。共享内存的价值正是让一个线程加载的数据可以供其他线程复用。

运行追踪 一个 2×2 tile

计算输出小块 $(\begin{smallmatrix} c_{10}^{00} & c_{11}^{01} \\ c_{10}^{10} & c_{11}^{11} \end{smallmatrix})$ 时，四个线程共同加载四个 A 元素和四个 B 元素。线程 $(0,0)$ 累计 $a_{00}b_{00} + a_{01}b_{10}$ ；线程 $(0,1)$ 使用同一行 A ，但搭配 B 的第二列。一次加载的 A 元素被多个输出使用。

两个屏障各守住一道边界

第一个屏障位于加载之后，防止计算线程读到尚未加载的共享数据。第二个屏障位于计算之后，防止较快线程进入下一轮并覆盖共享 tile，而较慢线程还在使用当前轮数据。

读之前要等生产者写完，覆盖之前也要等消费者读完。只保留第一个屏障，通常不足以保证循环中的共享缓冲区复用正确。

边界不完整时，超出 M, N, K 的加载位置填 0。无效输出线程仍参与加载和屏障，最后才跳过写出。若在核函数开头因为 $row >= m$ 或 $col >= n$ 就提前返回，可能破坏块内协作。

5.3 用数据流量估计复用收益

忽略缓存、边界和最终写出，一个 $T \times T$ 输出 tile 在每个 K 分块阶段执行约 $2T^3$ 次浮点运算，加载 $2T^2$ 个单精度数，共 $8T^2$ 字节。于是算术强度约为

$$I_{\text{tile}} \approx \frac{2T^3}{8T^2} = \frac{T}{4} \text{ FLOP/byte.}$$

相比每次乘加独立加载两个数的简化模型，复用显著增加。这个估计没有计入跨块缓存复用，也没有计入共享内存和寄存器访问，因此它是帮助定位瓶颈的模型，不是逐条指令的精确流量统计。

若整个 $n \times n$ 矩阵乘法理想地只读取 A, B 各一次、写出 C 一次，单精度的最低必要流量约为 $12n^2$ 字节，算术强度约为 $n/6$ 。实际分块实现能达到多接近这个理想值，还取决于缓存、容量与调度。

tile 越大为什么未必越快

更大的 tile 能增加复用，却同时增加块内线程、共享内存和寄存器需求，可能减少一个 SM 能驻留的 block 数量。边缘不完整 tile 还会增加无效工作。若矩阵某个维度很窄，一个看起来适合大方阵的配置也可能浪费大量线程。

让每条线程计算多个输出，通常能进一步增加寄存器复用，但需要更多寄存器并改变加载安排。优化的核心是平衡复用、可调度并行性和指令成本，不能只优化某一个数字。

5.4 Roofline：算得慢，还是搬得慢

设一次任务有 F 次浮点运算，需要从所分析的内存层次搬运 Q 字节，定义算术强度

$$I = F/Q.$$

若计算峰值为 $P_{\max}$ 、可用带宽为 β ，理想化上界为

$$P \leq \min(P_{\max}, \beta I), \quad t \geq \max(F/P_{\max}, Q/\beta).$$

- Bandwidth bound
 - Bound by how fast the data can be moved through the memory system
- Compute bound
 - Bound by how fast the computation can be done, if the application's arithmetic intensity is higher than the machine balance
- GPU programs are **often (not always) bandwidth-bounded**
 - Improve the memory efficiency

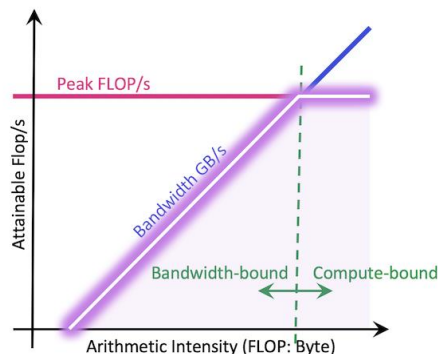


图 5.2 低算术强度区间主要受带宽约束；强度足够高后，计算能力可能成为主要限制。实际性能通常位于上界下方。（课程第 06 讲，第 13 页）

运行追踪 单位要一起检查

假设某个系统可用带宽为 500 GB/s，某算子的算术强度为 2 FLOP/byte，则带宽给出的上界约为 1000 GFLOP/s，即 1 TFLOP/s。即使该精度下的计算峰值为 10 TFLOP/s，也不能仅靠增加算术指令吞吐让它达到峰值。这里的数字是教学假设。

复用数据、融合算子或压缩表示可以降低 Q ；使用适合的硬件指令可以提高可达到的计算吞吐。若任务很小，启动开销可能比两者都显著。Roofline 不会自动描述所有这些额外开销。

occupancy 不是最终目标

occupancy 常用来描述驻留活跃 warp 与设备上限的比例。较高的驻留度有助于隐藏延迟，但一个复用充分、单 warp 工作效率高的 kernel，不必因为 occupancy 较低就一定更慢。为提高驻留度而过度减少寄存器，反而可能引入 spill 与额外内存访问。

比较实现时，应同时记录正确性、输入形状、精度、耗时和所用硬件配置。仅凭一个百分比或一个方阵上的结果，很难判断另一种工作负载。

5.5 cuBLAS：接口中的布局也是数学的一部分

成熟库通常比教学 kernel 有更复杂的分块、指令选择和调度策略。自己实现简单版本的意义，是理解它必须解决的问题；实际系统应根据需求选择并验证库接口。

经典 `cublasSgemm` 使用列优先约定。[5] 若我们持有行优先的 A, B, C ，可以利用

$$C^T = B^T A^T$$

解释同一段内存：行优先的 A 恰好可视作列优先的 A^T 。于是交换乘法顺序和相应维度，而不实际复制转置。

代码示例 包含存储、句柄与结果检查的完整调用

```
#include "common.cuh"
#include <cublas_v2.h>
void blas_check(cublasStatus_t s) {
    require(s==CUBLAS_STATUS_SUCCESS,"cuBLAS error");
}
int main() {
    const int m=2,n=4,k=3;
    std::vector<float>a{1,2,3,4,5,6},b{1,0,2,1,0,1,1,2,2,3,0,1};
    DeviceBuffer<float>da(a.size()),db(b.size()),dc(m*n);
    da.upload(a);db.upload(b);
    cublasHandle_t handle;blas_check(cublasCreate(&handle));
    const float alpha=1,beta=0;
    // 利用 C转置=B转置*A转置，交换输入与维度解释行优先存储。
    blas_check(cublasSgemm(handle,CUBLAS_OP_N,CUBLAS_OP_N,
                           n,m,k,&alpha,db.data(),n,da.data(),k,
                           &beta,dc.data(),n));
    // 取回结果前等待库提交的设备计算完成。
    CUDA_CHECK(cudaDeviceSynchronize());auto c=dc.download();
    for(int i=0;i<m;++i)for(int j=0;j<n;++j) {
        float expected=0;for(int q=0;q<k;++q)expected+=a[i*k+q]*b[q*n+j];
        require(near(c[i*n+j],expected),"row-major cuBLAS mismatch");
    }
    blas_check(cublasDestroy(handle));std::puts("row-major cuBLAS: passed");
}
```

按程序中给定的输入，预期结果为

$$C = \begin{bmatrix} 7 & 11 & 4 & 8 \\ 16 & 23 & 13 & 20 \end{bmatrix}.$$

程序逐项与直接内积比较，便于发现行列或 leading dimension 是否传反。这里传给库的逻辑结果形状是 $N \times M$ ；第一个输入指针是 B ，其列优先解释的行数为 N ，所以 leading dimension 为 N ；第二个输入来自 A ，leading dimension 为 K 。输出的 leading dimension 为 N 。

易错点 leading dimension 是存储步长

对经典列优先 BLAS，它表示从一列起点到下一列起点的元素距离。紧凑矩阵中常等于行数，但有填充或子矩阵时不一定如此。它不是任选的“矩阵大小”，也不是字节数。

通用 GEMM 形式为 $C \leftarrow \alpha AB + \beta C$ 。若 $\beta \neq 0$ ，原 C 必须包含定义好的值。库句柄、设备、流和标量指针模式也属于调用契约。实际程序宜明确管理句柄寿命并复用，避免每次微小算子都反复建立和销毁资源。

库抽象不取消设备与指针约定

Thrust 提供 `transform`、`reduce`、`scan` 和 `sort` 等算法。带有设备信息的迭代器可以帮助选择执行后端；裸指针则不能仅凭数值“自动知道”它是设备地址。使用设备裸指针时，应选合适的设备指针封装或显式执行策略。

cuSPARSE 面向稀疏线性代数，cuRAND 负责相应随机数生成，cuDNN 覆盖许多神经网络算子。这些库各有布局、数据类型、流和资源约定。选库之后，仍要理解自己传入的数组代表什么。

5.6 稀疏矩阵：少做乘法之后，瓶颈去了哪里

稀疏矩阵省略零元素，但必须额外存储索引。COO 保存非零项的行号、列号和值；CSR 按行组织，用 `row_ptr` 指明每行非零项在值数组中的范围。

- Coordinate list (COO)

$$\begin{pmatrix} 5 & 0 & 0 & 0 \\ 0 & 8 & 0 & 0 \\ 0 & 0 & 3 & 0 \\ 0 & 6 & 0 & 0 \end{pmatrix}$$

$$\begin{aligned} V &= [5 \ 8 \ 3 \ 6] \\ \text{COL_INDEX} &= [0 \ 1 \ 2 \ 1] \\ \text{ROW_INDEX} &= [0 \ 1 \ 2 \ 3] \end{aligned}$$

- Compressed sparse row (CSR, CRS)

$$\begin{pmatrix} 10 & 20 & 0 & 0 & 0 & 0 \\ 0 & 30 & 0 & 40 & 0 & 0 \\ 0 & 0 & 50 & 60 & 70 & 0 \\ 0 & 0 & 0 & 0 & 0 & 80 \end{pmatrix}$$

$$\begin{aligned} V &= [10 \ 20 \ 30 \ 40 \ 50 \ 60 \ 70 \ 80] \\ \text{COL_INDEX} &= [0 \ 1 \ 1 \ 3 \ 2 \ 3 \ 4 \ 5] \\ \text{ROW_INDEX} &= [0 \ 2 \ 4 \ 7 \ 8] \end{aligned}$$

图 5.3 COO 与 CSR 的不同表示。CSR 的行指针数组长度为行数加一，最后一个值等于存储的非零项数量。（课程第 06 讲，第 19 页）

代码示例 CSR 矩阵向量乘法

```
def csr_matvec(values, columns, row_ptr, x):
    rows = len(row_ptr) - 1
    y = np.zeros(rows, dtype=np.result_type(values, x))
    for row in range(rows):
        # 行指针决定该行的非零区间，空行自然不进入循环。
        for p in range(row_ptr[row], row_ptr[row + 1]):
            y[row] += values[p] * x[columns[p]]
    return y
```

```
values = np.array([1., 2., 3., 4.])
columns, row_ptr = [0, 2, 1, 2], [0, 2, 2, 4]
y = csr_matvec(values, columns, row_ptr, np.array([1., 2., 3.]))
print(y.tolist()) # 预期[7.0, 0.0, 18.0]，第二行没有非零元素
```


第 i 行只需要遍历区间 $[\text{row_ptr}[i], \text{row_ptr}[i + 1])$ 。空行对应一个空区间，结果自然为 0。索引应在范围内，行指针应单调，值与列号数组长度应一致。若允许重复坐标，还需说明重复项是否代表应相加的贡献。

少了零乘法之后，可能更受不规则读取、索引开销和负载不均衡限制。一条线程处理一行很简单，但行长度差异大时，有的线程很快结束，有的线程持续工作。让一个 warp 或一个块合作处理长行，再用归约合并，可以改善这种情况，却也带来短行的资源浪费。因此，稀疏度高不等于任意稀疏格式和映射都更快。

5.7 矩阵乘法的反向为什么又是矩阵乘法

令标量损失对输出的梯度为 $G = \partial L / \partial C$ 。从分量形式出发：

$$\frac{\partial L}{\partial A_{iq}} = \sum_j \frac{\partial L}{\partial C_{ij}} \frac{\partial C_{ij}}{\partial A_{iq}} = \sum_j G_{ij} B_{qj}.$$

右侧正是矩阵 $GB^\top$ 的 (i, q) 元素。同理，

$$\frac{\partial L}{\partial A} = GB^\top, \quad \frac{\partial L}{\partial B} = A^\top G.$$

不是为了凑形状才加转置，而是共享的求和下标决定了转置的位置。形状检查是验证推导的一步，不能代替推导。

若再加广播偏置 b ，同一个偏置分量参与批次中许多输出，因此其梯度要沿批维相加。前向的复用关系，到了反向就体现为梯度贡献的累积。这个规律也会在卷积、广播和分布式梯度归约中反复出现。

第 6 章 神经网络算子：从前向索引追到反向累积

前面已经有了线程映射、归约和矩阵乘法。现在把它们组成神经网络算子。重点不是再记一遍层的名称，而是明确输入输出、参数共享和梯度流向。一个算子的前向写入方式，往往已经提示了它的反向需要怎样累积。

6.1 卷积的形状与下标约定

NCHW 的四个轴依次表示批大小、通道、高度和宽度。采用这一布局，输入为 $X \in \mathbb{R}^{N \times C_{in} \times H \times W}$ ，卷积核为 $K \in \mathbb{R}^{C_{out} \times C_{in} \times K_h \times K_w}$ 。先考虑不分组的情形，步长为 s_h, s_w ，零填充为 p_h, p_w ，膨胀率为 d_h, d_w 。

$$Y_{n,o,i,j} = \sum_{c,r,t} K_{o,c,r,t} X_{n,c, is_h - p_h + rd_h, js_w - p_w + td_w} + b_o.$$

式中 n 选择样本， o 选择输出通道， c 遍历输入通道； (i, j) 是输出位置， (r, t) 遍历卷积核。超出输入范围的位置按 0 处理。深度学习库通常将这个不翻转卷积核的运算称为卷积；从离散信号处理的命名看，它是互相关。实现时应依据公式，而不是靠名称猜测是否翻转下标。

- Convolution layers
 - Slide a kernel with shape $k \times k$ over the input feature map to produce a new feature map
- Localized connections
 - *Drastically* reduces the parameter count
 - *Typical* convolution kernel size: 3×3
- Share weights across all spatial locations
 - Translation equivariance
 - Translating input features results in a shift in the output features

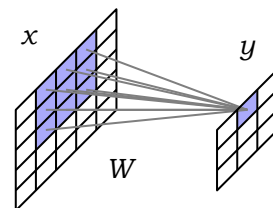
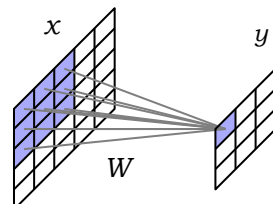


图 6.1 一个局部核在不同空间位置重复使用，形成局部连接与参数共享。（课程第 07 讲，第 4 页）

输出高度为

$$H_{out} = \left\lfloor \frac{H + 2p_h - d_h(K_h - 1) - 1}{s_h} \right\rfloor + 1,$$

宽度同理。分子中的 $d_h(K_h - 1) + 1$ 是核在输入上覆盖的有效高度。每移动一次输出位置，窗口在输入上移动 s_h 个位置。

运行追踪 先计算窗口能放几次

输入高度为 5，核高为 3，填充为 1，步长为 2，膨胀率为 1，则有效输入高度为 7，窗口高度为 3，可从位置 0、2、4 开始，共输出 3 行。公式给出 $\lfloor (5 + 2 - 3)/2 \rfloor + 1 = 3$ 。

参数共享减少参数数量，并在相应边界与采样条件下带来平移等变性。边界填充和大于 1 的步长会改变可保持的平移关系；不能无条件宣称任意离散平移都产生完全相同的输出平移。

6.2 直接实现：一条线程负责一个输出

将 (n, o, i, j) 展平为一个编号，就可以给每个输出分配独立线程。该线程沿输入通道与核窗口求和。

代码示例 NCHW、零填充与步长的基本实现

```
__global__ void conv2d(const float* x, const float* w, float* y,
                      int batch, int ci, int co, int h, int width,
                      int kh, int kw, int stride, int pad, int oh, int ow) {
    int index = blockIdx.x * blockDim.x + threadIdx.x;
    int total = batch * co * oh * ow;
    if (index >= total) return;
    // 先从最快变化的输出列开始，逐层还原四维输出坐标。
    int col = index % ow, row = (index / ow) % oh;
    int out_c = (index / ow / oh) % co, n = index / (ow * oh * co);
    float value = 0;
    for (int c = 0; c < ci; ++c) for (int r = 0; r < kh; ++r) for (int s = 0; s < kw; ++s) {
        // 把输出位置映射回输入窗口；减去 padding 后坐标可能为负。
        int in_r = row * stride - pad + r, in_c = col * stride - pad + s;
        // 越界位置按零处理，不对设备数组执行非法读取。
        if (in_r >= 0 && in_r < h && in_c >= 0 && in_c < width)
            value += x[((n * ci + c) * h + in_r) * width + in_c]
                    * w[((out_c * ci + c) * kh + r) * kw + s];
    }
    y[index] = value;
}
```

展开编号时，最右边的维度变化最快：先用取余得到输出列，再依次得到行、输出通道和批次。代码中的两个地址公式分别对应 NCHW 输入和 OIHW 权重。in_r、in_c 必须先用有符号整数计算，因为减去 padding 后可能为负。

这个版本每个输出只有一个写入者，容易验证；但邻近输出会反复读取重叠窗口，性能还有很大优化空间。它是理解索引与边界的起点，不代表卷积只能按这套循环执行。

6.3 im2col：把局部窗口排成矩阵

把每个输出位置用到的输入窗口展开为一行，得到

$$P(X) \in \mathbb{R}^{(NH_{\text{out}}W_{\text{out}}) \times (C_{\text{in}}K_hK_w)}.$$

把卷积核展平成 $K_{\text{flat}} \in \mathbb{R}^{C_{\text{out}} \times (C_{\text{in}} K_h K_w)}$ ，则卷积可以写成

$$Y_{\text{flat}} = P(X) K_{\text{flat}}^T.$$

这让窗口内积变成 GEMM。输入窗口之间重叠，也意味着同一个输入元素可能被复制到 $P(X)$ 的多个位置。

$$\begin{bmatrix} y_1 \\ y_2 \\ y_3 \\ y_4 \\ y_5 \end{bmatrix} = \begin{bmatrix} 0 & x_1 & x_2 \\ x_1 & x_2 & x_3 \\ x_2 & x_3 & x_4 \\ x_3 & x_4 & x_5 \\ x_4 & x_5 & 0 \end{bmatrix} \begin{bmatrix} w_1 \\ w_2 \\ w_3 \end{bmatrix}$$

$\underbrace{\hspace{10em}}_{\hat{X}}$

图 6.2 把相邻窗口分别展开，形成可以与展平权重相乘的矩阵。展开方向可以选择行或列，只要矩阵公式和布局约定一致。（课程第 07 讲，第 16 页）

显式 im2col 的代价是额外存储和数据移动。对步长较小、核较大的情形，展开矩阵可能远大于原输入。隐式 GEMM 等实现可在需要时生成窗口地址，减少完整物化；专用卷积算法还可能根据形状采用其他变换。是否更快要结合完整的数据路径判断。

分组与逐通道卷积

分组数为 g 时，输入输出通道被划成对应的 g 组，各组只在内部连接。通常要求通道数能被 g 整除，权重形状变为

$$(C_{\text{out}}, C_{\text{in}}/g, K_h, K_w).$$

depthwise 卷积的组数等于输入通道数；每个输入通道独立处理，可有相应通道倍率。它减少了乘加次数，也减少了跨通道复用，因而更容易受访存和启动开销限制。乘法数量变少，不保证端到端速度按同一比例增长。

6.4 卷积的反向：共享的位置都要累加

设 $G = \partial L / \partial Y$ 。对某个权重元素，

$$\frac{\partial L}{\partial K_{o,c,r,t}} = \sum_{n,i,j} G_{n,o,i,j} X_{n,c, is_h - p_h + r d_h, js_w - p_w + t d_w}.$$

同一个核参数在许多窗口使用，因此它的梯度是这些窗口贡献之和。对某个输入元素，则要收集所有覆盖它的输出窗口的贡献。

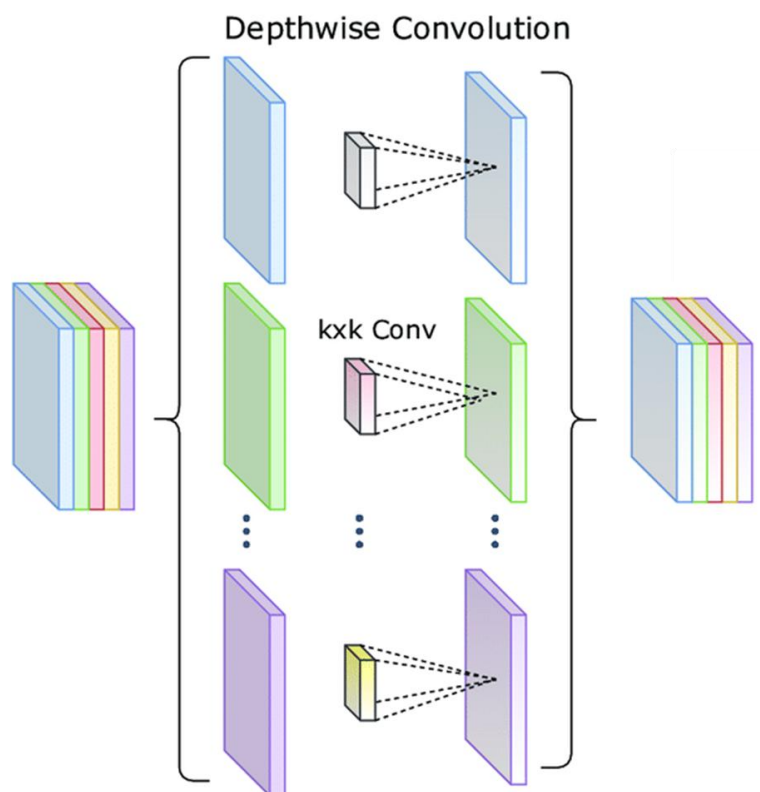


图 6.3 逐通道空间卷积与后续逐点通道混合承担不同职责。（课程第 07 讲，第 19 页）

代码示例 用直接循环写清反向的数学关系

```
def conv_vjp(x, w, grad, stride=1, padding=0, dilation=1, groups=1):
    batch, cin, h, width = x.shape
    cout, cg, kh, kw = w.shape
    xp = np.pad(x, ((0,0), (0,0), (padding,padding), (padding,padding)))
    dxp = np.zeros_like(xp)
    dw = np.zeros_like(w)
    for n, co, i, j in np.ndindex(grad.shape):
        group = co // (cout // groups)
        for ci, r, s in np.ndindex((cg, kh, kw)):
            pos = (n, group*cg+ci, i*stride+r*dilation, j*stride+s*dilation)
            # 同一权重被许多窗口使用，反向贡献必须累计。
            dw[co,ci,r,s] += grad[n,co,i,j] * xp[pos]
            # 一个输入也可能被多个窗口覆盖，不能用赋值覆盖旧贡献。
            dxp[pos] += grad[n,co,i,j] * w[co,ci,r,s]
    return dxp[:, :, padding:padding+h, padding:padding+width], dw
```

这里 `dw` 和 `dxp` 都使用 `+=`，不是赋值覆盖。GPU 若按输出元素并行实现这种反向，一个输入梯度位置就可能同时接收多条线程的贡献。可以使用原子加，也可以改变映射，让一个线程负责收集某个输入位置的全部贡献，或先计算部分结果再归约。

col2im 是伴随操作，不是简单去重

令 P 表示窗口提取。前向是 $P(X)$ ，反向把展开矩阵的梯度通过 P^T 加回原输入。重叠位置必须相加，因此一般有

$$P^T P(X) \neq X.$$

例如一个内部像素被四个窗口提取，提取后再累加回去就得到原值的四倍。理解这一点，便不会把 col2im 错写成只保留最后一个窗口的值。

转置卷积不是逆卷积

固定卷积核后，前向卷积可看作一个线性映射 A 。对输入的反向是 $A^T G$ ，转置卷积实现的核心关系也来自这个伴随映射。 A^T 不等于 A^{-1} ，一般不能从卷积结果无损恢复原输入。

- Deconvolution is used to upsample feature maps, which is widely used in image segmentation / image generation.
- Deconvolution can be implemented by reverse the forward and backward of convolution.

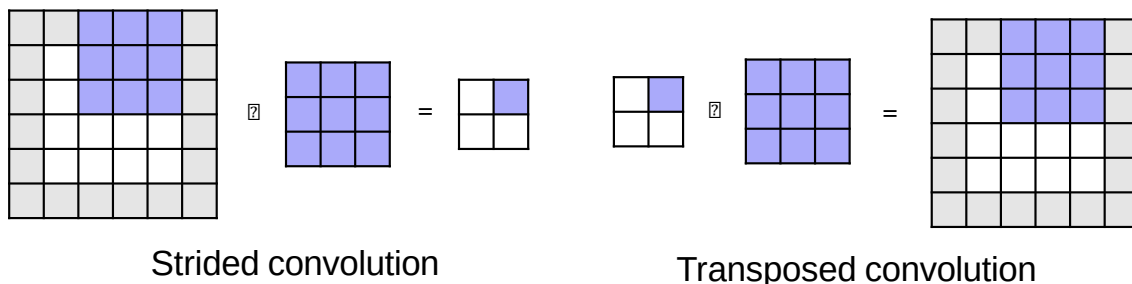


图 6.4 下采样卷积与相应转置映射的空间关系。尺寸扩大不等于恢复了被丢弃的信息。（课程第 07 讲，第 40 页）

6.5 最大池化：保存选择，反向再分配梯度

最大池化在每个窗口中取最大值。前向不仅要输出最大值，还应保存被选择的输入位置，供反向使用。位置应采用合适的整数类型；用浮点数保存大数组索引，可能受到精度限制。

代码示例 重叠窗口必须累加梯度

```
def max_pool2d(x, window=2, stride=1):
    h, w = x.shape
    oh, ow = (h-window)//stride+1, (w-window)//stride+1
    y = np.empty((oh, ow))
    mask = np.empty((oh, ow), dtype=np.int64)
    for i, j in np.ndindex(y.shape):
        patch = x[i*stride:i*stride+window, j*stride:j*stride+window]
        local = int(np.argmax(patch)) # first maximum in row-major order
        r, c = divmod(local, window)
        y[i, j] = patch[r, c]
        mask[i, j] = (i*stride+r)*w + j*stride+c
```


- What is the gradient of $\max(x_1, x_2, \dots, x_k)$?

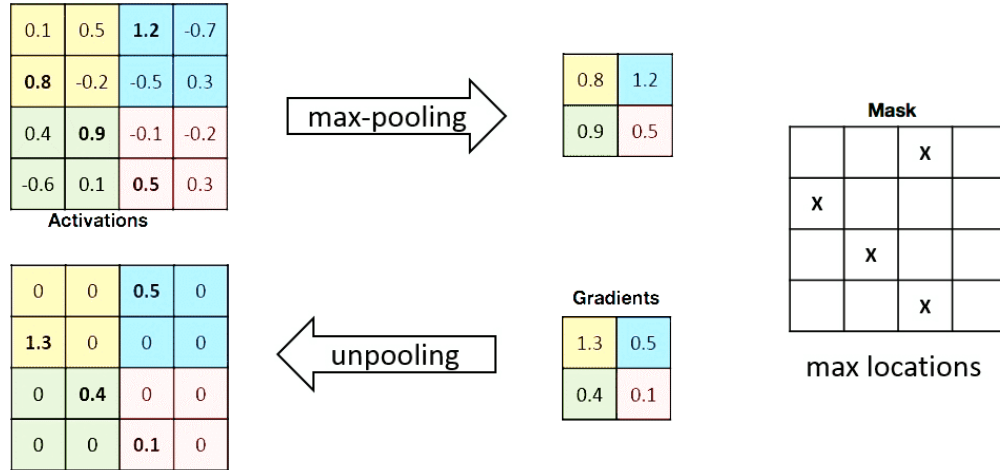


图 6.5 前向保存最大值位置，反向把输出梯度送对应位置。多个窗口可能选择同一个输入。（课程第 07 讲，第 28 页）

```

return y, mask

def pool_vjp(mask, grad, input_shape):
    dx = np.zeros(input_shape)
    # 重复的池化获胜位置要多次累加，不能按普通索引覆盖。
    np.add.at(dx.reshape(-1), mask.reshape(-1), grad.reshape(-1))
    return dx

```

`np.add.at` 在这里处理重复索引的累加。若把它直接换成带重复索引的普通高级索引赋值，不应假设仍得到相同的逐次累积语义。

当窗口中最大值唯一时，局部导数对该位置为 1，对其余位置为 0。出现相等最大值时，函数在相应位置可能不可微，需要一个明确且与前向一致的子梯度约定。示例选行优先顺序的第一个最大值。它假设输入有限；NaN 的处理应由接口另行规定。

配套 CUDA 程序使用一条线程处理一个窗口，反向用 `atomicAdd` 合并重叠贡献。对中心值 9 同时成为四个重叠窗口最大值的 3×3 输入，若各输出梯度均为 1，则中心输入梯度的预期值是 4。

6.6 Softmax：两次归约与数值范围

对一行 logits $z \in \mathbb{R}^C$,

$$p_i = \frac{e^{z_i}}{\sum_j e^{z_j}}.$$

直接计算指数可能溢出。选 $m = \max_j z_j$ ，同时在分子分母乘以 e^{-m} ，得到等价式

$$p_i = \frac{e^{z_i - m}}{\sum_j e^{z_j - m}}.$$

对有限 logits，指数参数都不大于 0，且至少一个指数为 1。这避免了正向指数溢出，并保证分母不因所有项一起下溢而成为 0。

实现上可以让一个块负责一行：先归约求最大值，再求移位指数和，最后归一化。类别数大于块大小时，每条线程用跨步循环处理多个类别。

代码示例 共享存储先保存最大值，再保存和

```
constexpr int B = 128; // 一行由一个含128条线程的块处理
__global__ void softmax_rows(const float* x, float* y, int cols) {
    __shared__ float s[B];
    int t=threadIdx.x, row=blockIdx.x;
    float local_max=-CUDART_INF_F;
    // 每个线程处理若干列，类别数可以超过块内线程数。
    for(int c=t; c<cols; c+=B) local_max=fmaxf(local_max, x[row*cols+c]);
    s[t]=local_max; __syncthreads();
    for(int step=B/2; step; step/=2) {
        if(t<step) s[t]=fmaxf(s[t], s[t+step]);
        __syncthreads();
    }
    // 把共享最大值读入自己的寄存器，再允许共享数组复用。
    float maximum=s[0];
    __syncthreads(); // all threads finish reading s[0] before reusing s
    float local_sum=0;
    for(int c=t; c<cols; c+=B) {
        // 减去整行最大值，避免大正数直接进入指数运算。
        float value=expf(x[row*cols+c]-maximum);
        y[row*cols+c]=value; local_sum+=value;
    }
    // 共享存储第二次用于求和；前后两个阶段不能互相覆盖。
    s[t]=local_sum; __syncthreads();
    for(int step=B/2; step; step/=2) {
        if(t<step) s[t]+=s[t+step];
        __syncthreads();
    }
    // 所有列共用同一个分母，形成和为1的一行概率。
    for(int c=t; c<cols; c+=B) y[row*cols+c]/=s[0];
}
```

输入按连续行存储时，启动配置为：

```
softmax_rows<<<rows, B>>>(d_x, d_y, cols); // 一块对应一行
```

rows 是行数，cols 是每行类别数。核函数中的 blockIdx.x 选择当前行，threadIdx.x 决定该行内的分工。

注意读取 s[0] 到局部变量后又有一次屏障。它保证所有线程都已经取得最大值，才允许复用共享数组存放局部和。省略这道边界，较快线程可能先覆盖较慢线程尚未读取的 s[0]。例子中的两次归约要求启动恰好 B 条线程，且 B 是 2 的幂。

Softmax 的梯度是一种低成本的向量运算

由商法则，

$$\frac{\partial p_i}{\partial z_j} = p_i(\delta_{ij} - p_j).$$

若上游梯度为 $g_i = \partial L / \partial p_i$ ，则

$$\frac{\partial L}{\partial z_j} = \sum_i g_i p_i (\delta_{ij} - p_j) = p_j \left(g_j - \sum_i g_i p_i \right).$$

不需要建立 $C \times C$ 的 Jacobian。先计算一次点积 $\sum_i g_i p_i$ ，再逐元素运算即可。这个例子会帮助我们理解自动微分为何通常计算向量与 Jacobian 的乘积，而不物化完整导数矩阵。

6.7 交叉熵：把稳定性延伸到损失计算

对正确类别 y ，

$$\ell(z, y) = -\log p_y = \log \sum_j e^{z_j} - z_y = m + \log \sum_j e^{z_j - m} - z_y.$$

即使 Softmax 已经减去最大值，很小的 p_y 仍可能下溢到 0。先算概率再取对数会得到无穷大；直接使用最后一种 log-sum-exp 形式，可以避免这类中间结果问题。

代码示例 从 logits 直接计算损失与梯度

```
def cross_entropy(logits, labels):
    shifted = logits - logits.max(axis=1, keepdims=True)
    exp_scores = np.exp(shifted)
    denominator = exp_scores.sum(axis=1, keepdims=True)
    # 直接用移位后的 logits 算损失，避免先得到0概率再取对数。
    loss = np.mean(np.log(denominator[:, 0])
                  - shifted[np.arange(len(labels)), labels])
    grad = exp_scores / denominator
    grad[np.arange(len(labels)), labels] -= 1
    return loss, grad / len(labels)
```

对 z_j 求导：

$$\frac{\partial \ell}{\partial z_j} = \frac{e^{z_j}}{\sum_k e^{z_k}} - \mathbf{1}[j = y] = p_j - \mathbf{1}[j = y].$$

批次平均损失再乘以 $1/N$ 。这说明第一章中“概率减独热标签”的简短代码来自什么推导。若换成求和损失、类别权重或标签平滑，归一化和目标分布也会改变，不能仍无条件使用完全相同的公式。

例如 logits 为 $[1000, -1000, 0]$ ，正确类别是第二类，预期损失约为 2000，而不是无穷大。这个输入很适合检查实现是否真正采用稳定的损失形式。

检查算子时，同时覆盖非方形输入、尾部、padding、stride 及重叠窗口，并在可微点核对反向梯度。尤其要检查重叠与广播产生的累积关系。

第 7 章 连接 Python 与底层算子：边界上的约定

CUDA 算子解决了怎样计算，Python 接口解决了怎样把它交给上层程序。这个连接并不只是“给函数起一个 Python 名字”。数组的数据类型、步长、设备、寿命和梯度，都必须跨越语言边界保持一致。

本章先用一个小型 pybind11 模块看清数据边界，再说明怎样把自定义算子接入自动微分。具体工程文件放在配套目录，正文保留需要理解的部分。

7.1 模块名、函数名与编译产物

pybind11 用 C++ 描述 Python 可访问的函数与类。一个最小模块可以写成：

代码示例 先绑定一个可以直接调用的加法函数

```
#include <pybind11/pybind11.h>
int add(int a, int b) { return a+b; } // 普通C++函数
PYBIND11_MODULE(notes_ops, m) {
    m.def("add", &add, "Add two integers"); // 建立Python名字与函数的联系
}
```

PYBIND11_MODULE(notes_ops,m) 声明模块初始化入口，m 是绑定时操作的模块对象。m.def 建立 Python 名字 add 与 C++ 函数地址之间的联系。编译生成的扩展模块必须与导入名称及 Python ABI 相匹配。

```
import notes_ops
print(notes_ops.add(2, 3)) # 预期输出5
```

头文件库并不意味着不需要编译器。绑定代码仍要经过 C++ 编译并生成供 Python 动态加载的模块。若需要 CUDA，还要把相应设备代码纳入构建流程；普通 C++ 扩展的构建方法不会自动识别任意核函数。

7.2 NumPy 数组：形状之外还有步长

NumPy 的数组描述包含数据地址、元素类型、维度、形状与步长。数组转置或切片后，可能仍指向原存储，但相邻逻辑元素不再在物理地址上连续。直接写 ptr[i] 的循环，仅适用于相应的连续布局约定。

下面的接口显式请求连续 float32 输入，并允许绑定层为了满足该条件而转换：

代码示例 输入可转换，输出由新数组拥有

```

py::array_t<float> relu(
    py::array_t<float, py::array::c_style|py::array::forcecast> input) {
    // 取得数组形状和数据地址；接口已请求连续的 float32 表示。
    auto info=input.request();
    std::vector<py::ssize_t> shape(info.shape);
    // 由返回数组自己拥有输出存储，函数返回后数据仍然有效。
    py::array_t<float> output(shape);
    const float* x=static_cast<const float*>(info.ptr);
    float* y=output.mutable_data();
    // 在C++侧逐元素计算；不修改传入的数组。
    for(py::ssize_t i=0;i<info.size;++i)
        y[i]=x[i]>0.0f?x[i]:0.0f;
    return output;
}

```

`py::array_t<float,...>` 描述元素类型；`c_style` 要求 C 顺序的连续排列；`forcecast` 允许转换为所需表示。因此，传入 `float64` 数组或非连续视图时，可能先产生一份临时连续 `float32` 副本。这是接口设计的一种选择，应明确其精度和复制成本。[6]

输出通过 `py::array_t<float> output(shape)` 分配，由返回的 Python 数组对象负责寿命。C++ 函数结束后，数据不会随着某个局部 `std::vector` 一起被释放。

易错点 零复制必须同时解决布局与寿命

返回一个包装局部数组地址的 NumPy 视图，会让 Python 获得悬垂存储。若希望共享外部数据，应让返回对象持有真实拥有者，或通过合适的 `capsule` 与释放规则建立寿命关系。指针非空不等于可以长期使用。

把这个 `relu` 函数放在模块定义前，再向同一个模块加入一行绑定：

```
m.def("relu", &relu, "Return a new float32 array"); // 放入PYBIND11_MODULE的函数体
```

随后就能从 Python 调用：

```

import numpy as np
x = np.array([-1.0, 2.0], dtype=np.float32)
y = notes_ops.relu(x)
print(y.tolist()) # 预期输出 [0.0, 2.0]，x本身保持原值

```

另一种合理接口是拒绝不连续或错误 `dtype` 的输入，让调用者显式转换。也可以保留任意步长，在 C++ 中按步长访问。三者都有适用场景，不能一边允许任意 NumPy 视图，一边默默按连续数组读取。

输入输出是否共享存储也是契约

本例返回新数组，不修改输入。调用者因此可以保留输入供反向传播使用。若改成原地操作，应同时说明输入是否可写、是否与其他视图别名，以及是否会破坏梯度计算保存的数据。对 CUDA 异步调用，还要保证所需缓冲区至少存活到设备使用结束。

7.3 构建：让依赖于明确的目标

配套模块使用 CMake：

```
cmake_minimum_required(VERSION 3.20)
project(notes_bindings LANGUAGES CXX)
# 使用当前Python环境对应的开发头文件与模块构建规则。
find_package(Python COMPONENTS Interpreter Development.Module REQUIRED)
find_package(pybind11 CONFIG REQUIRED)
# 生成可由Python导入的notes_ops扩展模块。
pybind11_add_module(notes_ops ops.cpp)
target_compile_features(notes_ops PRIVATE cxx_std_17)
```

Python 开发头文件、当前解释器以及 pybind11 的 CMake 配置需要被构建系统找到。构建与导入最好使用同一个 Python 环境。若构建时链接了一个解释器版本，而运行时换成另一个不兼容版本，导入失败并不表示数值算法本身有错。

若把 CUDA 算子接入现有框架，可使用框架支持的扩展构建入口，避免自行猜测其头文件、ABI 和链接配置。构建系统负责组织依赖，运行时仍负责检查设备和数据契约。

7.4 接入 PyTorch：前向函数还缺少哪些信息

框架中的 Tensor 已携带形状、步长、dtype、device 和存储拥有关系。C++/CUDA 扩展应先验证自己支持的条件，再提取数据指针。对一个仅支持连续 CUDA float32 的实现，至少应明确检查设备、元素类型和布局；矩阵或卷积还需检查维度之间的对应关系。

```
import torch
import notes_cuda as cuda_extension # 导入配套工程建立的扩展模块

class MyReLU(torch.autograd.Function):
    @staticmethod
    def forward(ctx, x):
        x_contiguous = x.contiguous() # 底层 kernel 只处理连续数组
        ctx.save_for_backward(x_contiguous)
        return cuda_extension.relu(x_contiguous)

    @staticmethod
    def backward(ctx, grad_output):
        (x,) = ctx.saved_tensors
        return grad_output * (x > 0) # 乘局部导数，不覆盖上游梯度
```

这里的 `cuda_extension.relu` 代表绑定好的底层实现。`forward` 保存反向确实需要的输入，`backward` 接收上游梯度，再按局部导数生成对输入的梯度。返回项应与前向输入一一对应；对于不求导的参数，相应位置按接口约定返回 `None`。本例在 0 处采用导数为 0 的约定。[\[7\]](#)

让底层调用实际接入设备与流

配套的 `torch_extension` 工程把第二章的 ReLU kernel 包装成 `notes_cuda.relu`。包装函数先检查输入条件，再从 Tensor 取得设备指针与当前流：

```
torch::Tensor relu_cuda(torch::Tensor input) {
```



```

TORCH_CHECK(input.is_cuda(), "expected CUDA input");
TORCH_CHECK(input.scalar_type()==at::kFloat, "expected float32");
TORCH_CHECK(input.is_contiguous(), "expected contiguous input");
c10::cuda::CUDAGuard guard(input.device()); // 在输入所属设备上分配与启动
auto output=torch::empty_like(input);
auto n=input.numel();
if(n==0)return output;
int blocks=int(std::min<std::int64_t>((n+255)/256,65535));
auto stream=c10::cuda::getCurrentCUDAStream(input.get_device());
relu_kernel<<<blocks,256,0,stream.stream()>>>(<
    input.data_ptr<float>(),output.data_ptr<float>(),n);
C10_CUDA_KERNEL_LAUNCH_CHECK();           // 检查启动，不强制全设备同步
return output;
}

```

TORCH_CHECK 在条件不满足时报告错误。CUDAGuard 把操作放到输入所属设备，并在作用域结束后恢复原设备上下文。输出 Tensor 自己管理存储，数据指针只用于当前底层计算。

在配套扩展目录中使用与 PyTorch 兼容的 CUDA 工具链构建：

```

cd examples/torch_extension
python setup.py build_ext --inplace
python example.py

```

Python 使用 MyReLU.apply(x) 调用这个可求导包装。输入为 $[-2, 0, 3]$ 时，预期输出为 $[0, 0, 3]$ ；上游梯度为 $[0.2, 0.4, 0.6]$ 时，预期输入梯度为 $[0, 0, 0.6]$ 。这样，底层数值计算与 Python 反向规则就在同一次调用中配合起来。

Python 能调用，不代表框架理解其语义

简单绑定可以使 Python 调用 C++ 函数，但要与图捕获、自动微分、设备分派或编译优化良好配合，还可能需注册算子 schema、实现相应 dispatch 路径，以及提供形状推断等信息。不同接入方式服务不同复杂度的需求。[8]

设备端执行也必须接入正确的 CUDA 流。框架可能使用非默认流；扩展若擅自在默认流启动，又不建立依赖，就可能在简单测试中看似正确，却在并发使用时读到尚未准备好的数据。当前设备和当前流应来自相应框架上下文。

GIL 只处理 Python 层的一部分并发

对不访问 Python 对象的长时间 C++ 计算，可以考虑在安全范围内释放 GIL。但引用计数、Python API 调用和对象构造仍有相应要求。释放 GIL 不会自动同步 CUDA，也不会使共享 C++ 数据结构变得线程安全。应分别分析这几层并发责任。

7.5 测试要对准边界，而不只跑一个随机数组

一个好的算子测试至少回答三类问题。

1. **数值是否正确。**与独立、清晰的参考比较，选用与精度和规模相符的绝对、相对误差。
2. **接口是否守约。**检查非连续视图、空数组、dtype 转换、设备要求以及输入是否被意外修改。

3. 梯度是否正确。在可微点使用小规模双精度输入，以中心差分核对相应梯度。

常用浮点比较形式为

$$|a - b| \leq \varepsilon_{\text{abs}} + \varepsilon_{\text{rel}}|b|.$$

接近 0 时，相对误差单独使用可能失去意义；数值很大时，仅用很小的绝对误差也可能过严。容差应来自算法和精度分析，不能为了通过测试不断放宽到掩盖错误。

梯度检查为什么要避开不可微点

中心差分

$$\frac{f(x+h) - f(x-h)}{2h}$$

用两侧函数值估计导数。若区间跨过 ReLU 的折点或最大池化的最大值切换位置，差分未必等于程序所选的子梯度。先用平滑、远离分界的位置检查局部导数，再单独检查分界约定。

h 太大会带来截断误差，太小又会放大浮点相消影响。双精度与合适步长能改善检查，但数值梯度本身也不是无误差的真值。仅支持 float32 的 kernel 不能直接用于双精度检查；可以另提供双精度实现，或先用高精度参考核对导数，再按目标精度设置比较容差。对大参数空间，可以用随机方向上的方向导数检查，降低逐元素扰动成本。

GPU 错误可能在后续接口才出现

核函数越界、未初始化读取或同步错误，可能在后续同步时才报告。调试时应缩小输入，在有意义的边界增加同步，并使用内存、竞争及同步检查工具。测试用例尤其应覆盖尺寸略小于、等于和略大于块大小的情况，避免只测整齐的方阵或整块长度。

第 8 章 自动微分：把链式法则变成可执行的程序

手工推导过卷积、矩阵乘法和 Softmax 的梯度后，一个自然问题是：能否只为每个基本算子写一次局部导数，让系统自动组合它们？自动微分正是把链式法则变成一套可执行的数据流规则。

本章从一个只有两个输入的函数开始。先逐步计算前向值和局部导数，再解释 JVP、VJP、拓扑顺序与梯度累加。最后把这些规则装入一个小型 Tensor 类。数学表示和实现会使用同一套符号。

8.1 先区分三种求导方式

符号微分操作表达式，例如把 x^2 转成 $2x$ ，结果仍是一个表达式。若把复杂嵌套表达式完全展开，结果可能变得很大；保留公共子表达式与图结构可以缓解这一问题。

数值微分根据附近函数值估计导数，例如中心差分。它不需要知道内部算子，却需要额外函数求值，并受到截断误差和舍入误差影响，因此特别适合作为小规模检查工具。

自动微分则沿实际计算的基本操作应用已知导数。它既不需要把整个函数展开成巨大公式，也不靠一个小步长近似导数。导数计算仍使用浮点运算，因此仍有舍入、溢出和算法条件数等数值问题。

核心概念 自动微分组合的是局部规则

若程序由基本算子组成，并为每个算子提供相应的导数作用规则，系统便可以沿计算依赖把这些局部信息组合起来。这里“自动”的是组合过程，不是自动理解任意外部函数的数学含义。

8.2 把一个函数拆成图上的节点

考虑

$$f(a, b) = \log a + ab - \sin b, \quad a > 0.$$

把中间值命名为

$$u = \log a, \quad v = ab, \quad w = \sin b, \quad r = u + v, \quad f = r - w.$$

每条有向边表示一个值被后续操作使用。 a 同时进入对数和乘法， b 同时进入乘法和正弦，因此图中已经出现共享输入。

在 $a = 2, b = 5$ 处，前向计算依次得到：

图中的“节点”可以表示值，也可以在另一种中间表示里表示算子。重要的是明确输入、输出和依赖，不要在两种约定之间混淆一个节点到底保存了什么。

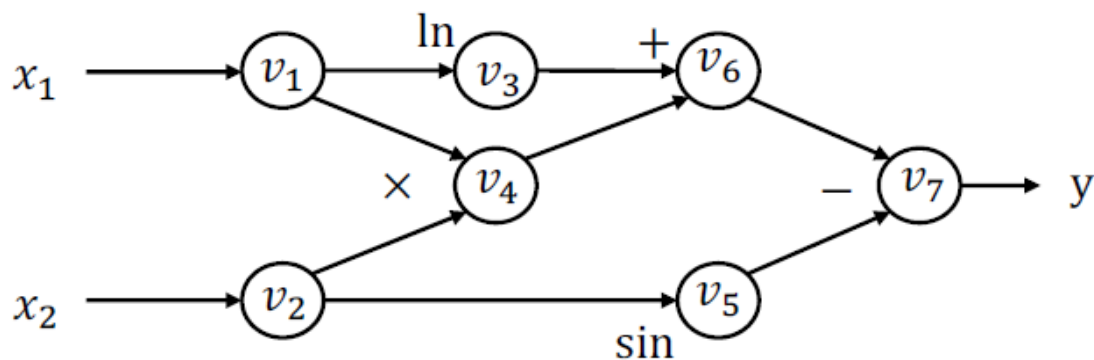


图 8.1 同一个输入可以沿多条路径影响输出。反向计算必须收集所有路径的贡献。(课程第 09 讲, 第 29 页)

节点	计算	数值 (近似)
u	$\log a$	0.693147
v	ab	10
w	$\sin b$	-0.958924
r	$u + v$	10.693147
f	$r - w$	11.652071

表 8.1 前向计算先建立反向可能需要的中间值。

8.3 前向模式：把一个输入方向沿图推进

假设输入随一个小参数 ε 改变：

$$a(\varepsilon) = a + \varepsilon \dot{a}, \quad b(\varepsilon) = b + \varepsilon \dot{b}.$$

点号表示对 ε 的导数，即选定输入方向引起的一阶变化。根据链式法则，

$$\dot{u} = \dot{a}/a, \quad \dot{v} = \dot{a}b + a\dot{b}, \quad \dot{w} = \cos b \dot{b}, \quad \dot{f} = \dot{u} + \dot{v} - \dot{w}.$$

如果想求 $\partial f / \partial a$ ，就选 $(\dot{a}, \dot{b}) = (1, 0)$ 。在本例中，

$$\dot{u} = 0.5, \quad \dot{v} = 5, \quad \dot{w} = 0, \quad \dot{f} = 5.5.$$

选 $(0, 1)$ 则得到 $\partial f / \partial b = 2 - \cos 5$ 。选其他方向，也能一次得到该方向上的方向导数。

对偶数把值与变化放在一起

用一对数 $(v, \dot{v})$ 表示中间结果，基本算子同时计算新值与新切向量。例如乘法为

$$(a, \dot{a})(b, \dot{b}) = (ab, \dot{a}b + a\dot{b}).$$

代码示例 一个足够说明前向模式的小类型

```

from dataclasses import dataclass
import math

@dataclass
class Dual:
    value: float
    tangent: float
    def __add__(self, other):
        return Dual(self.value+other.value, self.tangent+other.tangent)
    def __mul__(self, other):
        # 乘积的切向量同时包含左边变化和右边变化的贡献。
        return Dual(self.value*other.value,
                    self.tangent*other.value+self.value*other.tangent)
    def log(self):
        return Dual(math.log(self.value), self.tangent/self.value)
    def sin(self):
        return Dual(math.sin(self.value), math.cos(self.value)*self.tangent)
    def __sub__(self, other):
        return Dual(self.value-other.value, self.tangent-other.tangent)

```

```

a, b = Dual(2.0, 1.0), Dual(5.0, 0.0)  # 选取沿a变化的方向
result = a.log() + a*b - b.sin()
print(result.tangent)                    # 预期5.5

```

运算符重载只让普通表达式自动调用这些局部规则，并没有改变链式法则。每个操作完成后，数值部分与导数部分一起继续向前传递，因此不需要等整个函数计算结束才开始求导。

8.4 反向模式：从输出的重要性回传到输入

现在固定一个标量目标 L ，用

$$\bar{v} = \frac{\partial L}{\partial v}$$

表示中间值 v 对目标的影响。它称为伴随量或上游梯度。若一个局部操作为 $z = \phi(x)$ ，已经知道 $\bar{z}$ ，便可计算

$$\bar{x} += \bar{z} \phi'(x).$$

加号非常重要：同一个 x 可能还从其他使用者那里获得贡献。

在本例中令 $L = f$ ，所以起点是 $\bar{f} = 1$ 。从最后一步开始：

$$f = r - w \quad \Rightarrow \quad \bar{r} = 1, \quad \bar{w} = -1.$$

再由 $r = u + v$ 得到 $\bar{u} = 1, \bar{v} = 1$ 。随后分别传向两个输入：

$$\begin{aligned}
 \bar{a} &= \underbrace{\bar{u}/a}_{\text{对数路径}} + \underbrace{\bar{v}b}_{\text{乘法路径}} = 0.5 + 5 = 5.5, \\
 \bar{b} &= \underbrace{\bar{v}a}_{\text{乘法路径}} + \underbrace{\bar{w}\cos b}_{\text{正弦路径}} = 2 - \cos 5 \approx 1.716338.
 \end{aligned}$$

原理剖析 反向不是沿一条路径找导数

输出依赖某个变量的每一条可微路径，都可能贡献一部分梯度。共享节点的梯度应先收集完整，再通过该节点的局部规则继续向前面的输入传播。

一个输入在同一算子里出现两次也要算两次

对 $u = x \cdot x$ ，乘法的左输入与右输入虽然指向同一个对象，仍代表两个使用位置：

$$\bar{x} += \bar{u} x, \quad \bar{x} += \bar{u} x.$$

因此总贡献是 $2x\bar{u}$ 。若构建图时把算子的输入列表粗暴改成集合，就可能把重复的输入边丢掉。拓扑遍历可以对节点去重，局部导数传播却必须保留输入位置的重复关系。

8.5 JVP 与 VJP：通常不需要完整 Jacobian

对 $f: \mathbb{R}^n \rightarrow \mathbb{R}^m$ ，Jacobian 为

$$J_{ij} = \frac{\partial f_i}{\partial x_j}, \quad J \in \mathbb{R}^{m \times n}.$$

一个小输入扰动 $v \in \mathbb{R}^n$ 产生的一阶输出变化为

$$\dot{y} = Jv.$$

这就是 Jacobian-vector product，简称 JVP，前向模式自然计算它。

若有输出权重 $g \in \mathbb{R}^m$ ，定义标量目标 $L = g^\top f(x)$ ，则

$$\nabla_x L = J^\top g.$$

这是反向模式的基本计算。按行向量书写时等价于 $g^\top J$ ，因此通常称为 vector-Jacobian product，即 VJP。本文梯度默认用列向量或与原 Tensor 同形状的数组表示。

运行追踪 向量输出的 backward 在问什么

若 $y = [x_1^2, x_1 x_2]$ ，传入种子 $g = [2, 3]$ ，计算的是 $L = 2x_1^2 + 3x_1 x_2$ 的梯度： $\bar{x}_1 = 4x_1 + 3x_2$ 、 $\bar{x}_2 = 3x_1$ 。它不是一次自动返回整个 2×2 Jacobian。

对单个标量损失和大量参数，反向模式很合适：一次种子为 1 的传播就得到所有参数的梯度。若输入维数很小、输出很多，并且需要完整 Jacobian，前向模式可能更省。真正成本还与所需方向数量、内存、算子实现和向量化方式有关，不能只比较 m, n 就断言所有任务的最快方法。

8.6 张量的局部反向：形状变化如何逆向处理

逐元素加法、乘法、矩阵乘法的规则已经出现过。更容易遗漏的是广播、归约和视图操作，因为它们的前向代码通常很短。

广播的反向是沿重复使用的方向求和

若 $Y_{ij} = X_{ij} + b_j$ ，同一个 b_j 在每个批次位置都被使用一次，所以

$$\bar{X}_{ij} = \bar{Y}_{ij}, \quad \bar{b}_j = \sum_i \bar{Y}_{ij}.$$

一般广播把长度为 1 的轴扩展，或在前面补维。反向就先消去额外前导维，再沿被扩展的长度为 1 的轴求和，并恢复原形状：

代码示例 把输出形状的梯度还原到输入形状

```
def sum_to_shape(grad, shape):
    # 先消去广播添加的前导维，再处理原来长度为1的轴。
    while grad.ndim > len(shape):
        grad = grad.sum(axis=0)
    axes = tuple(i for i, size in enumerate(shape)
                  if size == 1 and grad.shape[i] != 1)
    if axes:
        grad = grad.sum(axis=axes, keepdims=True)
    return grad.reshape(shape)
```

假设原输入形状为 (1,3,1)，广播后参与 (2,3,4) 的计算，则梯度应沿第 0、2 轴求和，并保留为 (1,3,1)。只删掉多余维度，而没有累加贡献，会得到错误的数值。

归约的反向是把上游梯度送回各参与位置

若 $y = \sum_i x_i$ ，每个输入的局部导数都是 1，所以 $\bar{x}_i = \bar{y}$ 。对部分轴求和，先把被删掉的轴以长度 1 放回去，再广播到输入形状。平均值则还需除以相应归约元素数量。

重排的反向使用逆重排

reshape 只改变元素组织时，反向把梯度恢复原形状；transpose 的反向应用逆轴置换。若前向复制了某个输入到多个位置，反向仍要累加，不能再把它归入纯粹的一一重排。

对二维矩阵乘法 $C = AB$,

$$\bar{A} = \bar{C}B^T, \quad \bar{B} = A^T\bar{C}.$$

批量矩阵乘法还需处理批维广播；配套的小引擎将矩阵乘法明确限定为二维，让基本规则保持清晰。

8.7 实现一个小型反向引擎

先把节点需要的状态落实到一个对象：data 保存前向值，parents 记录输入节点，vjp 保存局部反向函数，grad 保存反向得到的结果。完整实现见附录，核心建立方式如下：

```
class Tensor:
```

```
    def __init__(self, data, requires_grad=True, parents=(), vjp=None):
        self.data = np.array(data, dtype=np.float64, copy=True)
        # 保存前向值供反向使用，避免无意的原地修改破坏导数。
        self.data.flags.writeable = False
        self.requires_grad = requires_grad
        self.parents = tuple(parents) if requires_grad else ()
        self.vjp = vjp
        self.grad = None
```



```
@staticmethod
def coerce(value):
    return value if isinstance(value, Tensor) else Tensor(value, requires_grad=
False)
```

普通常量经 `coerce` 包装成不求导节点，因此也能参与统一的操作。接着以乘法为例，定义怎样建立新节点并保存其局部规则：

代码示例 局部 VJP 同时处理重复边与广播

```
def __mul__(self, other):
    other = Tensor.coerce(other)
    # 局部反向使用前向时对应的两个操作数值。
    a, b = self.data, other.data
    def vjp(g):
        return (sum_to_shape(g * b, a.shape),
                sum_to_shape(g * a, b.shape))
    return Tensor(a * b, self.requires_grad or other.requires_grad,
                  (self, other), vjp)
```

`parents` 保存一个有序元组。若写 `x*x`，同一个节点会出现两次；`vjp` 也返回两份贡献。保存 `a, b` 是为了让反向使用前向计算时对应的数值。

为什么需要反向拓扑顺序

正向拓扑顺序要求每个节点的输入先于该节点出现。反向遍历这个顺序时，一个节点的所有后续使用者都已经传播过贡献，才能把该节点的总梯度继续传给输入。

代码示例 先收集贡献，再通过局部规则传播

```
def backward(self, seed=None, accumulate=False):
    if seed is None:
        if self.data.size != 1:
            raise ValueError("a non-scalar output needs an explicit seed")
        seed = np.ones_like(self.data)
    seed = np.asarray(seed, dtype=np.float64)
    if seed.shape != self.data.shape:
        raise ValueError("seed shape must equal output shape")
    order, seen = [], set()
    # 先访问输入再记录当前节点，得到正向拓扑顺序。
    def visit(node):
        if node in seen:
            return
        seen.add(node)
        for parent in node.parents:
            visit(parent)
        order.append(node)
    visit(self)
    grads = {self: seed}
```

```

# 反向拓扑序保证下游使用者先传播贡献。
for node in reversed(order):
    if node not in grads or not node.requires_grad:
        continue
    g = grads[node]
    if accumulate and not node.parents and node.grad is not None:
        node.grad = node.grad + g
    else:
        node.grad = g.copy()
    if node.vjp is None:
        continue
    for parent, contribution in zip(node.parents, node.vjp(g)):
        if parent.requires_grad:
            # 同一输入可能有多条路径或重复边，每份贡献都要相加。
            grads[parent] = grads.get(parent, 0) + contribution

```

seen 防止共享节点被重复遍历，但 `zip(node.parents, node.vjp(g))` 仍逐个处理输入位置。梯度字典中的加法负责合并不同路径，而不是简单用后一条路径覆盖前一条。

运行追踪 一个小图检查三种机制

令 $x = 3$, $u = x * x$, $L = u + u + x$, 则 $L = 21$, 预期 $\partial L / \partial x = 13$ 。乘法内部重复使用 x , 加法又重复使用 u , 最后还存在一条直接的 x 路径。这个例子能同时检查重复输入边、共享节点和梯度累加。

本实现每次 backward 使用一份新的临时梯度表，默认覆盖本次节点梯度。显式传入 `accumulate=True` 时，才把本次叶节点梯度加到原有结果。生产框架可能选择其他默认语义，例如常见训练循环需要在新批次之前清空参数梯度。应区分一次反向内部的多路径累加和多次反向之间的梯度累积。

一个稳定的损失也可以由基本操作组成

代码示例 由基本算子组合 log-sum-exp 与交叉熵

```

def logsumexp(x, axis=-1, keepdims=False):
    # 稳定移位不求导；完整 log-sum-exp 中对移位的依赖会抵消。
    maximum = Tensor(x.data.max(axis=axis, keepdims=True), requires_grad=False)
    result = (x - maximum).exp().sum(axis=axis, keepdims=True).log() + maximum
    if not keepdims:
        shape = list(result.data.shape)
        del shape[axis]
        result = result.reshape(tuple(shape))
    return result

def cross_entropy(logits, labels):
    n, classes = logits.data.shape
    target = np.eye(classes)[labels]

```

```
correct_score = (logits * target).sum(axis=1)
return (logsumexp(logits, axis=1) - correct_score).mean()
```

这里把移位用的最大值视为常量。为什么没有漏掉必须的梯度？因为

$$m + \log \sum_j e^{z_j - m}$$

对 m 的导数为 $1 - \sum_j p_j = 0$ 。稳定移位的变化在完整表达式中抵消，因此把它用作不求导的数值辅助量，仍可得到正确的 logits 梯度。这个论证依赖完整公式，不能据此随意把其他中间量停止求导。

8.8 保存值、原地修改与高阶导数

乘法反向需要前向输入，指数反向可使用前向输出。若为了省内存提前覆盖这些值，反向可能使用到另一个时刻的数据。实际框架需要通过保存张量、版本检查或明确限制，维护这种前后关系。

配套引擎使用只读数组保存节点值，参数更新时建立新的叶节点。它用 NumPy 数值执行局部 VJP，因此实现的是一阶反向；这些 NumPy 梯度本身没有继续记录为 Tensor 运算。

若希望计算高阶导数，必须让梯度计算也形成可微的程序。可以构建扩展图，再对梯度程序求导，也可以组合前向和反向模式。

Hessian-vector product 不必建立 Hessian

对标量 $L(x)$ ，令 $H = \nabla^2 L(x)$ 。很多二阶方法只需要 Hv ，而不是整个 $n \times n$ 矩阵。可以对梯度函数 $g(x) = \nabla L(x)$ 计算方向 v 的 JVP；在相应光滑条件下，也可以对 $\nabla L(x)^\top v$ 再求梯度。

需要保留前向中间值与需要记录梯度计算，是不同需求。仅保留已有图或缓冲区，并不会自动把一个原本不可微分的梯度实现变成高阶可微实现。

8.9 控制流与不可微操作的边界

动态图可以记录当前执行路径上的操作，包括循环实际展开的次数和已经选定的分支。它通常求的是这条路径所定义的局部可微计算，而不会自动为离散的分支选择生成平滑导数。

索引、排序、取整、布尔比较等操作需要分别说明数学意义与导数约定。有些操作对数据值可微而对索引不可微；有些操作分段可微；还有些需要任务设计中的特殊近似。自动微分可以正确组合已定义的规则，但不能替代对问题本身是否可微的判断。

第 9 章 计算图与训练框架：把算子组织成一个系统

自动微分引擎已经知道怎样记录依赖并传播梯度。要让它成为可训练的小框架，还需要组织参数、网络层、更新步骤与图的生命周期。同时，计算图不只服务于求导，它也为编译、内存复用和执行调度提供了统一表示。

9.1 数据、算子与参数承担不同责任

一个 Tensor 的数据表示至少需要说明形状、dtype、设备和存储布局；如果它参与求导，还需关联产生它的操作和输入节点。参数则是需要在训练过程中更新的特殊状态，不是某个中间结果恰好取了一个名字就自动成为参数。

对一个线性层，可以把职责拆成三部分：参数对象保存 W, b ；前向表达式计算 $XW + b$ ；自动微分机制为这些操作传播梯度。层本身不必再手工实现一份与基本算子重复的链式法则。

存储与 Tensor 视图也应区分。多个视图可以共享底层存储，同时拥有不同的形状、偏移和步长。释放一个视图不应无条件释放仍被其他对象使用的存储；修改共享存储则可能影响所有视图。框架的数据模型必须表达这些关系。

9.2 用已有算子训练一个小网络

下面建立一个两层网络：

$$H = \text{ReLU}(XW_1 + b_1), \quad S = HW_2 + b_2.$$

它使用前面的小型自动微分引擎。参数初始化采用与层输入规模有关的方差，避免一开始就让中间值过大或过小。

代码示例 层是参数与基本操作的组合

```
class MLP:
    def __init__(self, seed=8):
        rng = np.random.default_rng(seed)
        self.w1 = Tensor(rng.normal(size=(2, 8)) * np.sqrt(2/2))
        self.b1 = Tensor(np.zeros(8))
        self.w2 = Tensor(rng.normal(size=(8, 2)) * np.sqrt(2/8))
        self.b2 = Tensor(np.zeros(2))

    def __call__(self, x):
        # 第一层线性变换后做逐元素激活，再交给第二层。
        hidden = (x @ self.w1 + self.b1).relu()
        return hidden @ self.w2 + self.b2
```

```
def step(self, rate):
    for name in ("w1", "b1", "w2", "b2"):
        parameter = getattr(self, name)
        updated = parameter.data - rate * parameter.grad
        # 把新参数建立为叶节点，不把每轮优化更新连成无限增长的图。
        setattr(self, name, Tensor(updated))
```

这个实现将参数更新结果放入新的叶节点，切断上一轮训练计算的历史。若把更新写成受自动微分记录的普通表达式，并一直保留所有历史，图可能随着训练步数不断增长。需要对优化过程本身求导的元学习是另一种任务，届时才应有意识地保留对应路径。

代码示例 一个完整的小训练循环

```
def train(steps=500, rate=0.1):
    x = Tensor([[ -1, -1], [-1, 1], [1, -1], [1, 1]], requires_grad=False)
    labels = np.array([0, 1, 1, 0])
    model = MLP()
    history = []
    for _ in range(steps):
        logits = model(x)
        loss = cross_entropy(logits, labels)
        history.append(float(loss.data))
        # 先取得当前参数梯度，再执行本次更新。
        loss.backward()
        model.step(rate)
    return model, x, labels, history
```

四个输入对应 XOR 分类，标签为 [0,1,1,0]。使用配套参数时，500 步后的预期损失约为 0.020，预测标签为 [0,1,1,0]。这个例子检查的是框架各部分能否协作，不代表模型已经具备对真实数据泛化的能力。

训练循环中的逻辑顺序是：构建当前批次的前向图，生成损失，传播梯度，更新参数，再开始下一次前向。实际框架还会组织训练与评估模式、随机状态、学习率调度和持久化，但这些功能不应破坏这条基本依赖链。

参数梯度与中间梯度的寿命不同

中间梯度通常只为当前反向服务，可以在满足依赖后释放。参数梯度则可能需要保留到优化器读取，或用于跨微批次累积。上一轮已经用完的梯度若没有按约定清空，会与新一轮混在一起；过早清空，又会丢失本来需要累计的贡献。

用常见框架编写循环时，应核对其默认累积语义，而不是照搬另一份教学引擎的实现。是否保留一个图、是否保留中间值、是否累计叶节点梯度，是三个相关但不同的决策。

9.3 图表示的不只是函数调用顺序

计算图记录数据依赖。若 $u = A(x)$, $v = B(u)$, 则 B 必须等待 A 产生所需结果。没有直接数据边的两个操作, 也可能因为共享缓冲区、随机状态或外部副作用而需要保持顺序。

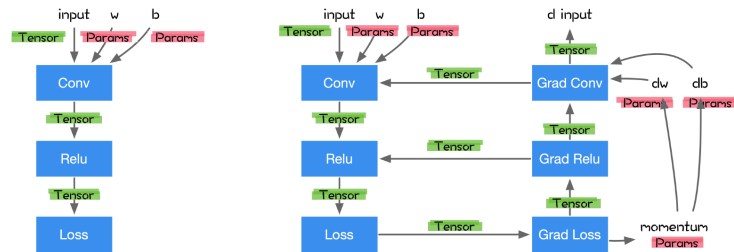


图 9.1 前向图与反向图包含不同的数据流；保存的前向结果会成为反向操作的输入。（课程第 10 讲，第 4 页）

因此, 图调度需要区分纯数值依赖与其他执行约束。两个节点在纸面上没有连线, 不足以证明可以安全并行。若它们读写同一个外部数组, 而图中没有表达这种别名关系, 优化器就可能做出错误决定。

一次展开的无循环数值计算常用有向无环图 (DAG) 表示。包含循环或条件的程序, 则可以在 IR 中显式保留控制流节点或区域, 也可以针对一次执行展开。不能把“计算图常用 DAG”理解为所有程序表示都必须把每个循环永久展开。

9.4 即时执行与先建图执行

即时执行 (eager) 在程序调用算子时就执行相应操作, 求导所需关系可以同时记录。这与 Python 的普通控制流配合自然, 便于检查中间结果。

先建图执行则先得到某种可分析的程序表示, 再安排运行。这样可以在执行前做跨算子优化、内存规划或部署转换, 但必须说明图适用于哪些输入、状态和控制流条件。

现代系统常同时使用两种方式: 用户按即时方式编程, 编译器捕获其中适合优化的部分, 再缓存并执行编译结果。动态图不等于无法编译, 静态图也不等于不能表示控制流; 关键在于具体的捕获、表示与执行机制。

9.5 图捕获的核心问题: 这次记录能否用于下一次

考虑一个根据输入值选择分支的函数:

```
def f(x):
    # 分支取决于数据值, 不能把一次追踪的选择永久当作常量。
    if x.sum() > 0:
        return 2 * x
    return x - 2
```

只记录一次正和输入所执行的操作, 可能得到“乘以 2”的图。但对负和输入, 原函数应走另一个分支。这个图不能无条件复用。

代码示例 用显式条件表达一个特化图的适用范围

```
def capture_one_branch(example):
    # 这里只记录示例执行时选择的分支。
    positive = bool(np.sum(example) > 0)
    # 复用这份特化图之前，检查输入仍满足当时的适用条件。
    def guard(x):
        return x.shape == example.shape and bool(np.sum(x) > 0) == positive
    def graph(x):
        return x * 2 if positive else x - 2
    return guard, graph
```

这是解释特化的最小模型，并不是某个框架追踪器的复刻。真实系统可以使用 shape、dtype、device、Python 状态等 guard，或把部分条件显式保留在图中。guard 不成立时，需要重新捕获、重新编译，或回到可处理该情况的执行路径。

数据依赖分支与形状依赖分支

“这个张量的维度是否为二维”与“它的元素和是否为正”不是相同的条件。前者可在许多情况下作为元数据条件处理；后者可能要求实际读取设备上的数据，并影响运行时控制流。形状也可以是动态的，因此不能把所有 shape 值都随意固定成第一次输入的常量。

追踪方法也有区别。有的方法根据真实输入执行并记录，有的方法使用符号代理对象；遇到无法表达的 Python 控制流时，可能拒绝追踪，而不是记录一个看似可用的图。阅读框架示例时，要先确认使用的是哪种机制。

graph break 为什么不一定是错误

无法捕获的操作可能使编译器把程序拆成多个图，之间由普通执行衔接。这仍可能得到正确结果，只是跨断点的优化机会减少。若要求整段程序成为一个图，可以使用相应严格模式帮助定位不支持的部分。

性能问题常来自频繁重新编译、小图过多或 CPU 与 GPU 来回同步。要观察实际形成了哪些图及其适用条件，不能仅凭代码外面包了一个编译入口，就认定整个训练步骤已经成为单个高效 kernel。

9.6 前向图与反向图可以一起优化

反向计算本身也是程序。若把前向和相应反向都表示成图，编译器便能分析哪些中间结果必须保留，哪些可以重算，以及哪些梯度操作可以融合。

例如前向计算 $u = \exp x$ ，反向需要 $\bar{x} = \bar{u} u$ 。可以保存 u ，也可以在反向时重新计算 $\exp x$ 。前者多占存储，后者多做计算。系统需要根据中间值大小、重算成本和整个训练路径做取舍。

先提取可微图，再把联合计算分成前向与反向部分，是现代编译训练路径中的一种常见组织方式。它与简单地“保留一个 Python 对象”不同：分割边界决定哪些值跨越前后向阶段存活，也直接影响内存和计算成本。

第 10 章 编译器前端：看见整个计算，才能跨算子优化

计算图把原来分散在 Python 调用中的依赖显式化。编译器前端要在保持语义的前提下，把这些信息转成适合分析和优化的表示。它主要回答：哪些计算可以预先完成，哪些结果可以共享，哪些算子可以合并，以及哪些存储可以复用。

10.1 IR 是一套明确的程序约定

中间表示 (IR) 应表达操作、输入输出、类型和必要属性。对张量程序，类型信息常包括 dtype、秩和形状约束；布局、设备与副作用也可能影响合法变换。

AST 更接近源代码的语法结构，计算图更直接地表达值之间的依赖。相同子表达式在语法树中可能出现两次，而在允许共享的图表示中可以对应同一节点。

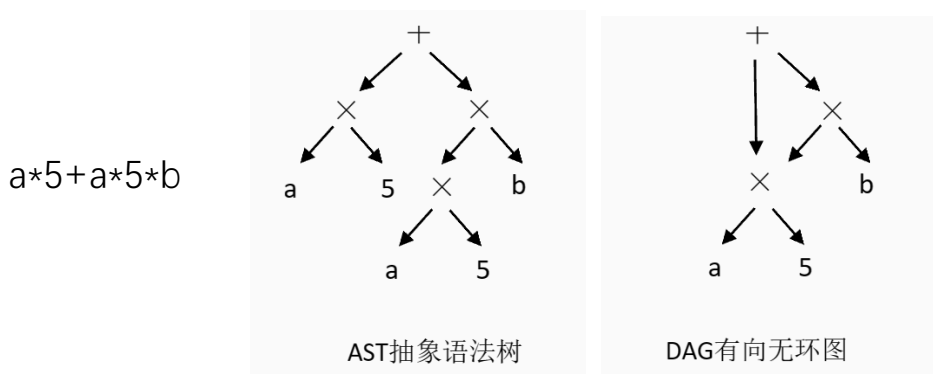


图 10.1 语法结构与共享数据依赖提供不同的观察方式。共享计算是否安全，还要检查类型、状态与副作用。（课程第 11 讲，第 13 页）

一个系统也可能使用多层 IR：高层保留矩阵乘法或卷积等语义，较低层显式描述循环、缓冲区和线程。过早把所有算子降成细碎指令，会丢失某些高层优化机会；只保留高层名字，又不足以安排具体硬件执行。

10.2 常量折叠与推理时的 BatchNorm 合并

若一段纯计算的全部输入在编译时已知，可以直接求出结果，运行时不必重复执行。这里“已知”是相对于某次编译或模型冻结状态而言：训练中的可更新权重不能因为当前有数值，就被永久当成常量。

一个重要例子是推理模式下的 BatchNorm。若固定均值 μ 、方差 v 、缩放 γ 与偏移 β ，则

$$\text{BN}(z) = \frac{\gamma}{\sqrt{v + \varepsilon}}(z - \mu) + \beta.$$

设 $z = Wx + b$ ，令 $a = \gamma/\sqrt{v + \varepsilon}$ ，逐输出通道整理可得

$$W' = aW, \quad b' = a(b - \mu) + \beta.$$

这样卷积或线性层的输出可直接成为原来 BN 后的结果。 aW 表示对每个输出通道的权重乘以对应缩放，而不是任意矩阵乘法。

训练模式下，BN 的统计量可能依赖当前批次；把它替换成固定参数变换通常会改变函数。即使是推理合并，也要核对 epsilon、布局、精度和相应模型状态。

10.3 公共子表达式与死代码消除

若两个纯操作的类型、输入、属性都相同，可以考虑只计算一次，让使用者共享结果。例如两处相同的 $x + y$ 。不能只比较操作名称：沿第 0 轴求和与沿第 1 轴求和是不同操作；随机数生成即使命令相同，也不一定应被共享。

死代码消除从需要的输出反向标记依赖，删除不影响输出的纯计算。有副作用的操作需要额外保留规则，例如状态更新、必要通信或外部写入，不能仅凭结果未被读取就删除。

先定义一个小型 IR 和执行器。输入值由字典提供，节点保存操作名称、输入名字与可选常量：

```
from dataclasses import dataclass
import numpy as np

@dataclass
class Node:
    name: str
    op: str
    inputs: tuple = () # 保存输入名字，真正数值由执行环境提供
    value: object = None

def calculate(op, values):
    if op == "add": return values[0] + values[1]
    if op == "mul": return values[0] * values[1]
    if op == "relu": return np.maximum(values[0], 0)
    raise ValueError(op)

def execute(nodes, inputs, output):
    env = dict(inputs)
    for node in nodes:
        env[node.name] = (np.asarray(node.value) if node.op == "const" else
                           calculate(node.op, [env[k] for k in node.inputs]))
    return env[output]
```

代码示例 一个只处理纯算子的简化图优化器

```
def optimize(nodes, output):
    aliases, constants, available, result = {}, {}, {}, []
    for node in nodes:
        # 先把已合并节点替换成代表名字，后续操作才能继续识别共享。
        args = tuple(aliases.get(k, k) for k in node.inputs)
        if node.op == "const":
            value = np.asarray(node.value)
            current = Node(node.name, "const", value=value)
            # 只有输入均为已知常量时，才在优化阶段直接求值。
        elif all(k in constants for k in args):
            value = calculate(node.op, [constants[k] for k in args])
            current = Node(node.name, "const", value=value)
        else:
            current = Node(node.name, node.op, args)
        if current.op == "const":
            v = np.asarray(current.value)
            key = ("const", v.dtype.str, v.shape, v.tobytes())
        else:
            key = (current.op, current.inputs)
        # 纯操作的相同表达式复用已有节点。
        if key in available:
            aliases[node.name] = available[key]
            continue
        available[key] = current.name
        result.append(current)
        if current.op == "const":
            constants[current.name] = np.asarray(current.value)
    output = aliases.get(output, output)
    # 从需要的输出反向找依赖，删除未使用的纯计算。
    required = {output}
    for node in reversed(result):
        if node.name in required:
            required.update(node.inputs)
    return [n for n in result if n.name in required], output
```

```
nodes = [Node("a", "add", ("x", "y")),
         Node("b", "add", ("x", "y")),
         Node("out", "add", ("a", "b"))]
optimized, output_name = optimize(nodes, "out")
feed = {"x": np.array([1., 2.]), "y": np.array([3., 4.])}
print(execute(optimized, feed, output_name).tolist()) # 预期[8.0, 12.0]
```

原来两个相同的加法节点 a、b 合并为一个，out 则两次读取这同一个结果。优化后的图仍要把它加两次，不能连两条使用边也一起删掉。

代码先把被共享的输入名字替换成已有代表；再对全常量操作求值；用操作和输入生成公共表达式键；最后从指定输出反向筛选。真实 IR 还需把轴、布局、类型及副作用等属性纳入分析。本例只定义常量、加法、乘法和 ReLU，因此把适用范围保持得很窄。

实数恒等式未必是浮点程序恒等式

把 $x \cdot 0$ 直接替换成 0，在 x 为 NaN 或无穷大时可能改变结果。重排加法会改变舍入顺序，融合乘加也可能改变中间舍入。有符号零、溢出与舍入模式同样可能影响严格等价性。

优化必须选择并遵守明确的数值契约。若允许一定误差或放宽 IEEE 语义，应让这个选择属于系统约定，而不是因为一个代数式看起来更短就默认成立。

10.4 算子融合：省掉哪些实际工作

设有

$$u = x + b, \quad v = \text{ReLU}(u), \quad y = v \odot s.$$

三个独立 kernel 可能分别写出、再读入中间数组。若把它们融合，每条线程可以在寄存器中完成对应的局部计算，只写最终结果，从而减少中间流量和启动开销。

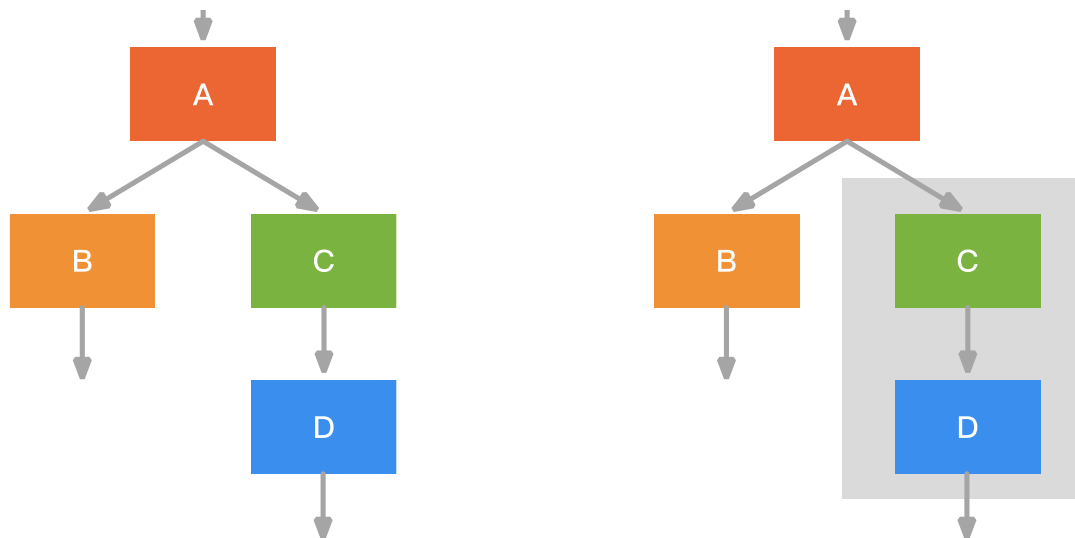


图 10.2 生产者与消费者可以合并执行。分叉结构中的某个结果若仍被其他路径需要，则必须继续保存或重新计算。（课程第 11 讲，第 27 页）

融合并非总能获益。它可能增加寄存器需求，降低驻留度，扩大代码体积，或者破坏原本成熟库算子的高效调度。归约、排序和复杂布局变换也会改变可融合边界。判断依据应是依赖、实现能力和成本模型，而不是尽量把所有节点合成一个。

多使用者让融合变得更有取舍

若 u 同时被 v 和 w 使用，把 u 融入 v 之后， w 仍需要它。可以继续物化 u ，也可以在两条分支中分别重算，或构造多输出融合。重算节省一次存储，却增加运算；多输出融合又可能影响并行安排。

例如 A 的结果分给 B、C，两支最后都进入 D，而且不存在绕过 D 的其他出口。从 A 到出口的每条路径都经过 D，就称 D 后支配 A。这样的汇合结构有助于确定完整的候选融合区域；若还存在绕行使用者，就不能只盯着其中一条链。

支配树或后支配树提供这些结构关系，最近公共祖先查询应在相应树上解释。图分析先确认依赖与使用者，再结合算子可融合条件和成本决定边界，不能把一次普通 DFS 当成完整的融合决策。

前向能融合，反向需要什么也要考虑

推理时可删除的中间结果，在训练中可能被反向使用。例如激活函数的反向需要输入或输出的某种信息。融合的训练算子可以保存必要状态，或设计重算路径。只看前向少了多少个 kernel，还不足以评价完整训练步骤。

10.5 内存规划：复用已经结束的生命期

设某个临时张量在步骤 s 产生，最后一次使用在步骤 e ，可用区间 $[s, e]$ 表示其存活范围。两个区间互不重叠时，可以考虑复用同一物理缓冲区，前提是容量、对齐、设备及其他要求相容。

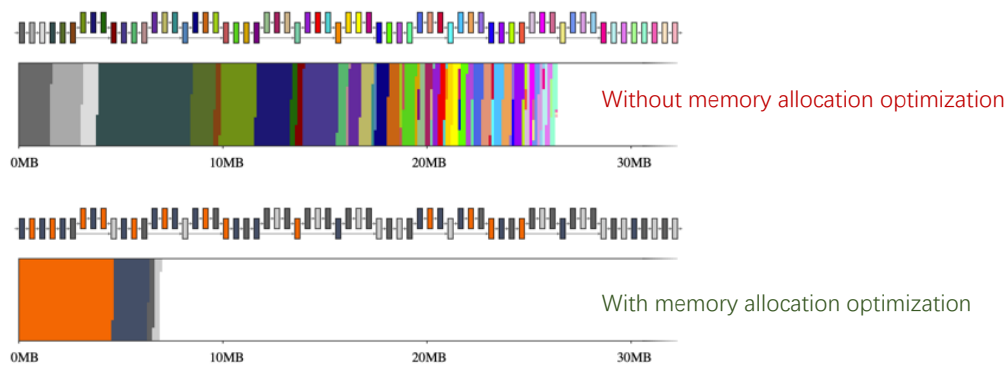


图 10.3 课件中的内存规划对比展示了中间结果复用的潜力。具体节省取决于图、调度与保存策略。（课程第 11 讲，第 42 页）

代码示例 按已知生命期分配可复用缓冲区

```
def assign_buffers(intervals):
    """Each interval is (name, first_step, last_step, required_bytes)."""
    buffers, assignment = [], {}
    for name, first, last, size in sorted(intervals, key=lambda x: x[1]):
        free = [i for i, b in enumerate(buffers)
                # 同一步还在读取的输入不能直接被新输出覆盖。
                if b["last"] < first and b["capacity"] >= size]
        if free:
            index = min(free, key=lambda i: buffers[i]["capacity"])
        else:
            index = len(buffers)
            buffers.append({"capacity": size, "last": -1})
        assignment[name] = index
```



```

buffers[index]["last"] = last
assignment[name] = index
return assignment, sum(b["capacity"] for b in buffers)

```

例如 *a* 在步骤 0 到 2 存活、需 128 字节，*b* 在步骤 1 到 3 存活、需 64 字节，*c* 在步骤 3 到 5 存活、需 128 字节。*c* 可以复用 *a* 已经结束的缓冲区，因此总池容量为 192 字节，而不是 320 字节。条件 `last < first` 采用保守约定：一个操作执行时，输入仍活着，输出需要先获得存储，不能因为输入“在这一步最后使用”就立即覆盖。原地计算能否允许复用，必须另行证明算子支持读写别名。

运行追踪 同一步的读取与写入仍可能重叠

若 *a* 在步骤 2 最后被读取，*c* 也在步骤 2 产生，普通分配器不能直接让 *c* 覆盖 *a*。只有证明该操作的原地实现不会破坏尚未读取的数据，才可放宽这个限制。

当多个流或设备异步执行时，生命期不是简单的主机提交编号。一个缓冲区在主机看来“已经提交最后一次使用”，设备可能还没真正读完。运行时必须用事件或等价依赖保证复用发生在使用结束之后。

省内存可能牺牲并行性

两个分支若使用独立缓冲区，可以同时执行。若为节省内存强迫它们复用同一段存储，就可能需要增加顺序依赖。优化内存峰值与优化执行时间是相关但不同的目标，系统需要明确取舍。

10.6 布局变换与执行分派

布局决定逻辑元素怎样对应物理地址。转换到更适合 SIMD（单指令、多数据）、Tensor Core 或特定库算子的排列，可能提高计算效率，但需要支付转换成本。

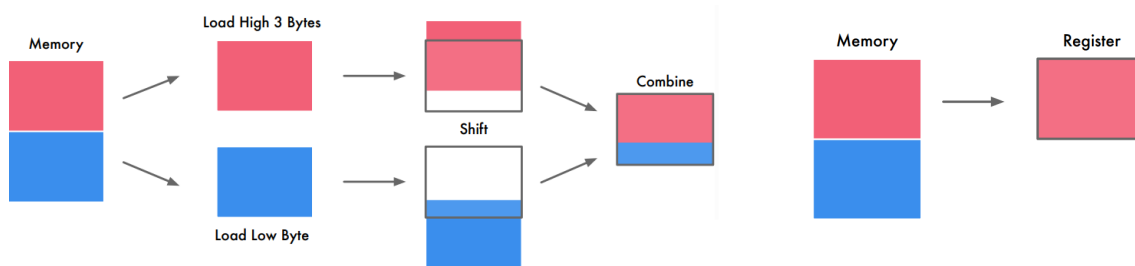


图 10.4 按硬件访问方式重新组织数据，有时可以减少零散加载与重排指令。转换本身也应计入完整成本。（课程第 11 讲，第 37 页）

如果前后多个算子都能接受同一种新布局，就有机会把转换移到更外层或彼此抵消；若每个算子前后都转换一次，局部加速可能被数据搬运抵消。元数据转置与真实重排尤其需要区分。

第 11 章 优化器与数值精度：梯度算出来以后怎样更新

自动微分给出了当前位置的梯度，却没有决定应该走多远、怎样利用过去的方向，或怎样在有限精度下保留微小更新。优化器负责这些问题。本章先用一个二维二次函数解释步长与曲率，再把动量、Adam 和混合精度落实到可检查的更新过程。

11.1 梯度下降：方向正确，步子仍可能太大

考虑

$$L(x_1, x_2) = \frac{1}{2}x_1^2 + 50x_2^2, \quad \nabla L = (x_1, 100x_2).$$

第二个方向更陡。普通梯度下降为

$$x'_1 = (1 - \eta)x_1, \quad x'_2 = (1 - 100\eta)x_2.$$

若 $\eta = 0.01$ ，第二个分量一步变成 0；若 $\eta = 0.03$ ，它每步乘以 -2 ，符号交替且幅度扩大。负梯度是局部下降方向，不代表沿它走任意距离都下降。

对正定二次函数 $L(x) = \frac{1}{2}x^T Hx$ ，沿 Hessian 特征向量分解后，每个方向都类似地乘以 $1 - \eta\lambda_i$ 。要让这些分量收缩，需要

$$0 < \eta < 2/\lambda_{\max}.$$

若最大与最小特征值相差很大，步长要照顾陡方向，平缓方向就可能进展缓慢。这给出了条件数影响收敛的直觉。

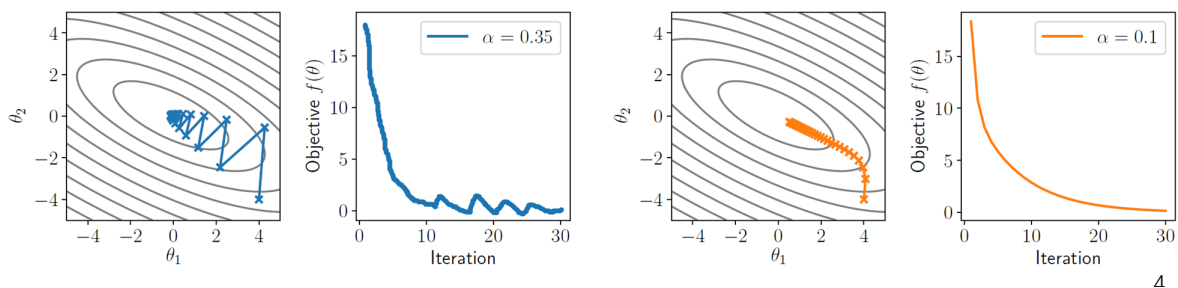


图 11.1 同一个二次目标上，不同步长会改变迭代轨迹与损失下降过程。（课程第 12 讲，第 4 页）

小批次梯度改变了计算与噪声的平衡

数据损失为 $L(\theta) = N^{-1} \sum_i \ell_i(\theta)$ 。用均匀抽样批次的平均梯度估计全量梯度，可以减少每步成本；在相应抽样条件下，它是全量梯度的无偏估计。但不同批次仍会产生不同方向和大小，更新轨迹因此具有随机性。

批大小变化还会改变每个 epoch 的更新次数。比较方案时，不仅要看单步损失，也要说明样本量、更新次数、学习率与时间成本，否则容易把不同训练预算混在一起。

11.2 动量：把过去的方向保留下来

一种常见约定是

$$v_t = \beta v_{t-1} + g_t, \quad \theta_{t+1} = \theta_t - \eta v_t, \quad v_0 = 0.$$

如果梯度连续朝同一方向，速度逐渐积累；如果某个方向的梯度正负摆动，累积可能部分抵消。动量由此改变不同方向上的时间响应，而不是简单地每步多乘一个固定系数。

运行追踪 两步更新看清状态的作用

设一维参数初值为 3，学习率 0.1， $\beta = 0.9$ ，前两步梯度都为 2。第一步 $v_1 = 2$ ，参数变成 2.8；第二步 $v_2 = 0.9 \times 2 + 2 = 3.8$ ，参数变成 2.42。第二步的更新不仅由当前梯度 2 决定，还包含保留下来的速度。

也有实现使用指数平均 $m_t = \beta m_{t-1} + (1 - \beta)g_t$ 。两种记号的有效缩放不同，不能把学习率数值直接照搬而忽略约定。

Nesterov 的一种写法先观察沿旧速度前进后的位置：

$$g_t = \nabla L(\theta_t - \eta \beta v_{t-1}), \quad v_t = \beta v_{t-1} + g_t, \quad \theta_{t+1} = \theta_t - \eta v_t.$$

它在预计要到达的位置获得修正信息。实际库也可能使用等价或相关的重写形式，比较代码时应同时检查梯度计算位置、速度定义与学习率位置。

11.3 二阶信息：校正不同方向的曲率

在当前位置附近，用二阶模型近似损失：

$$L(\theta + \Delta) \approx L(\theta) + g^\top \Delta + \frac{1}{2} \Delta^\top H \Delta.$$

若这个局部模型的 Hessian 正定，其最小点满足

$$H \Delta = -g.$$

这就是 Newton 步。实现时通常求解线性方程，而不是显式计算 H^{-1} 。对于固定正定二次函数，完整 Newton 步可以直接到达极小点；对一般非线性函数，局部模型的有效范围有限，可能需要阻尼或线搜索。

非凸问题中，Hessian 可能不定或奇异，解出的方向未必下降。自动微分能够帮助计算二阶信息或 Hessian-vector product，困难主要在存储、计算成本和如何使用这些信息，而不是“自动微分无法求 Hessian”。

BFGS 通过两次梯度之间的变化学习曲率

令

$$s_t = \theta_{t+1} - \theta_t, \quad y_t = g_{t+1} - g_t.$$

对局部二次模型，梯度变化近似等于 Hessian 乘参数变化。因此近似矩阵应满足割线条件 $B_{t+1} s_t = y_t$ 。若用 H_t 表示逆 Hessian 近似，则逆 BFGS 更新为

$$H_{t+1} = (I - \rho s_t y_t^\top) H_t (I - \rho y_t s_t^\top) + \rho s_t s_t^\top, \quad \rho = (y_t^\top s_t)^{-1}.$$

这个结构为何符合割线信息？把更新式右乘 y_t ，右侧括号使 $(I - \rho y_t s_t^\top) y_t = 0$ ，第一项消失；第二项成为 $\rho s_t (s_t^\top y_t) = s_t$ 。所以得到 $H_{t+1} y_t = s_t$ ，即梯度变化经逆曲率近似还原为参数变化。

正定性需要相应前提，典型条件是原近似正定且 $y_t^\top s_t > 0$ 。曲率条件不满足时，要使用合适的跳过、阻尼或线搜索策略，不能仍盲目套公式。

L-BFGS 只保存最近若干 (s, y) ，通过两遍循环计算 $H_t g_t$ ，避免存储完整矩阵：

代码示例 从有限历史计算一个搜索方向所需的乘积

```
def lbfgs_product(grad, history, scale=1.0):
    q = grad.copy()
    saved = []
    for s, y in reversed(history):      # 先从最近的曲率信息向前消去
        curvature = y @ s
        if curvature <= 0:
            raise ValueError("positive curvature is required")
        rho = 1.0 / curvature
        alpha = rho * (s @ q)
        q -= alpha * y
        saved.append((s, y, rho, alpha))
    r = scale * q                      # 初始逆 Hessian 近似为 scale*I
    for s, y, rho, alpha in reversed(saved):
        beta = rho * (y @ r)
        r += s * (alpha - beta)
    return r                          # 搜索方向取负号：-H*g
```

返回值近似 $H_t g_t$ ，搜索方向再取负号。历史数量控制存储与曲率信息的取舍；噪声很大的批次梯度也会影响这些差分信息的可靠性。

11.4 自适应缩放：每个坐标使用不同尺度

Adagrad 累计每个坐标的梯度平方：

$$r_t = r_{t-1} + g_t \odot g_t, \quad \theta_{t+1} = \theta_t - \eta \frac{g_t}{\sqrt{r_t} + \varepsilon}.$$

长期梯度较大的坐标会得到较小的有效步长。但平方和只增不减，某些任务中后期步长可能越来越小。

RMSProp 改成平方梯度的指数平均：

$$r_t = \beta r_{t-1} + (1 - \beta) g_t \odot g_t.$$

这样状态更偏重近期尺度。这些操作通常按元素进行，分母不是把整个梯度向量求一个范数之后统一除掉。状态保存的是平方梯度的估计，不能不加区别地称为梯度方差。

11.5 Adam 与 AdamW：状态、偏差修正与权重衰减

Adam 同时维护梯度的一阶平均和平方平均：

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t, \quad v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2.$$

从零初始化时，前几步累积权重还不足 1，所以使用

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}, \quad \theta_{t+1} = \theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon}.$$

这种修正补偿零初始化的影响，并不保证在任意非平稳梯度序列上消除所有估计偏差。

AdamW 将参数衰减与梯度自适应缩放分开：

$$\theta_{t+1} = (1 - \eta\lambda)\theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon}.$$

代码示例 每个参数需要保存自己的优化器状态

```
class AdamW:
    def __init__(self, shape, rate=0.01, beta1=0.9, beta2=0.999,
                 eps=1e-8, decay=0.0):
        self.m = np.zeros(shape)      # 一阶状态：平滑后的梯度
        self.v = np.zeros(shape)      # 二阶状态：平滑后的梯度平方
        self.t = 0
        self.rate, self.beta1, self.beta2 = rate, beta1, beta2
        self.eps, self.decay = eps, decay

    def step(self, weight, grad):
        self.t += 1
        self.m = self.beta1*self.m + (1-self.beta1)*grad
        self.v = self.beta2*self.v + (1-self.beta2)*grad*grad
        # 补偿从零状态开始时累计权重不足1的影响
        m_hat = self.m / (1-self.beta1**self.t)
        v_hat = self.v / (1-self.beta2**self.t)
        # 权重衰减与梯度的自适应缩放分开进行
        return ((1-self.rate*self.decay)*weight
                - self.rate*m_hat/(np.sqrt(v_hat)+self.eps))
```

为什么 L2 惩罚与 AdamW 不相同

若目标中加入 $\frac{\lambda}{2}\|\theta\|^2$ ，梯度会多出 $\lambda\theta$ 。对普通 SGD，这可整理成相同形式的乘法衰减。对 Adam，若把这一项放入梯度，它也参与一阶、二阶状态更新并被自适应缩放，通常不再等价于独立衰减。

以初值 10、数据梯度 1、学习率 0.01、衰减系数 0.1 为例，忽略极小 epsilon，第一步耦合 L2 的 Adam 把梯度 2 归一化后，参数约为 9.99；AdamW 单独衰减 0.01，再做约 0.01 的梯度更新，得到约 9.98。这个小例子比只记两个算法名字更能说明差别。

11.6 训练策略仍要与模型和数据配合

学习率预热、衰减与梯度裁剪是不同工具。预热逐渐增加初始步长；衰减降低后期步长；范数裁剪则在某一步梯度过大时限制更新信号：

$$g \leftarrow g \min\left(1, \frac{c}{\|g\|}\right) \quad (\|g\| > 0).$$

裁剪会改变优化方向的尺度，不能用它长期掩盖本来就不正确的梯度实现。

初始化还要避免层间信号迅速放大或缩小，以及多个隐藏单元因完全对称而学到相同表示。Xavier 与 He 初始化等方法从方差传播作近似分析，分别适应相应激活与连接假设，不是随意给权重乘一个小数。

正则化关注泛化，而不仅是使当前训练损失下降。早停通常依据验证指标与耐心窗口等规则，不应简单等同于“相邻两次参数变化很小”。验证集用于选择策略，最终测试集应保留作独立评估。

归一化与 Dropout 改变了什么

BatchNorm 通常按每个通道聚合批次和空间轴，LayerNorm 通常在单个样本的指定特征轴上归一化，GroupNorm 则在每个样本的通道组内聚合。它们使用的轴不同，因此对批大小、布局和训练推理模式的依赖也不同。

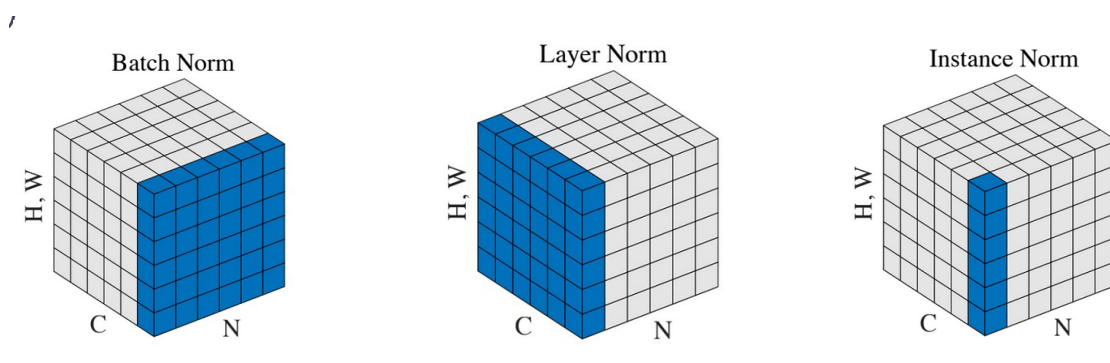


图 11.2 归一化统计的范围由所选轴决定。蓝色区域展示不同方式聚合的元素集合。（课程第 12 讲，第 42 页）

反向还需包含均值、方差对输入的依赖，不能把归一化只当成除以一个永远固定的常量。推理时采用固定统计量的 BN 是前端折叠中讨论的特殊情形。

训练中的 inverted dropout 可写为 $y = m \odot x / (1 - p)$ ，其中 m_i 以概率 $1 - p$ 取 1；推理时不再按同样方式随机屏蔽。这保证局部操作的期望尺度，不表示整个非线性网络的随机输出与确定输出逐点等价。

11.7 混合精度：表示范围与表示精细程度

float32 (FP32)、float16 (FP16) 与 bfloat16 (BF16) 都不能表示所有实数。指数位主要影响范围，尾数位主要影响相邻可表示数之间的精细程度。

格式	指数位	显式尾数位	主要取舍
float32	8	23	范围和精度较宽，存储成本较高
float16	5	10	存储较少，但动态范围较窄
bfloat16	8	7	范围接近 float32，尾数更短

表 11.1 同样 16 位的两种格式，解决的范围与精度问题不同。

混合精度不是把所有数组统一改成 float16。通常根据算子和硬件选择计算精度，在必要位置保留较高精度的累加、参数主副本或优化器状态。若一个参数已经很大，微小更新小于其可表示间距，直接在低精度参数上相加就可能丢失更新。

- 混合精度，避免大数吃小数问题

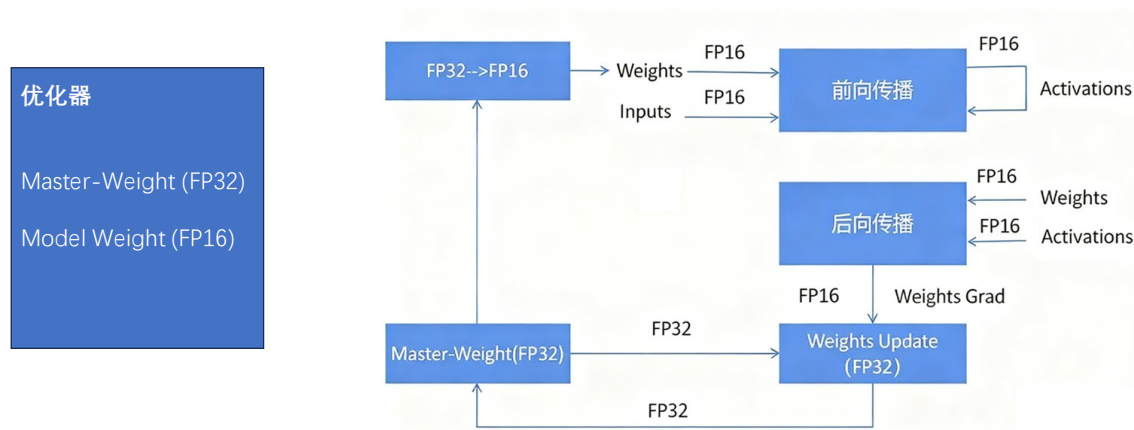


图 11.3 一种混合精度数据路径：低精度计算与较高精度参数更新分别承担不同职责。（课程第 12 讲，第 52 页）

损失缩放为什么有用

把损失乘以正数 s ，根据链式法则，反向梯度也乘以 s 。这样一些原本过小、会在低精度中下溢的中间梯度可以进入可表示范围。更新前再把梯度除以 s ，恢复原尺度。

例如 10^{-8} 直接转成 float16 可能成为 0；先乘 65536，再转成 float16，最后以高精度除回，可以保留一个接近原值的结果。缩放也可能让原本很大的梯度溢出，因此需要检查非有限值并按策略调整尺度。

代码示例 先还原尺度，再检查、裁剪和更新

```
def scaled_step(weight, optimizer, scaled_grad, scale, clip_norm):
    grad = np.asarray(scaled_grad, dtype=np.float64) / scale # 先还原真实尺度
    if not np.all(np.isfinite(grad)):
        return weight.copy(), False # 本步跳过，不推进动量与时间步
    norm = np.linalg.norm(grad)
    if norm > clip_norm:
        grad *= clip_norm / norm # 再按照未缩放梯度的范数裁剪
    return optimizer.step(weight, grad), True
```

若梯度为 [3,4]，范数为 5，阈值为 1，裁剪后应为 [0.6,0.8]。假设损失缩放 1000 倍，应先把 [3000,4000] 除回，再裁剪；若先裁剪再除回，就会把真实梯度压得小 1000 倍。

跳过溢出的更新时，参数、动量和优化器时间步也应遵循一致的跳过语义。缩放只能改善相应数值范围，不能修正原本错误的算子或梯度公式。

11.8 多个目标与约束：先明确优化的对象

若目标为 $L = \sum_i w_i L_i$ ，梯度也是相应加权和。权重不仅表达偏好，还受到各损失尺度和单位影响。两个梯度内积为负时，一个任务的下降方向可能伤害另一个任务；正交投影或其他多目标方法需要明确其具体优化准则，不能仅靠“把方向正交化”保证所有目标同时改进。

硬约束也有不同实现方式。例如区间约束可以在更新后投影回可行集合；某些结构约束可以通过参数化天然满足。选择惩罚、投影还是结构化参数化，取决于可行性要求与计算成本。优化器只有在目标、状态和约束都明确以后，才是一套可解释的更新规则。

第 12 章 编译器后端：把一个算子变成硬件上的工作

前端可以把一串计算整理成更好的图，但图中的 MatMul 仍只是一个数学操作。后端还要决定循环顺序、分块尺寸、存储层次、指令和线程安排。本章把第五章手写 CUDA 的经验转成编译器视角：同一个公式可以有許多实现，但只有满足依赖和数值契约的变换才是候选。

12.1 计算定义与执行安排分开

矩阵乘法定义了

$$C_{ij} += A_{iq}B_{qj}.$$

它没有唯一规定三重循环的嵌套顺序，也没有规定一个线程计算多少输出、数据何时进入共享内存。把这些执行选择称为 schedule，可以先保持计算定义，再改变执行安排。

例如行优先 CPU 参考中，可把循环写成 i, q, j ，让内层沿 B 和 C 的连续列方向前进：

代码示例 外提一个可复用元素，改变内层访问方向

```
def matrix_product_ikj(a, b):
    m, k = a.shape
    n = b.shape[1]
    c = np.zeros((m, n))
    for i in range(m):
        for q in range(k):
            value = a[i, q]          # 对这一行输出复用同一个 A 元素
            for j in range(n):
                c[i, j] += value * b[q, j]  # B 与 C 的列方向连续
    return c
```

这里每个 C_{ij} 的归约仍按相同 q 顺序进行，输出不依赖其他输出的中间值。这个 Python 程序用来说明顺序与地址关系，不表示解释器循环已经变成硬件向量指令。

12.2 循环变换必须尊重依赖

常见变换包括交换循环、分块、展开、融合和拆分。先问“会读到哪个时刻的值”，再问“是否更局部”。

考虑如下递推：

```
for i in range(1, rows):
    for j in range(cols-1):
        a[i, j] = a[i-1, j+1] + 1  # 依赖上一行右边的位置
```

外层按行前进，能保证上一行已经更新。若把 j 放到外层且仍从小到大遍历，程序可能在上一行右侧还没更新时就读取它。配套例子中，原顺序得到 $a[2,0]=4$ ，错误交换后得到 1。循环体看起来一样，执行的函数却已经改变。

- **Reorder:** switch loop nodes

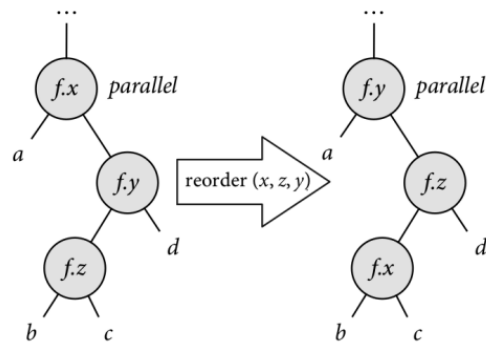


图 12.1 循环交换改变调度树中的嵌套关系；每次变换都需检查原有依赖。（课程第 13 讲，第 29 页）

多面体模型提供一种检查语言

对规则仿射循环，可以把迭代实例写成整数点 (i, j) ，把边界写成线性约束，把访问和依赖写成点之间的关系。执行顺序是给每个点分配一个时间坐标。

上例依赖 $(i-1, j+1) \rightarrow (i, j)$ 。按 (i, j) 字典序执行时，来源在目标之前；按 (j, i) 执行时，来源的第一坐标反而更大，便违反依赖。这个模型帮助系统搜索合法顺序，但动态索引、复杂控制流和非仿射访问需要其他分析或保守处理。

12.3 分块、展开与向量化分别改变什么

分块把大循环拆成外层块编号与内层块内编号。数据在小工作集内多次复用以后，再换下一个块。tile 大小应结合缓存或共享内存容量、并行性和边界浪费选择。

展开把若干迭代的操作显式放在一起，减少循环控制并暴露更多指令调度机会。它也可能增加代码体积和寄存器压力，不保证展开越多越快。

向量化让一条相应指令处理多个数据元素。必须处理数据依赖、对齐、向量宽度和不足一个向量的尾部；不能仅把标量指针重新解释成宽向量指针就认定访问合法。

张量化利用专门的小块矩阵指令

Tensor Core 等单元把一组矩阵乘加映射成专门指令。编译器需要把计算块组织成硬件支持的尺寸、dtype、布局与参与线程形式，再建立相应加载、累加和写出路径。

这不是把每条标量乘法简单换个名字。一组线程往往合作持有矩阵 fragment，单个线程看到的 fragment 元素布局也不能随意当作普通二维数组索引。学习这类接口时，应先确认合作范围和数据契约，再读具体的加载与矩阵乘加调用。

12.4 存储层次与双缓冲

第五章的 tile 复用减少重复的全局读取。进一步还可以用寄存器保存一个线程负责的多个累计结果，用不同共享缓冲区轮换，让下一批加载与当前计算有重叠机会。

双缓冲的基本流程是：加载块 0，开始计算块 0；同时准备另一缓冲区的块 1；确认块 1 加载完成且旧缓冲区不再被使用，再交换角色。异步复制接口只是提供建立这种流水线的能力，正确性仍依赖完成等待与覆盖边界。

硬件支持、指令版本和对齐要求都影响可用方法。仅仅升级 Toolkit，不等于任意 GPU 都具备相同的异步加载或矩阵指令能力。

原理剖析 从时间表看缓存的寿命

一个缓冲区何时可以重用，取决于它的最后一个消费者何时完成，而不是生产者何时发出了加载指令。同步、事件和流水线阶段必须围绕这个实际依赖组织。

12.5 自动调优：搜索的是有限且受约束的实现空间

手工优化时选择 tile、线程数、循环顺序和融合范围；自动调优把这些选择参数化，用规则或成本模型筛选，再测量部分候选。调优并不是无条件枚举所有程序，也不能保证在任意预算内找到全局最优。

一个实用搜索过程可以是：

1. 根据算子形状、dtype 和硬件能力产生合法候选。
2. 先排除超出共享内存、寄存器或线程约束的配置。
3. 对候选做数值验证，再测量代表性输入。
4. 记录硬件、软件版本、形状及配置，把结果用于相同条件或适当泛化范围。

代价模型可能用已知特征预测执行时间，但最终仍受测量噪声与搜索空间覆盖影响。只针对一个方阵调出的配置，不一定适合小批次、细长矩阵或动态尺寸。

12.6 减少提交开销，不等于把整个网络变成一个 kernel

有些任务算得很快，CPU 逐次提交小 kernel 的成本却占了很大比例。把稳定的任务依赖预先组织并重放，可以减少重复的解释和提交工作。

需要区分深度学习计算图与 CUDA Graph：前者主要表达张量运算和求导语义，后者表达 CUDA 工作提交及其依赖。一个框架编译结果可能借助后者执行，但二者不是同一层表示。

即时执行也可以显式使用并发流，图模式也可能生成多个 kernel 或调用库函数。因此，“eager 一定完全串行”“graph 一定是单个 kernel”都不能作为判断方法。应观察实际的依赖、提交与执行轨迹。

12.7 训练内存：参数只是其中一部分

设模型有 P 个参数。若使用 16 位模型权重、32 位梯度、32 位主权重副本和两份 32 位 Adam 状态，仅这些状态就需要

$$2P + 4P + 4P + 8P = 18P \text{ 字节.}$$

若梯度也以 16 位存储，同样的账目为 $16P$ 字节。真实实现可能没有单独主副本，或采用分片、压缩与其他状态格式，因此应列清楚各项，而不是记一个通用“每参数多少字节”。

还要加上前向激活、反向临时量、通信缓冲区、算子工作空间、分配器保留空间等。推理也并非总能仅靠两个激活缓冲区循环：分支、跳连、长寿命输出以及 KV cache 都可能延长存活范围。

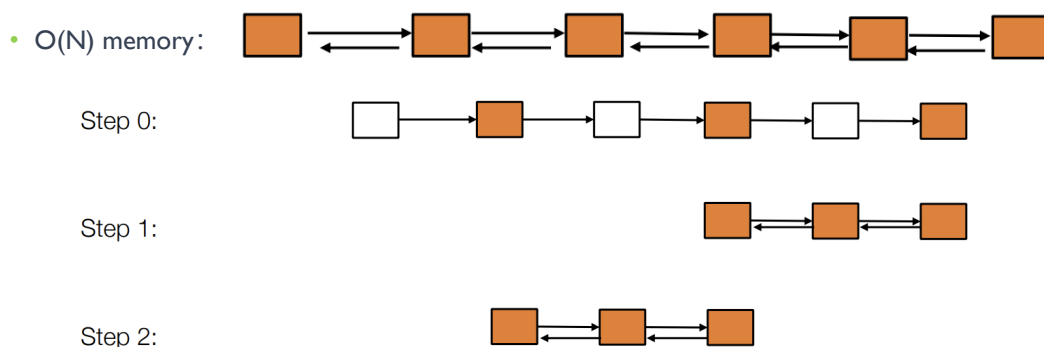
12.8 激活重算：用额外计算换内存

反向可能需要许多前向中间值。对一条含 N 层的简单链，若每层都保存一个等大激活，内存随 N 增长。可以每隔 K 层保存一个检查点；反向处理某段时，从最近检查点重做这一小段前向，再计算它的梯度。

粗略地说，保存边界需要 $O(N/K)$ 个激活，当前重算段需要 $O(K)$ 个，合计

$$O(N/K + K).$$

在这个等大小链模型中， K 取约 $\sqrt{N}$ 能平衡两项。真实网络中层大小和重算成本不一致，需要更细的计划。



- Only checkpoint colored nodes (step 0)
- Recompute the missing intermediate nodes in small segments (step 1, 2)

图 12.2 保存部分边界值，需要反向时再逐段重算中间激活。（课程第 13 讲，第 67 页）

代码示例 只在处理当前段时重建其前向值

```
def checkpoint_chain(weights, x, segment):
    saved = {0: x}
    h = x
    for i, w in enumerate(weights):
        h = np.tanh(w*h)
        if (i+1) % segment == 0 or i+1 == len(weights):
            saved[i+1] = h          # 只保留分段边界
    boundaries = sorted(saved)
    grad = np.zeros_like(weights)
    adjoint = 1.0
    peak_values = len(saved)
    for start, end in reversed(list(zip(boundaries[:-1], boundaries[1:])))
        local_values = [saved[start]]
```



```
for i in range(start, end):    # 反向需要这一段时，重新执行该段前向
    local_values.append(np.tanh(weights[i]*local_values[-1]))
peak_values = max(peak_values, len(saved)+len(local_values))
for i in reversed(range(start, end)):
    j = i-start
    local = adjoint * (1-local_values[j+1]**2)
    grad[i] = local * local_values[j]
    adjoint = local * weights[i]
return h, grad, peak_values
```

配套例子使用 100 层标量链，每 10 层存一个边界。与保存 101 个链上值相比，这份简化实现同时保留的值约为 22 个，预期梯度保持相同。这个数字只统计示例中的激活值，不包含权重、程序对象或框架额外空间。

重算必须对应原来的函数。若段内有随机屏蔽、外部状态更新或其他副作用，直接再执行一次可能改变结果。需要保存随机状态或采用能够保持语义的机制，也要避免在重算时重复执行不应重复的外部副作用。

第 13 章 数据管线与异构系统：让计算单元持续拿到数据

一个 kernel 再快，如果下一批数据迟迟没有准备好，GPU 也只能等待。深度学习系统的执行路径从存储开始，经过读取、解码、变换、组批和复制，最后才到算子。本章先建立一条具体数据路径，再把它放入多进程和异构硬件环境中。

13.1 Dataset 与 DataLoader 分别负责什么

Dataset 描述如何取得一个样本，DataLoader 决定怎样抽样、组批和安排读取工作。把两者分开，便能在不改模型的情况下更换批大小、顺序或数据来源。

代码示例 六个本地样本，不依赖外部下载

```
import torch
from torch.utils.data import Dataset, DataLoader

class SmallDataset(Dataset):
    def __len__(self):
        return 6

    def __getitem__(self, index):
        # Dataset 返回一个样本；DataLoader 再把若干样本拼成批次
        features = torch.tensor([index, index*index], dtype=torch.float32)
        label = index % 2
        return features, label
```

调用 `SmallDataset()[2]` 得到特征 `[2, 4]` 和标签 `0`。DataLoader 把相邻两个样本组为一个批次：

```
loader = DataLoader(SmallDataset(), batch_size=2,
                    shuffle=False, num_workers=0)
features, labels = next(iter(loader)) # 取得第一批，而不是整个数据集
```

预期特征形状为 $(2, 2)$ ，内容为 $[[0, 0], [1, 1]]$ ，标签为 $[0, 1]$ 。如果单个样本本身是 (C, H, W) 图像，普通组批通常会在最前面增加批维。长度不一致的序列则可能需要自定义 collate 逻辑，用填充、分桶或其他表示组织。

按索引读取与流式读取

map-style 数据集提供长度和索引访问，适合由采样器决定下一批索引。iterable-style 数据集逐个产生样本，适合文件流、在线数据或无法便捷随机访问的来源。流式接口不自动保证多个工作进程各读不同的数据，分片责任必须落实。

13.2 多工作进程：提速之前先避免重复读取

设有 R 个训练进程，每个进程有 W 个读取工作者。可以给总计 RW 个工作者分配不同的全局分片编号，再从同一个样本顺序中各取一部分：

代码示例 训练进程与读取工作者共同决定分片

```
def worker_indices(size, world_size, rank, workers, worker_id, seed, epoch):
    order = np.random.default_rng(np.random.SeedSequence([seed, epoch])).
    permutation(size)
    shards = world_size * workers
    shard_id = rank * workers + worker_id
    return order[shard_id::shards] # 每个样本只属于一个进程、一个工作线程
```

例如 17 个样本、2 个训练进程、每进程 2 个工作者，就形成 4 个分片。把各分片合在一起应恰好覆盖原样本，不重复也不遗漏。这是读取层面的性质；同步训练还需要让各进程执行兼容的步数和集合通信顺序。

若训练需要等长批次，可以选择丢弃尾部、不足时补样本，或采用支持不均匀输入的策略。补齐会引入重复样本，丢弃会减少当轮覆盖，应按训练约定明确处理。不能一边假设样本恰好只出现一次，一边又无条件补齐每个分片。

随机状态也需要分工

如果所有工作者使用完全相同的随机流，数据增强可能出现不希望的重复；如果种子随意变化，又难以恢复执行。可从基准种子、epoch、进程和工作者标识派生相应随机状态，同时保持抽样和增强的职责清楚。

同一个种子不等于所有并行执行都逐位可重复。调度、算子算法、数据顺序和数值归约也可能影响结果。首先应明确希望恢复的是样本顺序、统计行为，还是严格的完整计算轨迹。

13.3 预取与流水线：把等待搬到前面

不做重叠时，一批数据的时间可能是

$$t_{\text{batch}} = t_{\text{read}} + t_{\text{copy}} + t_{\text{compute}}.$$

若读取、复制与计算可以在独立资源上形成稳定流水线，充分预热后的吞吐可接近由最慢阶段决定的水平：

$$t_{\text{steady}} \approx \max(t_{\text{read}}, t_{\text{copy}}, t_{\text{compute}}).$$

这个近似需要资源和依赖条件满足，不能仅靠增加工作者数量保证。

运行追踪 把启动成本也算进去

假设读、复制、计算分别耗时 2、1、5 个单位。10 批完全串行需要 80 个单位；理想三阶段流水线需要首批的 8 个单位，再加 9 次最慢阶段的 5 个单位，共 53。这里假设各阶段耗时恒定、资源独立，并能充分重叠。

预取队列需要容量上限，否则生产者过快可能耗尽内存。消费者提前停止或生产者失败时，也应有清楚的结束与错误传播机制。增加并行度的同时，要保持系统可以退出、可以定位错误。

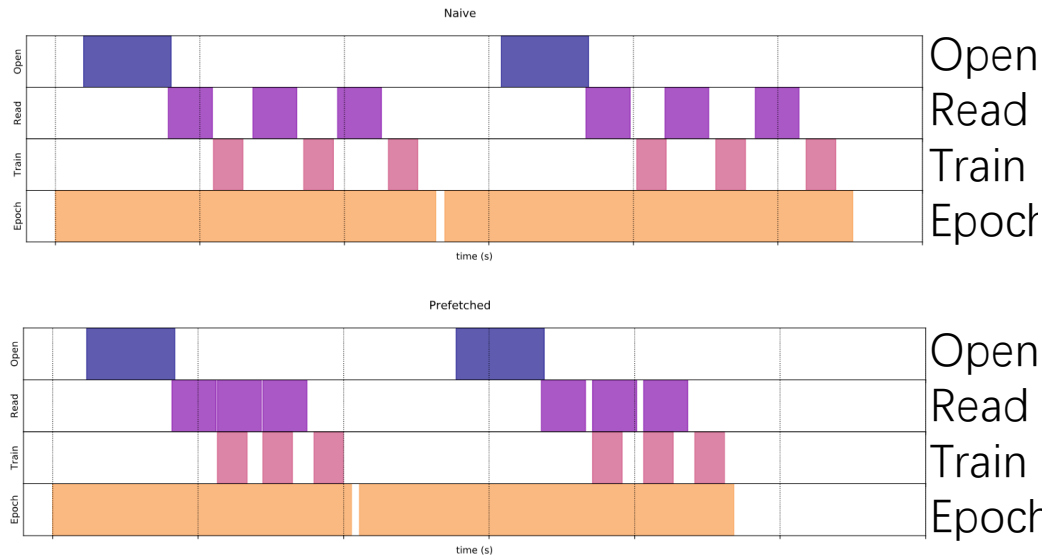


图 13.1 上方各阶段依次等待；下方预取提前准备后续批次，使数据处理与模型计算有重叠机会。（课程第 14 讲，第 35 页）

锁页内存与异步复制的边界

锁页主机内存能满足许多异步传输与重叠路径的要求，但它不是任意分配越多越好。还要确保复制使用的缓冲区在传输期间不被修改或释放。框架提供 pinned memory、非阻塞复制等入口时，也仍需配合正确的流和寿命管理。

数据变换未必全部适合搬到 GPU。小而复杂的解码、GPU 上重复的格式转换、CPU 与 GPU 之间反复往返，都可能抵消加速收益。应该测量完整批次路径，而不是只比较其中一个孤立函数。

13.4 存储格式与检查点

很多小文件会增加元数据与随机访问成本；单个巨大顺序压缩流又可能不方便并行随机读取。分片大小、压缩方式、缓存和索引，应结合真实访问模式选择。

模型检查点与激活重算中的 checkpoint 不是同一个用途。前者用于恢复训练或保存模型，后者用于在单次计算中权衡激活内存和重算。恢复训练通常不仅需要模型参数，还需要优化器状态、学习率调度、更新步数，以及影响后续数据和随机操作的状态。

代码示例 保存随机生成器状态，恢复后续抽样顺序

```
rng = np.random.default_rng(13)
_ = rng.permutation(10)                # 已经完成的那一轮
saved_state = copy.deepcopy(rng.bit_generator.state)
next_order = rng.permutation(10)       # 未中断时，下一轮应看到的顺序
restored = np.random.default_rng()
restored.bit_generator.state = saved_state
assert np.array_equal(restored.permutation(10), next_order)
```

只保存最初的种子并重新开始，会重放已经执行过的随机序列；保存中途状态才能从相应位置接续。实际检查点还应保证各组件来自同一个一致的训练时刻，避免模型参数属于第 100 步，而优化器状态属于第 99 步。

写出大型检查点时，可以先写临时文件、校验完整性，再以合适的原子替换方式发布。分布式检查点还要组织各分片与元数据，明确恢复时的设备数量、分片布局 and 版本兼容关系。

13.5 异构硬件：专用单元需要相应的数据组织

CPU、GPU 与专用加速器的优势来自不同的控制、并行与存储组织。CPU 擅长复杂控制与低延迟任务，GPU 以大量并行线程处理吞吐型工作，专用矩阵阵列则围绕特定数据流复用操作数。

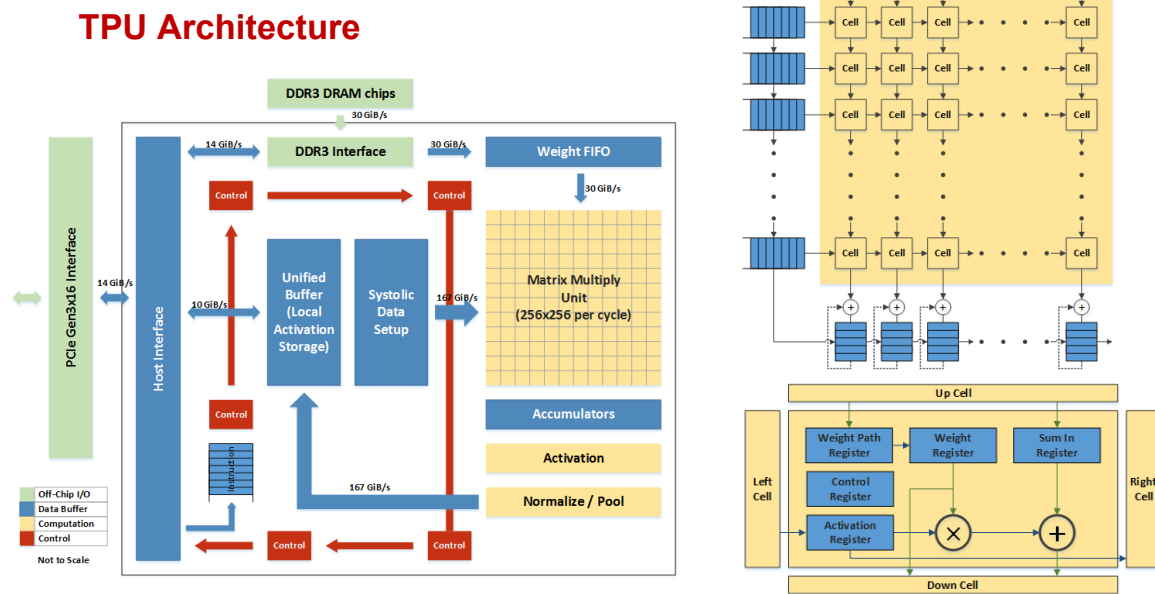


图 13.2 专用深度学习处理器的一种组织示意：矩阵单元、缓冲区和数据传输共同决定执行效率。（课程第 15 讲，第 13 页）

脉动阵列可以让矩阵元素沿相邻单元逐步传递，在移动过程中持续完成乘加。其关键是数据在计算单元之间被复用，而不是每次乘法都从远处重新读取所有输入。GPU 的 Tensor Core 也需要由周围的线程、寄存器与内存系统持续供给数据。

从 CUDA 线程到矩阵指令的合作

普通逐元素 kernel 常让一条线程负责一个输出；warp 级矩阵接口则让一组线程合作完成一个小矩阵块。概念步骤是：建立输入和累加 fragment，按规定布局加载，执行矩阵乘加，再按规定布局写出。调用中的模板参数描述矩阵角色、块尺寸与数据类型，不是普通运行时矩阵维度参数。

代码示例 一个 warp 合作计算十六阶矩阵块

```
#include <cuda_fp16.h>
#include <mma.h>
using namespace nvccuda; // WMMA 接口位于这个命名空间下
__global__ void small_tensor_product(const half* a, const half* b, float* c) {
    // 一整个 warp 合作持有 fragment，不是一条线程拥有完整矩阵
```

```

wmma::fragment<wmma::matrix_a,16,16,16,half,wmma::row_major> af;
wmma::fragment<wmma::matrix_b,16,16,16,half,wmma::row_major> bf;
wmma::fragment<wmma::accumulator,16,16,16,float> cf;
wmma::fill_fragment(cf,0.0f);      // 先把累加结果初始化为0
wmma::load_matrix_sync(af,a,16);   // 最后一个参数是存储行步长
wmma::load_matrix_sync(bf,b,16);
wmma::mma_sync(cf,af,bf,cf);       // C <- A*B + C
wmma::store_matrix_sync(c,cf,16,wmma::mem_row_major);
}

```

这个例子的完整程序是 15_wmma.cu，需要支持相应 WMMA 操作的架构与编译目标。程序使用 `<<<1,32>>>` 启动一个 warp。模板参数中的 `matrix_a`、`matrix_b`、`accumulator` 指定片段角色，三个 16 指定这次块运算的 M, N, K ，`half` 与 `float` 分别指定输入和累加类型。所有线程按同一流程参与加载、乘加与写出。

`load_matrix_sync` 的步长描述真实存储，并要求满足相应对齐与尺寸条件。示例使用独立设备分配与紧凑十六阶矩阵；推广到任意大矩阵时，还需要外层分块、边界与相应布局处理。只有满足支持的架构、块形状、对齐和参与线程条件，才能使用对应硬件路径。高层框架选择低精度，也仍需要存在真正适配形状与布局的底层实现，才可能获得预期收益。

13.6 节点互联：设备位置也会改变成本

单个服务器里可能有多个 CPU、GPU、网卡和存储设备。GPU 间的通信路径可能经过不同互联或 CPU 根节点；跨服务器又会增加网络协议、交换机和拓扑约束。

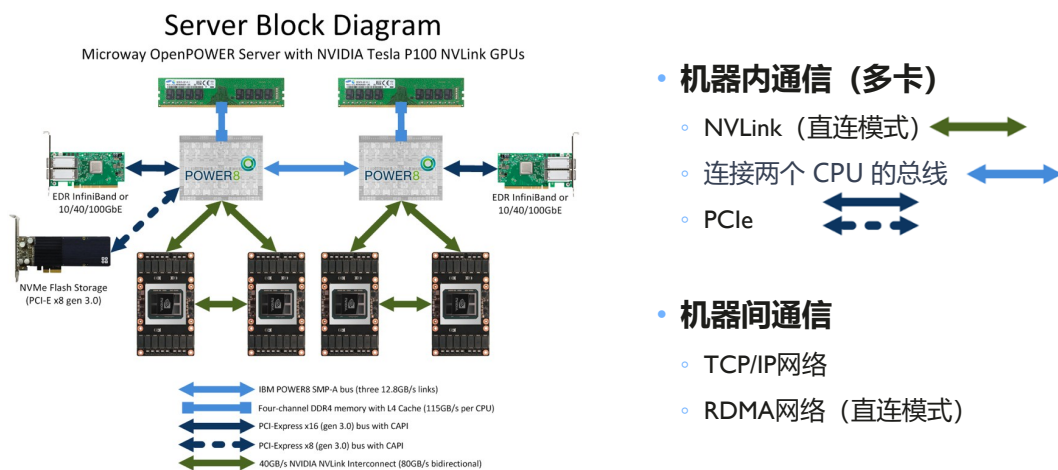


图 13.3 一个计算节点内，设备与互联构成不同的数据路径。应根据实际拓扑安排进程、设备和通信。（课程第 15 讲，第 46 页）

PCIe 是常见设备连接方式，NVLink 为支持它的设备提供相应互联。跨节点的 RDMA 则让网卡按协议访问已注册的远端内存区域，减少部分 CPU 参与和数据复制；它仍需要明确的缓冲区、完成通知与通信协议。

NUMA 位置、网卡亲和性、共享链路和并发流量，都可能让相同字节数具有不同时间成本。之后讨论分布式训练时，我们会用“延迟 + 数据量/带宽”的模型起步，再结合这些实际路径理解瓶颈。

第 14 章 分布式训练：把局部梯度合成同一次更新

当一个训练步骤分散到多张 GPU，首先要回答的不是能否同时启动线程，而是各张卡上的计算合起来是否仍代表原来的目标。本章从数据并行出发：每个工作者保存相同参数、处理不同样本，再把局部梯度组合起来。

14.1 进程、设备与 rank

分布式通信组中的每个参与进程有一个 rank。rank 是通信身份，不等同于某台机器上的 GPU 编号；多机环境下还需要本地 rank 来决定当前进程使用哪张设备。

一个常见配置是每张 GPU 对应一个训练进程。进程间通过通信库交换张量，进程内部则按普通 CUDA 依赖执行。数据进程如何分片、各 rank 进入了哪些集合通信、参数是否同一版本，都是正确性的一部分。

14.2 数据并行究竟平均什么

设第 r 个工作者处理 n_r 个样本，本地平均损失的梯度为

$$g_r = \frac{1}{n_r} \sum_{i \in \mathcal{B}_r} \nabla_{\theta} \ell_i(\theta).$$

如果目标是所有样本组成的全局平均损失，其梯度应为

$$g = \frac{\sum_r n_r g_r}{\sum_r n_r}.$$

仅当本地样本数相同，才可直接对本地平均梯度等权平均。

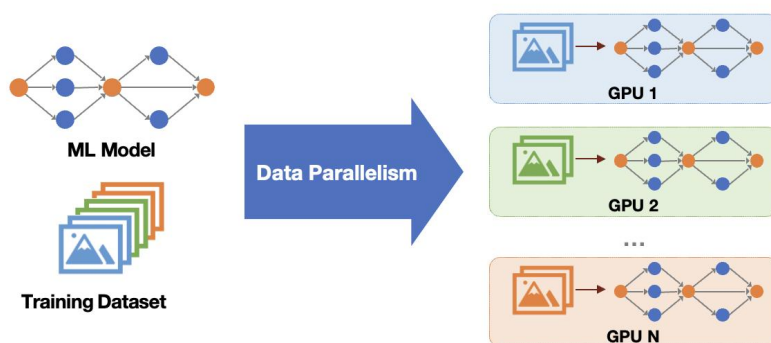


图 14.1 参数副本相同，样本分给不同工作者；梯度汇总后，各副本执行一致的参数更新。（课程第 16 讲，第 52 页）

运行追踪 两个工作者的简单算术

工作者 0 只有 1 个样本，平均梯度为 2；工作者 1 有 3 个样本，平均梯度为 6。全局平均应为 $(1 \times 2 + 3 \times 6) / 4 = 5$ ，不是 $(2 + 6) / 2 = 4$ 。平均的是样本贡献，而不是不加分析地平均任意两个已经归一化的数。

代码示例 把本地平均还原成相应样本贡献

```
def local_mean_gradient(x, y, weight):
    return x.T @ (x @ weight - y) / len(x)

def global_gradient(parts, weight):
    sizes = np.array([len(x) for x, _ in parts])
    local = [local_mean_gradient(x, y, weight) for x, y in parts]
    # 每份梯度已经是本地平均，应按本地样本数加权
    return sum(n*g for n, g in zip(sizes, local)) / sizes.sum()
```

还要明确损失是 sum 还是 mean，是否做了梯度累积，以及通信得到 sum 还是 average。多除一次批大小或世界大小，会直接改变有效学习率。

14.3 集合通信：按语义选择操作

操作	输入如何组织	每个参与者最终得到什么
Broadcast	一个根拥有完整输入	根输入的一份副本
Reduce	各 rank 有同形状贡献	指定根得到归约结果
AllReduce	各 rank 有同形状贡献	每个 rank 得到相同归约结果
AllGather	各 rank 拥有结果的一块	每个 rank 得到按 rank 组织的完整结果
ReduceScatter	各 rank 有完整贡献	各自得到归约结果的一块
AlltoAll	每个 rank 为不同目标准备分块	各自收到来自不同来源的相应分块

表 14.1 相似的数据传输可以服务不同语义，不能只按名字中的 Gather 或 Reduce 猜结果。

这些操作需要参与者对通信组、调用顺序、元素数量与 dtype 等要求达成一致。若某个 rank 因为条件分支跳过了一次必要的 collective，其他 rank 可能一直等待。[9]

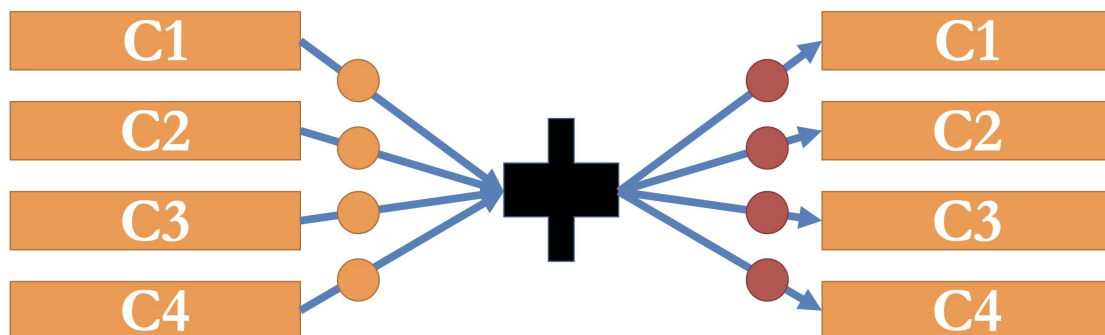
AllReduce 可以分成两个阶段

ReduceScatter 先让每个工作者取得全局归约结果中的一个分块，AllGather 再把这些完成的分块传播给所有工作者。两步组合便可实现 AllReduce。[9]

这项分解也解释了状态分片训练为何不一定要立刻 AllGather 梯度：如果某个工作者只负责更新自己拥有的一份参数状态，那么得到相应的归约梯度分块就已经足够。

All-Reduce

- Compute some reduction (usually a sum) of data on multiple machines and materialize the result on all those machines.

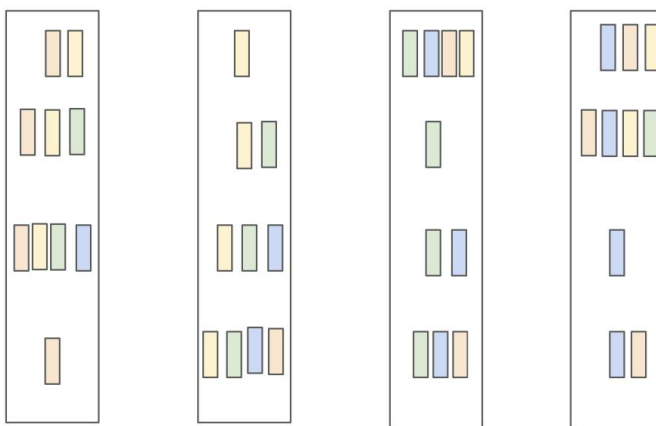


- All-Reduce: perform *Reduce* on all workers

图 14.2 AllReduce 先组合对应位置的贡献，再让所有 rank 持有组合后的结果。SUM 本身不会自动除以工作者数量。（课程第 16 讲，第 35 页）

14.4 环形归约：每一步只与邻居交换一块

设有 p 个 rank，把每个输入划为 p 块。环形 reduce-scatter 经过 $p - 1$ 步，每步向下一个 rank 发送一块，同时接收前一个 rank 的一块并相加。结束后，不同 rank 各持有一个已经合并所有贡献的分块。



Each node have correctly reduced result of one segment!
This is called *reduce scatter*

图 14.3 第一阶段结束时，完整归约结果分散在各参与者上。（课程第 16 讲，第 85 页）

接着执行 $p - 1$ 步 all-gather。这一阶段传递的是已经完成的分块，接收时复制而不再次求和。

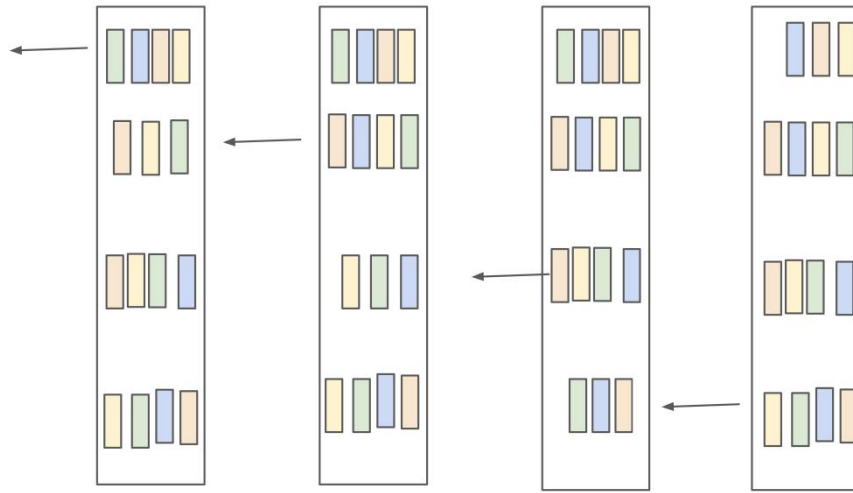


图 14.4 第二阶段逐步传播完成的分块，使各 rank 获得完整结果。（课程第 16 讲，第 88 页）

代码示例 用本地数组模拟每一轮的通信与更新

```
def ring_allreduce(values):
    ranks, length = values.shape
    chunk = (length+ranks-1)//ranks
    padded = np.pad(values, ((0,0),(0,chunk*ranks-length)))
    state = padded.reshape(ranks, ranks, chunk).copy()
    # 每一轮先取得快照，表示所有发送读的是该轮开始时的数据
    for step in range(ranks-1):          # reduce-scatter: 接收后相加
        previous = state.copy()
        for rank in range(ranks):
            source = (rank-1) % ranks
            part = (rank-1-step) % ranks
            state[rank, part] += previous[source, part]
    for step in range(ranks-1):          # all-gather: 传播已完成的分块，不再相加
        previous = state.copy()
        for rank in range(ranks):
            source = (rank-1) % ranks
            part = (rank-step) % ranks
            state[rank, part] = previous[source, part]
    return state.reshape(ranks, -1)[: , :length]
```

快照 `previous` 表示所有发送都读本轮开始时的状态，避免 Python 顺序执行循环时让后面的 rank 提前看到本轮新结果。实际通信实现通过缓冲区与异步请求管理同样的读写依赖，而不是完整复制所有 rank 的状态。

通信成本为什么包含延迟项

若每个 rank 的完整梯度大小为 S 字节，每阶段发送 $p-1$ 块，每块约 S/p 字节，那么每个 rank 的总发送量为

$$2 \frac{p-1}{p} S.$$

在简化的全双工环模型里，单步启动延迟为 α 、有效链路带宽为 β ，时间近似为

$$T_{\text{ring}} \approx 2(p-1)\alpha + 2\frac{p-1}{p}\frac{S}{\beta}.$$

这里接收可以与发送重叠，因此没有再把同样的接收字节机械加一遍。小消息时，很多轮的延迟可能主导；大消息时，分块流水和带宽利用更重要。

树形方案可以减少某些通信的依赖轮数，分层方案可以分别利用节点内与节点间互联。哪一种更快取决于消息大小、拓扑、并发流量与具体算法，不能脱离系统条件统一排名。

14.5 把同步 SGD 写成一个可运行步骤

下例使用 PyTorch 的 DDP 包装模型，假设每个 rank 有相同批大小。DDP 在反向阶段完成相应梯度同步，使参数副本使用一致的更新信号。[10]

代码示例 各进程处理自己的数据，再同步更新

```
"""Run with torchrun on two or more GPUs; tiny local data, no downloads."""
import os
import torch
import torch.distributed as dist
from torch.nn.parallel import DistributedDataParallel as DDP

def main():
    local_rank = int(os.environ["LOCAL_RANK"])
    torch.cuda.set_device(local_rank)          # 一个进程绑定一张本地 GPU
    dist.init_process_group(backend="nccl")
    rank, world = dist.get_rank(), dist.get_world_size()
    torch.manual_seed(14)
    model = torch.nn.Linear(3, 1, bias=False).cuda()
    wrapped = DDP(model, device_ids=[local_rank])
    torch.manual_seed(100+rank)
    x, y = torch.randn(4, 3, device="cuda"), torch.randn(4, 1, device="cuda")
    optimizer = torch.optim.SGD(wrapped.parameters(), lr=0.1)
    optimizer.zero_grad()
    loss = ((wrapped(x)-y)**2).mean()          # 本例各 rank 批大小相同
    loss.backward()                            # DDP 在反向中归约参数梯度
    optimizer.step()                          # 各 rank 用相同梯度更新相同参数
    weights = model.weight.detach()
    copies = [torch.empty_like(weights) for _ in range(world)]
    dist.all_gather(copies, weights)
    for other in copies:
        torch.testing.assert_close(weights, other)
    if rank == 0:
        print("DDP: parameters agree after one synchronized step")
    dist.destroy_process_group()

if __name__ == "__main__": main()
```

在拥有至少两张合适 GPU 的 Linux 环境中，可用：

```
torchrun --standalone --nnodes=1 --nproc-per-node=2 examples/torch/14_ddp.py
```

`torchrun` 建立两个进程并提供相应环境变量。每个进程通过 `LOCAL_RANK` 选择本地设备，通过通信组获得全局 `rank`。预期结果是一次更新后各参数副本一致。

实际训练还需要按 `rank` 分配样本，不能让所有 `rank` 无意读取完全相同的数据。使用分布式采样器时，应根据接口约定更新 `epoch` 状态；批次长度和最后一个批次的处理也要与损失缩放保持一致。

14.6 通信与反向重叠

反向通常从后面的层向前计算梯度。某一部分参数梯度已经就绪时，可以先启动它的归约，同时继续计算尚未得到的梯度。这比等整个反向完成后才发送全部数据更有机会隐藏通信。

通常把梯度组织成 `bucket`。`bucket` 太小，通信启动开销增多；太大，又要等待更多梯度就绪，重叠机会减少。真正需要优化的是关键路径上仍未被覆盖的通信尾部，而不是让时间线上出现任意重叠就算成功。

梯度累积改变更新频率

若多个微批次共同组成一次有效批次，可以先累计梯度，最后再同步与更新。每个微批次贡献应按有效批次的样本权重缩放。等大小的 k 个微批次可各除以 k ；不等大小时则不能只按微批次次数平均。

使用框架提供的不同步上下文时，还需遵守其对前向和反向范围的要求。所有 `rank` 最终必须以兼容的顺序执行相应同步，避免只有部分进程进入通信。

14.7 参数服务器、异步更新与扩展效率

参数服务器把参数或更新状态集中到一个或多个服务端，工作者拉取参数、计算梯度并提交。可以按参数分片，也可以采用同步或异步更新。它改变了通信与状态管理的位置，不必然比 `AllReduce` 更快或更慢。

异步工作者可能基于旧参数计算梯度，再把结果应用到较新版本。这样减少某些等待，却改变了更新的时间语义，带来梯度陈旧性等问题。不能直接把它当作完全等价的同步大批次 SGD。

增加工作者还会受到慢进程、数据管线、网络和优化过程的共同限制。若固定本地批大小，增加 `rank` 会增大全局批次；若固定全局批大小，每个 `rank` 的工作又会减少，通信占比可能上升。评价扩展收益时，应同时说明计算预算与优化目标是否保持一致。

第 15 章 模型、张量与流水线并行：切开以后怎样接回来

数据并行让每张卡保存完整模型，不能单独解决模型状态放不进一张卡的问题。另一条路线是把模型、算子或状态拆开。关键不在于“平均切几份”，而在于拆分之后，每个中间张量代表完整值还是部分贡献，以及何时需要拼接或求和。

15.1 先区分三种拆分对象

按层拆分，把不同网络阶段放到不同设备，是流水线并行的基础。把一个矩阵运算的维度拆给多张卡，是张量并行。把本来在数据并行副本中重复保存的参数、梯度或优化器状态分片，则是状态分片。

这几种方法可以组合，但它们消除的是不同的重复、引入的是不同的通信。先画出一个训练步骤中的数据和状态，再决定拆哪一部分。

15.2 列并行：拆输出特征，结果需要拼接

设 $Y = XW$ ，其中 $X \in \mathbb{R}^{B \times D}$ 、 $W \in \mathbb{R}^{D \times H}$ 。沿 W 的列拆分：

$$W = [W_0 \ W_1], \quad Y = [XW_0 \ XW_1].$$

每个 rank 需要完整输入 X ，但只保存一部分权重并计算一部分输出特征。两份结果是不同列，应当拼接，不应相加。

如果下一个操作也能在这些输出列上独立完成，比如逐元素 ReLU，就不必立刻把输出 All-Gather 成完整张量。延迟不必要的聚集，可以减少通信。

反向时，给定各输出分片的梯度 G_r ，

$$\bar{W}_r = X^T G_r, \quad \bar{X} = \sum_r G_r W_r^T.$$

权重梯度是局部的，输入梯度却是多条输出特征路径的贡献之和。因此输入反向通常需要对应的归约。

15.3 行并行：拆归约维，结果需要相加

沿输入特征维拆分：

$$X = [X_0 \ X_1], \quad W = \begin{bmatrix} W_0 \\ W_1 \end{bmatrix}, \quad Y = X_0 W_0 + X_1 W_1.$$

每张卡产生的是同一组输出位置的部分和，必须求和，而不是拼接。

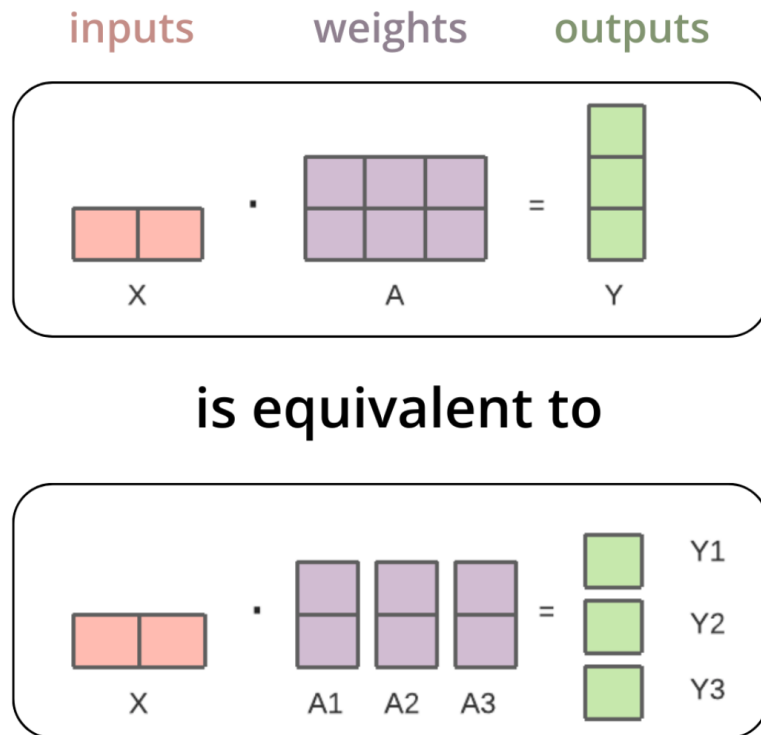


图 15.1 沿权重输出特征拆列，得到相应输出分片。每片都使用同一份输入。（课程第 17 讲，第 30 页）

Tensor Parallelism

Partition Attention Layers: Out Projection

- OutputProjection layer is splitted by **rows**.
- The computation can be done without synchronization.
- The outputs is then collected with **AllReduce**.

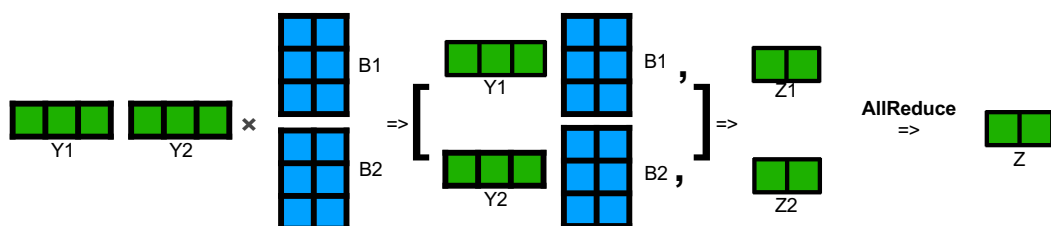
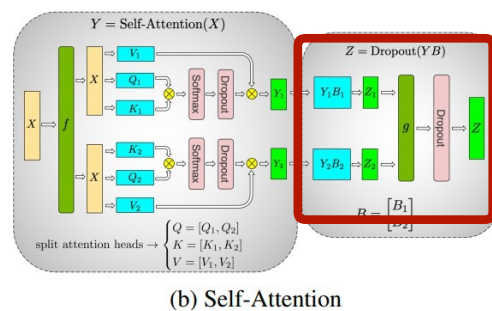


图 15.2 行分片得到相同输出位置的不同贡献，汇总阶段使用求和归约。（课程第 17 讲，第 37 页）

运行追踪 用最小内积区分拼接与求和

计算 $[1, 2, 3, 4] \cdot [5, 6, 7, 8]^T$ ，两份输入分别得到 $1 \times 5 + 2 \times 6 = 17$ 和 $3 \times 7 + 4 \times 8 = 53$ 。完整结果是 70，不是向量 $[17, 53]$ 。分片输出的形状相同，不代表它们已经各自算出了完整值。

给定共同的完整上游梯度 G ，各分片可计算 $\bar{X}_r = GW_r^T$ 与 $\bar{W}_r = X_r^T G$ 。通信放在哪里，应根据每一步实际持有的布局决定，而不能只按“这一层叫线性层”判断。

15.4 两层前馈网络怎样避免中间聚集

前馈块为

$$Y = \phi(XW_1 + b_1)W_2 + b_2.$$

第一层按输出列拆，得到隐藏特征分片；逐元素激活在本地完成；第二层按相应输入行拆，产生输出部分和；最后一次求和即可得到完整输出。

代码示例 一列拆分接一行拆分

```
def tensor_parallel_ffn(x, w1, b1, w2, b2, ranks=2):
    first_weights = np.split(w1, ranks, axis=1) # 沿第一层输出特征拆列
    first_biases = np.split(b1, ranks)
    second_weights = np.split(w2, ranks, axis=0) # 第二层沿对应输入特征拆行
    partial = []
    for a, bias, b in zip(first_weights, first_biases, second_weights):
        local_hidden = np.maximum(x @ a + bias, 0)
        partial.append(local_hidden @ b) # 每份只是最终输出的一部分贡献
    return sum(partial) + b2 # 输出偏置只能在求和后加一次
```

`np.split` 在这里模拟不同 rank 持有的数组，不会建立真实网络通信。关键是公式关系：中间隐藏维被一一对齐地拆开，最终输出相加。输出偏置 b_2 在相加后加一次；若各 rank 先加同一个偏置再 AllReduce，会把它错误地加上多次。

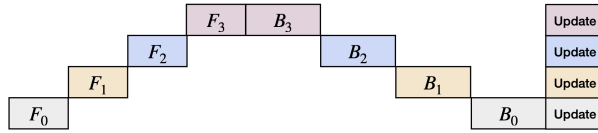
注意力常按完整头拆分

多头注意力的不同头可以分别计算相应 Q、K、V 和头内注意力，再通过输出投影组合。若切的是一个头内部的向量维，点积和归一化就可能需要额外跨卡协作，不能仍把各片当成完整独立的头。

序列维切分同样需要分析依赖。一个 query 通常要关注允许范围内许多 key；把序列简单分开后只做各片内部注意力，会改变模型。长上下文并行方法需要额外的数据交换或分布式归约规则。

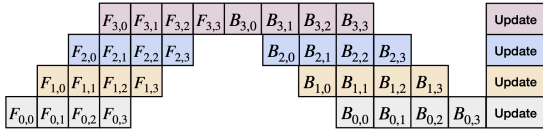
15.5 流水线：把不同微批次送进不同阶段

把网络分成 p 个阶段，一个完整批次直接逐层跨设备执行时，其他阶段可能大量等待。把批次拆成 m 个微批次后，第 0 阶段处理下一份输入时，第 1 阶段可以处理上一份，形成流水线。



(a). Naive model parallelism

- Split a single batch to micro batches
 - [16, 10, 512] ->
 - [4, 10, 512]
 - [4, 10, 512]
 - [4, 10, 512]
 - [4, 10, 512]



(b). Pipeline parallelism

- Motivation: model parameters are not changed during computation within a batch, thus we can pipeline computation and communication

图 15.3 微批次让不同阶段有机会同时工作，但开始、结束与依赖仍会形成空隙。（课程第 17 讲，第 22 页）

在各阶段前向和反向均耗时 t 、先完成全部前向再完成全部反向的理想化 GPipe 模型中，总时间约为

$$2(m + p - 1)t.$$

每个阶段做 $2m$ 次有效计算，因此利用率为

$$U = \frac{m}{m + p - 1}, \quad 1 - U = \frac{p - 1}{m + p - 1}.$$

$p = 4, m = 8$ 时，理想利用率为 $8/11$ 。增加微批次有助于减少相对气泡，但每份微批次过小也可能降低单个 kernel 的效率。

代码示例 把前向、反向放进显式时间槽

```
def gpipe_schedule(stages, microbatches):
    width = stages + microbatches - 1
    timeline = np.full((stages, 2*width), "", dtype=object)
    for stage in range(stages):
        for batch in range(microbatches):
            timeline[stage, batch+stage] = f"F{batch}"
            # 反向从最后阶段、最后微批次开始，再按依赖向前传递。
            backward_time = width+(microbatches-1-batch)+(stages-1-stage)
            timeline[stage, backward_time] = f"B{batch}"
    return timeline
```

F0 表示第 0 个微批次的前向，B0 表示它的反向。空字符串代表该阶段在相应理想时间槽中空闲。这个模型用于核对依赖与气泡，不包含真实通信时间和阶段不均衡。

调度还决定激活要保存多久

先做全部前向时，较早微批次的激活可能一直留到反向阶段。1F1B 等调度在预热后交替安排前向与反向，可以缩短部分激活的存活时间。它改变的是任务时间表，不等于自动改变了数学更新方式。

若希望保持同步批次语义，应在约定的全部微批次梯度完成以后统一更新参数。若各阶段在不同时间使用不同版本参数，则需要明确处理权重版本与梯度陈旧性，不能再直接当作相同的同步训练步骤。

15.6 状态分片：不让每个副本保存所有优化器状态

在数据并行中，参数更新相同，因此优化器状态常被重复保存。可以让不同 rank 分别负责一部分状态，再用通信获得相应计算需要的数据。

用一个明确账目说明：每参数 16 位权重 2 字节、16 位梯度 2 字节、32 位主权重 4 字节和两份 32 位 Adam 状态 8 字节，共 16 字节。对 p 个数据并行 rank，理想静态状态存储为：

阶段	分片对象	每 rank 的理想状态字节数
完整复制	无	$16P$
ZeRO-1	优化器及相应主参数状态	$4P + 12P/p$
ZeRO-2	再分片梯度	$2P + 14P/p$
ZeRO-3	再分片模型参数	$16P/p$

表 15.1 只统计所列模型状态；激活、通信缓冲区、工作空间和瞬时聚集未计入。

例如 $p = 8$ 时，账目分别为每参数 16、5.5、3.75、2 字节。它说明分片能显著降低重复状态，但不能直接用最后一个数字判断某个模型必然能放进显存。

参数分片不等于计算时永远只需一个小片

某层执行前可能需要 AllGather 它的参数，反向再通过 ReduceScatter 生成相应梯度分片。参数预取、层粒度和释放时机机会改变瞬时内存峰值。FSDP 等实现使用相关的分片思路，具体包装、通信和生命周期由框架设计决定。

参数服务器、ZeRO 分片与流水线并行不能混为同一种机制。可以用分片保存状态，也可以同时按层或按张量拆分计算，但每一维的通信组和数据含义都应单独说明。

15.7 组合并行：通信应发生在哪个组里

假设共有 8 张卡，采用数据并行 2、流水线并行 2、张量并行 2。可以用三元组 (d, s, t) 标识角色：

- 固定数据副本和阶段，改变 t ：同一个算子的张量并行组。
- 固定阶段和张量分片，改变 d ：同一份参数分片对应的数据并行组。
- 固定数据副本和张量角色，沿 s 传递：流水线前后阶段。

把某个 AllReduce 错放到所有 8 张卡组成的大组里，可能不仅浪费通信，还会把不应相加的张量混合。正确实现要从逻辑张量的分布方式出发，而不是只看参与进程数量。

15.8 选择划分时，把重新分布成本算进去

某种分片可能让当前算子很快，却迫使下一个算子重新 AllGather、AlltoAll 或重新切分。应把图中每个节点的计算成本与边上的数据重新分布成本一起考虑。

模型层数、隐藏维、序列长度、批大小、网络拓扑和单卡内存都会影响选择。没有一个只由“参数量大”决定的唯一并行方案。先在小规模上核对每份张量的数学意义，再分析每次通信的字节数与触发频率，才有可靠的扩展基础。

第 16 章 量化：用更少的位数表达计算

把模型从 32 位存储改成 8 位，表面上只是减小 dtype，实际上必须重新规定数值怎样映射、怎样相乘累加、怎样处理超出范围的值，以及误差会如何进入后续层。本章用可以手算的小例子建立这些关系。

16.1 整数并不知道自己代表多大的实数

仿射量化用整数 q 近似实数 x ：

$$q = \text{clip}(\text{round}(x/s) + z, q_{\min}, q_{\max}), \quad \hat{x} = s(q - z).$$

$s > 0$ 是 scale， z 是 zero-point。整数 z 对应实数 0，因此输入填充中的零也能按这套表示处理。 $q_{\min}, q_{\max}$ 由位数和有符号约定决定。

Symmetric INT8 Quantization

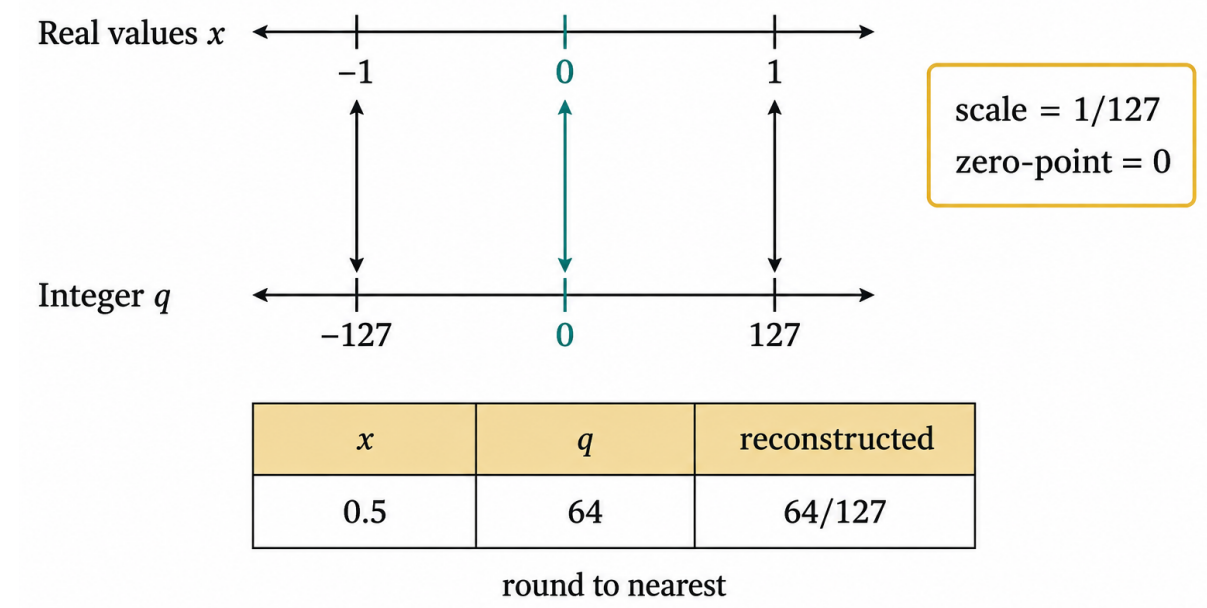


图 16.1 取 $s = 1/127, z = 0$ 建立对称格点。0.5 舍入到整数 64，再反量化得到 $64/127$ ，误差由格点间距决定。

例如用对称范围 $[-127, 127]$ 表示 $[-1, 1]$ ，可以取 $s = 1/127, z = 0$ 。输入 $[-1, 0, 0.5, 1]$ 量化后为 $[-127, 0, 64, 127]$ ，反量化的 0.5 约为 $64/127 \approx 0.50394$ 。这个误差来自格点间距，可以用所选尺度估计。

代码示例 舍入、裁剪与整数转换的顺序

```
def quantize_affine(x, scale, zero, qmin=-128, qmax=127):
    if scale <= 0 or not np.all(np.isfinite(x)):
        raise ValueError("positive scale and finite input are required")
    rounded = np rint(x / scale) + zero      # 本例采用最近偶数舍入
    return np.clip(rounded, qmin, qmax).astype(np.int32) # 先裁剪，后转整数

def dequantize(q, scale, zero):
    return scale * (q.astype(np.float64) - zero)
```

必须先裁剪再转成窄整数。直接把超出范围的值转成 int8，可能发生与预期饱和语义不同的结果。本例返回 int32 以便清楚展示后续计算；实际存储可以在范围检查后采用对应窄类型。

怎样选择 scale 与 zero-point

对一个需要覆盖的实数区间 $[r_{\min}, r_{\max}]$ ，常先让区间包含 0，再用

$$s = \frac{r_{\max} - r_{\min}}{q_{\max} - q_{\min}}, \quad z \approx q_{\min} - r_{\min}/s$$

并把 z 舍入、限制到整数范围。全零等退化区间要单独处理，不能让 scale 变成 0 后继续相除。

没有裁剪且采用最近格点舍入时，单个数的量化误差通常不超过 $s/2$ 。一旦输入越过覆盖区间，裁剪误差可能远大于这个界。因此 scale 的选择是在分辨率与可覆盖范围之间取舍。

16.2 矩阵乘法怎样转成整数累加

令

$$x_i \approx s_x(q_{x,i} - z_x), \quad w_i \approx s_w(q_{w,i} - z_w).$$

它们的内积近似为

$$\sum_i x_i w_i \approx s_x s_w \sum_i (q_{x,i} - z_x)(q_{w,i} - z_w).$$

整数部分需要比输入更宽的累加器。int8 输入不意味着内积也应在 8 位整数里累计。

代码示例 先在宽整数中处理零点和乘加

```
def quantized_matmul(x, w, sx, zx, sw, zw):
    qx = quantize_affine(x, sx, zx)
    qw = quantize_affine(w, sw, zw)
    # 参考实现用更宽整数，避免 int8 减法和乘加先发生溢出
    centered_x = qx.astype(np.int64) - zx
    centered_w = qw.astype(np.int64) - zw
    accumulator = centered_x @ centered_w
    return (sx*sw) * accumulator
```

若希望输出继续使用量化表示 (s_y, z_y) ，还要重新缩放：

$$q_y = \text{clip}\left(\text{round}\left[\frac{s_x s_w}{s_y} \text{acc}\right] + z_y\right).$$

实际整数 kernel 可以用相应定点乘数和移位近似这个比例；舍入、饱和和溢出范围仍要明确。偏置也要转换到匹配的累加尺度，不能把任意 float 偏置直接当作整数计数相加。

误差怎样进入内积

如果量化后的输入和权重分别为 $x + \delta x, w + \delta w$ ，则乘积变化包含

$$x \delta w + w \delta x + \delta x \delta w.$$

因此，相同绝对量化误差在不同层、不同通道或不同输入分布上，可能带来不同输出影响。只统计权重文件缩小多少，并不能说明任务效果一定保持。

16.3 量化粒度：一个 scale 还是很多个

per-tensor 让整个张量共用参数，形式简单，但某个极大通道可能迫使其他小范围通道也使用很粗的格点。per-channel 按通道分别设置尺度，per-group 则让若干元素共享一组参数。

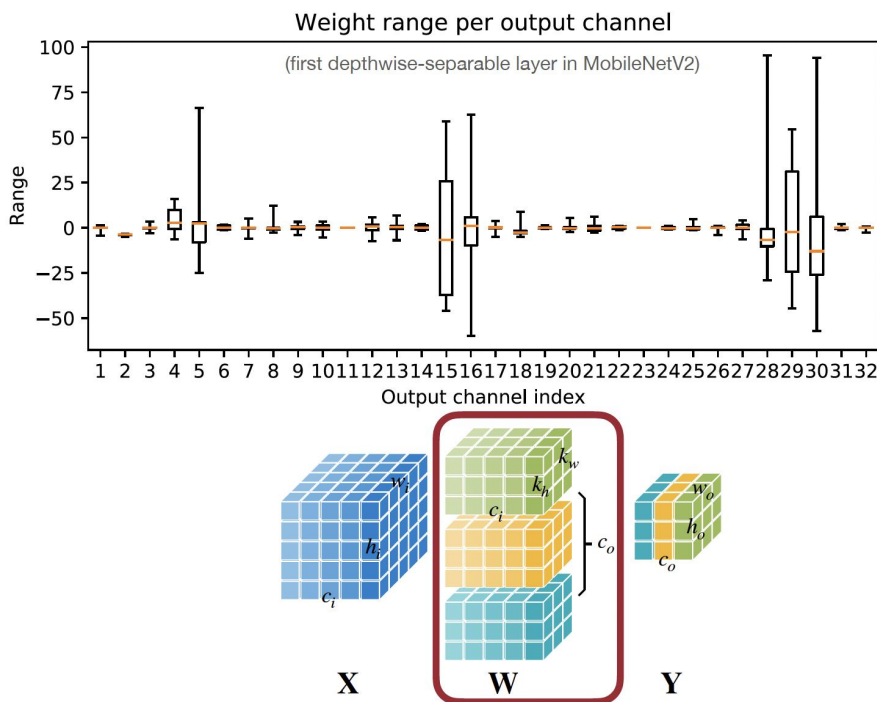


图 16.2 不同通道的范围可能相差很大，共用一个尺度会损失较小范围中的细节。（课程第 18 讲，第 45 页）

例如某列权重在 $[-0.02, 0.02]$ ，另一列接近 100。全张量 scale 若由 100 决定，小列可能几乎都量化为 0；按输出通道分别量化，就能保留小列的差异。代价是需要额外参数和相应 kernel 支持。

粒度还必须与算子布局对应。对形状为 $K \times N$ 的权重，若按输出特征分别量化，每个输出列有自己的 $s_{w,j}$ ，那么每列累加后的缩放也不同。不能在存储端按一种轴分组，执行端却按另一种轴解释。

16.4 校准与量化感知训练

训练后量化（PTQ）在已有模型上选择量化参数，通常需要代表性数据估计激活范围或优化误差指标。只取最小最大值容易受到离群点影响，也可以权衡裁剪与舍入误差。校准数据应代表预期使用分布，但不能拿最终测试集反复调到最好，再把它当作独立评价。

量化感知训练（QAT）在前向中模拟量化误差，让参数适应最终表示。由于 `round` 的真实导数几乎处处为 0，训练常使用人为定义的近似反向规则，例如直通估计器（STE）。

代码示例 前向模拟格点，反向采用明确的近似

```
def fake_quant(x, scale, qmin=-127, qmax=127):  
    rounded = np.clip(np rint(x.data / scale), qmin, qmax) * scale  
    inside = (x.data >= qmin*scale) & (x.data <= qmax*scale)  
    # STE 是人为选择的近似反向规则，不是 round 的真实导数  
    return Tensor(rounded, x.requires_grad, (x,), lambda g: (g*inside,))
```

本例在表示区间内把局部梯度视为 1，区间外视为 0。这是选择的优化近似，不是对 `round` 正确求导得出的结果，也不应拿普通有限差分强行验证其等于真实量化函数的导数。

训练中通常仍保留适合更新的参数表示，前向使用假量化结果。最终导出时，还需确认量化参数、算子组合、数据格式与目标后端支持一致。训练时模拟了低比特，不自动意味着部署时已经使用整数加速路径。

16.5 码本与剪枝：压缩表示不等于计算一定更快

另一种量化方式用少量中心值建立码本，每个权重保存一个码字索引。若多个权重共享一个可训练中心，中心梯度应累加所有使用位置的贡献。这与前面广播或重复索引的反向规律相同。

码本便于压缩，但不规则查表未必能直接用高效整数矩阵指令。线性量化的规则格点则通常更容易与乘加和缩放配合。应分别评估存储收益、计算路径和数据解码成本。

剪枝把一些权重或结构去掉。非结构化稀疏减少非零数目，却会引入索引与不规则访问；结构化剪枝直接减少通道、块或层，往往更容易映射到已有高效算子。把许多权重写成 0 后仍执行同一个稠密 GEMM，并不会自动减少对应乘加。

工程实践 评估最终执行路径

同时检查任务指标、模型存储、运行内存和端到端延迟。数据搬运、反量化、布局转换和小算子启动都可能占据显著成本。量化后的数学表示、导出格式与实际 kernel 必须配套，压缩收益才能进入真实系统。

第 17 章 大模型推理与服务：每生成一个 token，要做哪些工作

训练结束以后，系统目标从更新参数变成处理连续到来的请求。不同请求有不同提示词长度、输出长度与等待要求，而且生成过程需要保存历史状态。本章沿一次请求的路径，解释 KV cache、批处理、分页、FlashAttention 与推测解码分别改变了什么。

17.1 Prefill 与 Decode 的工作不同

自回归模型根据已有 token 预测下一个 token。Prefill 处理整个提示词，建立各层的相应状态，并产生预测第一个新 token 所需的 logits。随后 decode 每次处理新加入的 token，预测再下一个。

Prefill 的输入通常包含许多位置，可以使用较大的矩阵运算。Decode 中单个请求每步只增加一个位置，常更受权重读取、历史 KV 读取与小规模计算效率影响。具体瓶颈仍与批大小、上下文长度和实现有关，不能只用阶段名称断定。

先明确服务指标

首 token 延迟 (TTFT) 包括相应排队、提示词处理以及首次输出的等待。相邻输出 token 的间隔反映生成是否流畅。吞吐量则衡量单位时间处理的请求或 token 数，必须说明统计的是输入、输出还是两者合计。

优化大批次可能提高吞吐，却延长某个请求的等待；为了降低单请求延迟，也可能牺牲批内数据复用。评价服务系统需要把目标放在同一组指标上，而不是只比较一个孤立 kernel 的速度。

17.2 为什么缓存 K、V，而不必每次重算全部历史

对一个注意力头，

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V, \quad O = \text{softmax}\left(\frac{QK^T}{\sqrt{d_h}} + M\right)V.$$

因果掩码 M 阻止某个位置读取未来信息。因此，新 token 到来时，过去位置已经算出的相应表示不因未来 token 改变，过去的 K、V 可以继续使用。

第 t 个位置只需计算自己的 q_t, k_t, v_t ，将 k_t, v_t 追加到缓存，再用 q_t 与允许读取的全部历史 K 做匹配。过去的 Q 用来产生过去的输出，新的 query 不需要重用它们来计算当前注意力，所以通常缓存 K、V。

这个等价关系要求位置编码、掩码、模型状态与前缀保持一致。若改变了前缀内容或位置解释，就不能继续把原缓存当成相同计算的中间值。

缓存大小可以逐项数出来

设层数为 L ，批大小为 B ，缓存长度为 T ，KV 头数为 H_{kv} ，头维度为 d_h ，每元素 b 字节，则

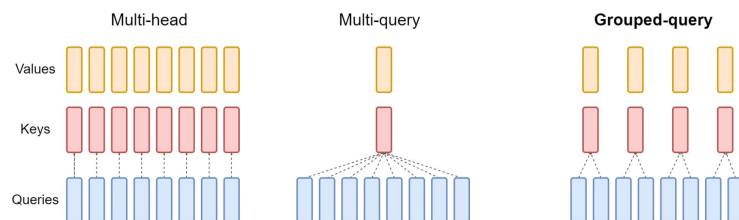
$$M_{KV} = 2LBTH_{kv}d_hb.$$

前面的 2 来自 K 与 V 两份数组，不是因为训练的前向和反向。对 $L = 32, B = 1, T = 4096, H_{kv} = 8, d_h = 128, b = 2$ ，结果为 512 MiB。它还没有包含模型权重、其他激活或分配器开销。

多查询注意力（MQA）让多个 query 头共享较少的 KV 头，分组查询注意力（GQA）则采用中间数量的 KV 组。这能减少相应缓存与读取，但属于模型结构选择，不是对任意已有模型无损删掉几份缓存。

Reduce the KV cache memory usage with MQA/GQA

- Reducing the KV cache size by reducing #kv-heads
 - Multi-head attention (MHA): N heads for query, N heads for key/value
 - Multi-query attention (MQA): N heads for query, 1 heads for key/value
 - Grouped-query attention (GQA): N heads for query, G heads for key/value (typically $G = N/8$)



Fast Transformer Decoding: One Write-Head is All You Need [Shazeer et al., 2019]
GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints [Ainslie et al., 2023]

图 17.1 query 头与 KV 头的对应关系决定缓存中需要保存多少独立的 K、V。（课程第 19 讲，第 27 页）

17.3 从固定批次到连续批处理

固定批次一起进入、一起保留到结束，较短请求完成后可能留下闲置位置。连续批处理在每次调度迭代重新组织活跃请求，完成的请求退出，新请求可以加入。

系统因此要维护每个请求的长度、缓存、完成状态和采样状态。批中的位置变化时，这些状态必须仍与对应请求匹配，不能因为把两个请求交换了行号就混用它们的 KV。

长 Prefill 会影响正在 Decode 的请求

新请求的提示词可能很长。如果一次完整 prefill 占据大量计算资源，原有请求的下一 token 就可能被延后。把 prefill 切成若干块，可以在调度中限制每次新增工作量，与 decode 更细地交错。

分块也会改变 kernel 形状、缓存使用和调度成本。需要根据延迟目标与资源预算选择块大小，不能只把任务切得越碎越好。

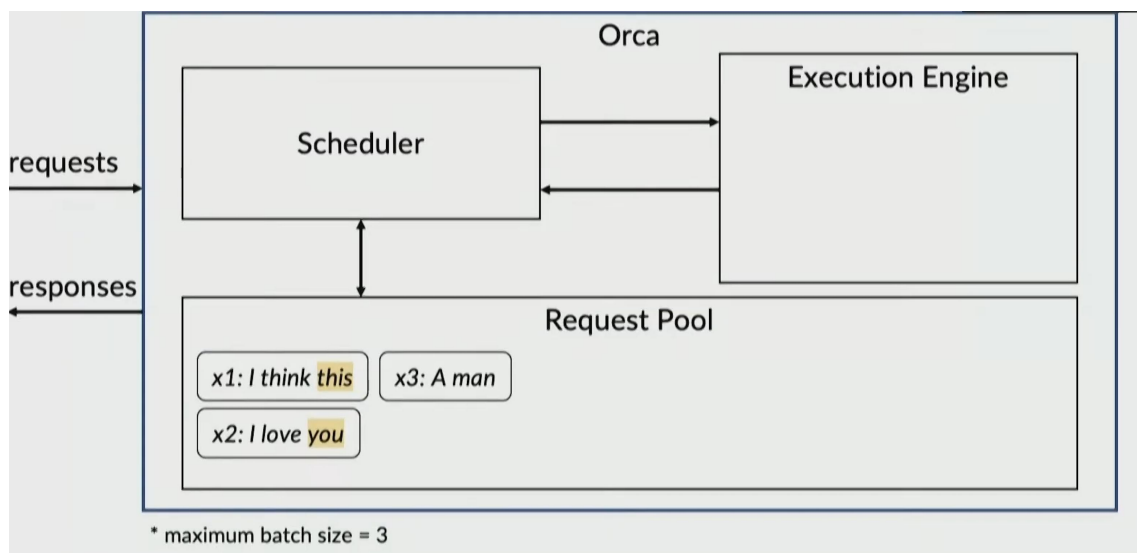


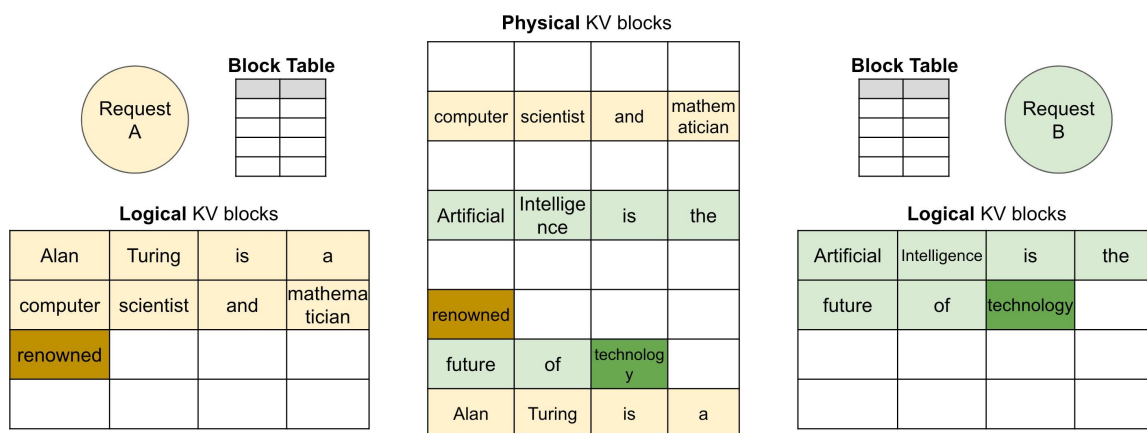
图 17.2 调度器维护请求池，执行引擎在迭代边界接收当前工作集合。（课程第 19 讲，第 51 页）

17.4 分页 KV cache：逻辑连续，物理可以分散

若每个请求都预留一块能够容纳最大生成长度的连续缓存，会浪费大量未使用空间。分页方法把缓存切成固定大小的块，用请求自己的块表把逻辑 token 位置映射到物理块。

PagedAttention

- The case for multiple requests



Efficient Memory Management for Large Language Model Serving with PagedAttention (Kwon et al., 2023)

图 17.3 不同请求用各自的逻辑块表访问物理 KV 块；可在相应条件下共享相同前缀的存储。（课程第 19 讲，第 81 页）

若块大小为 16，某请求的块表为 [7, 2, 9]，逻辑 token 20 位于逻辑块 1、块内偏移 4，对应物理位置 $2 \times 16 + 4 = 36$ 。kernel 必须按块表读取，而不能继续假定整条序列都在一段连续地址中。[12]

分页降低了要求连续大块存储所带来的浪费，但最后一块仍可能未用满，也有块表和管理成本。共享前缀时还需处理引用计数与写时复制：若多个请求共享一个未满足块，其中一个继续写入，不能破坏其他请求的前缀。

17.5 FlashAttention：不保存完整注意力矩阵

直接实现注意力可能物化 $T \times T$ 的分数和概率矩阵，再多次读写。FlashAttention 的关键是按块读取 Q、K、V，在片上或相应局部存储中组合计算，并在线归一化维护结果，减少高带宽存储器中的中间流量。[11]

它不是简单省略 Softmax，也不是默认把注意力近似成稀疏模式。对相应精确注意力计算，分块改变了数据移动与运算顺序；浮点舍入仍可能与另一种实现略有不同。

在线 Softmax 如何合并两批分数

对已经处理的一批分数，保存最大值 m 与归一化前的和

$$\ell = \sum_i e^{s_i - m}.$$

新批分数最大值为 m_b 。取 $m' = \max(m, m_b)$ 后，旧结果必须先改到新尺度：

$$\ell' = e^{m - m'} \ell + \sum_{j \in b} e^{s_j - m'}.$$

如果不乘旧结果前面的缩放项，就会把两种不同基准下的指数和直接相加。

注意力还需要加权值的分子

$$u = \sum_i e^{s_i - m} v_i.$$

它遵守同样的缩放规则：

$$u' = e^{m - m'} u + \sum_{j \in b} e^{s_j - m'} v_j, \quad O = u' / \ell'.$$

因此，处理完一个块后，不必保留该块的全部概率矩阵，只需更新 m, ℓ, u 。

代码示例 在线注意力的 NumPy 参考

```
def online_attention(q, k, v, query_tile=8, key_tile=7, offset=0):
    output = np.empty((len(q), v.shape[1]))
    for first in range(0, len(q), query_tile):
        queries = q[first:first+query_tile]
        maximum = np.full((len(queries), 1), -np.inf)
        denominator = np.zeros((len(queries), 1))
        numerator = np.zeros((len(queries), v.shape[1]))
        for begin in range(0, len(k), key_tile):
            keys, values = k[begin:begin+key_tile], v[begin:begin+key_tile]
            scores = queries @ keys.T / np.sqrt(q.shape[1])
            allowed = (begin+np.arange(len(keys))[None, :])
                        <= offset+first+np.arange(len(queries))[:, None])
            scores = np.where(allowed, scores, -np.inf)
            new_max = np.maximum(maximum, scores.max(axis=1, keepdims=True))
            rescale = np.exp(maximum-new_max) # 把旧累计量改到共同的新尺度
            probabilities = np.exp(scores-new_max)
            numerator = rescale*numerator + probabilities @ values
```

```

denominator = rescale*denominator + probabilities.sum(axis=1, keepdims=
True)
maximum = new_max
output[first:first+len(queries)] = numerator / denominator
return output

```

`offset` 指明当前 query 的全局位置，确保增量解码时掩码仍按真实序列位置计算。本例假设至少存在一个合法历史 key，且输入有限。最后不足整块的部分由切片自然处理。

与稠密因果注意力相比，配套程序的在线结果应在浮点误差范围内一致；把每个 query 分别与对应历史前缀计算，也应得到相同结果。GPU 实现还要把这个数学合并规则与具体线程、共享内存和流水线结合起来。

17.6 推测解码：先提出候选，再由目标模型核验

一个较小的草稿模型可以快速提出多个候选 token，较大的目标模型再对候选前缀进行核验。若较多候选被接受，便有机会用较少的目标模型顺序调用生成多个 token。

Accelerating memory-bounded generation

- From generator to verifier

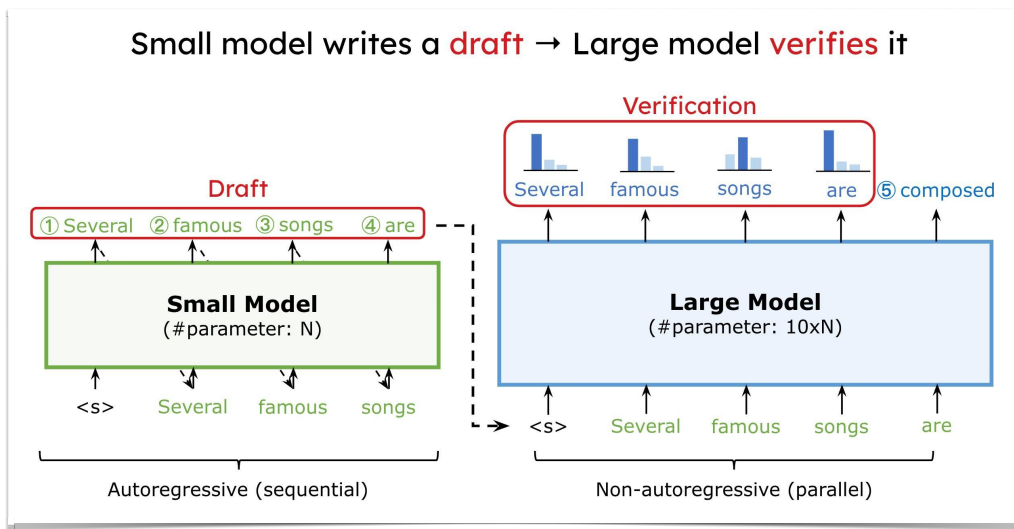


Image credit: vLLM First SF Meetup Slides

图 17.4 草稿负责提出候选，目标模型负责核验与修正。收益取决于接受率、草稿成本和批量核验效率。（课程第 19 讲，第 97 页）

对于随机采样，不能只接受两个模型最喜欢的相同 token，否则会改变目标分布。设目标分布为 p 、草稿分布为 q ，候选 $x \sim q$ 可按

$$a(x) = \min(1, p(x)/q(x))$$

接受；拒绝时，从归一化的剩余分布 $\max(p - q, 0)$ 采样。[13]

为什么这能保留目标分布？候选被接受的概率质量为 $q(x)a(x) = \min(p(x), q(x))$ ；拒绝后补回的质量正好是 $p(x) - \min(p(x), q(x))$ 。两部分相加为 $p(x)$ 。

运行追踪 两个 token 的分布

设 $p = [0.6, 0.4]$ 、 $q = [0.8, 0.2]$ 。第一种候选的接受率为 0.75，第二种为 1，接受质量为 $[0.6, 0.2]$ 。剩余拒绝质量 0.2 全部补到第二种 token，最终仍是 $[0.6, 0.4]$ 。

代码示例 检查接受与修正后的概率质量

```
def speculative_marginal(target, draft):
    accepted_mass = np.minimum(target, draft) # q(x)*min(1,p(x)/q(x))
    # 先统计未被接受的总质量，再用目标分布的剩余质量补回。
    rejection_probability = 1-accepted_mass.sum()
    residual = np.maximum(target-draft, 0)
    if rejection_probability > 1e-14:
        corrected = accepted_mass + rejection_probability*residual/residual.sum()
    else:
        corrected = accepted_mass
    return corrected
```

多 token 情形还要按已经接受的前缀逐步使用相应条件分布，拒绝后丢弃不再适用的后续候选，并正确处理 KV 状态。贪心解码采用不同的核验目标，应与保持随机采样分布的算法区分。

17.7 把服务优化放回完整请求路径

KV 缓存避免重算历史，分页改善缓存管理，连续批处理改善设备利用，FlashAttention 减少注意力中间流量，推测解码减少某些顺序核验成本。它们可以组合，却不是相互替代的同一个技巧。

最后仍要检查排队、调度、内存容量、数据移动、采样和输出路径。能容纳更多请求，不必然意味着每个请求都更快；吞吐提高，也不自动意味着尾延迟满足目标。只有把预期工作负载与服务指标说清楚，优化结果才有明确含义。

第 18 章 联邦学习：数据留在各处以后怎样共同训练

前面的数据中心训练通常能够统一安排设备和数据。联邦学习则面对另一类条件：数据分散在不同参与方，未必能集中，参与者的计算能力、在线时间和数据分布也可能不同。共同训练因此不仅是把同一个 SGD 程序多启动几份。

18.1 系统差异与数据差异同时存在

跨设备场景可能有大量间歇参与的终端；跨机构场景通常有较少但相对稳定的参与方。两者都需要考虑数据边界，但通信频率、失败处理和协调方式可能很不一样。

“非独立同分布”常简称 non-IID。这里最重要的直觉是：某个参与者的数据并不一定是全体数据的均匀缩影。例如不同参与者主要拥有不同类别或不同采集条件，局部最优方向就可能相互偏离。即使所有设备速度一样，这种统计差异仍然存在。

18.2 FedAvg：多走几步，再交换结果

设参与者 k 有 n_k 个样本，本地平均损失为 $F_k(w)$ 。一个常见全局目标是

$$F(w) = \sum_{k=1}^K \frac{n_k}{\sum_j n_j} F_k(w).$$

FedAvg 的一轮先广播公共参数 w_t ，选定客户在各自数据上做若干本地更新，再把结果交回服务器加权组合。[14]

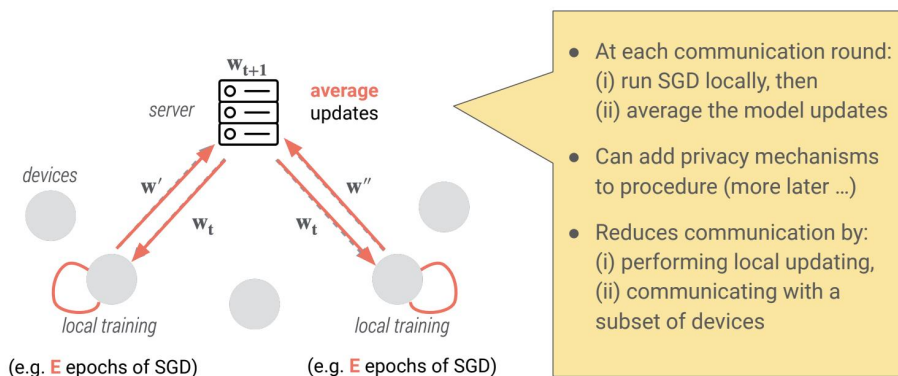


图 18.1 公共模型下发，本地执行若干训练步骤，再汇总本轮模型更新。（课程第 20 讲，第 30 页）

本节采用对本轮参与样本量归一化的组合约定：

$$w_{t+1} = \sum_{k \in S_t} \frac{n_k}{\sum_{j \in S_t} n_j} w_{t,E}^{(k)}.$$

E 表示本地更新步数。部分参与时，抽样机制与权重会影响全局估计，不能自动认为任意可用客户集合都代表原目标中的全部参与者。

代码示例 两个二次损失客户的一轮训练

```
def federated_round(global_weight, curvature, centers, counts,
                    local_steps, rate, proximal=0.0):
    local = np.full(len(curvature), global_weight, dtype=np.float64)
    for _ in range(local_steps):
        gradient = curvature * (local-centers)
        gradient += proximal * (local-global_weight) # 约束本轮内偏离公共起点的程度
        local -= rate * gradient
    weights = counts / counts.sum() # 本例所有客户参与
    return float(weights @ local), local
```

这里客户 k 的损失为 $F_k(w) = \frac{1}{2}a_k(w - c_k)^2$ ，所以梯度为 $a_k(w - c_k)$ 。`curvature` 保存 a_k ，`centers` 保存局部极小点 c_k ，`counts` 决定加权比例。代码中的 `proximal` 项取 0 时，就是这个简化模型上的本地更新与参数平均。

为什么一次本地更新与多次更新不同

若所有客户从同一个 w_t 开始，只做一次同学习率的完整局部梯度步骤，

$$\sum_k p_k(w_t - \eta \nabla F_k(w_t)) = w_t - \eta \sum_k p_k \nabla F_k(w_t),$$

恰好等于一次全局梯度下降。

多做几步以后，各客户的梯度在不同局部参数位置计算，这个等价关系通常不再成立。取两个客户曲率 $[1, 4]$ 、中心 $[-2, 3]$ 、样本权重比例 $1:3$ ，从 0 开始、学习率 0.1，一次本地更新后平均为 0.85；两次本地更新后平均为 1.345，而连续两次全局更新为 1.42375。

这个差异就是局部更新轨迹偏离的一种可见表现。更多本地步数减少通信频率，却可能增加统计偏移，不是无代价地把多轮同步通信省掉。

18.3 FedProx：限制本地模型离公共起点太远

FedProx 在本轮局部目标中加入近端项：

$$F_k(w) + \frac{\mu}{2} \|w - w_t\|^2.$$

其额外梯度为 $\mu(w - w_t)$ 。本地模型越偏离本轮公共起点，拉回作用越明显。[15]

这与普通权重衰减的参考点不同：普通 L2 惩罚把参数拉向 0，这里拉向当前轮的公共模型 w_t 。上一段程序把 `proximal` 设为正数，就能观察这种影响。

近端项可以帮助控制异质条件下的局部漂移，也允许结合不同精度的局部求解。但收敛结论依赖损失、异质性、步长和求解条件，不能理解为对任意数据与任意本地步数都保证更好。

18.4 一个共同模型，还是每个客户保留自己的模型

全局平均指标可能掩盖某些参与者效果较差的问题。如果数据分布差异很大，强迫所有参与者使用同一个参数向量，也未必是最合适的目标。

一种个性化思路是保留公共模型 w ，同时为客户 k 学习自己的 v_k ：

$$\min_{v_k} F_k(v_k) + \frac{\lambda}{2} \|v_k - w\|^2.$$

第二项让本地模型借用公共知识，又允许偏向自身数据。这类思想在 Ditto 等方法中得到具体组织。[16]

对于本章二次损失，固定 w 后可直接求得

$$v_k^* = \frac{a_k c_k + \lambda w}{a_k + \lambda}.$$

λ 很小时接近本地中心，较大时更接近公共模型。这个式子清楚展示了共享与个性化之间的取舍，不需要先记一个复杂框架名称。

公平目标要说明怎样聚合各客户损失

按样本量平均，意味着数据多的客户对目标影响更大；按客户等权平均，则是另一种目标。若更关心高损失客户，还可考虑平滑的倾斜目标：

$$F_\tau(w) = \frac{1}{\tau} \log \sum_k p_k e^{\tau F_k(w)}, \quad \tau > 0.$$

其梯度对局部梯度重新加权，高损失客户获得更大权重。这种选择也可能改变整体平均表现，不能把不同公平目标混成一个无需取舍的指标。

18.5 不上传原始样本，并不直接给出隐私保证

模型更新由数据计算而来，仍可能携带关于数据的信息。以单样本的线性层为例，若反向误差信号为标量 δ ，则

$$\nabla_W L = \delta x^\top, \quad \nabla_b L = \delta.$$

在相应前提下，观察者可能由二者的比值得到输入。这个简单关系说明：只把“上传数据”改成“上传梯度”，不能代替对信息泄露的分析。

安全聚合帮助协调方只获得某种聚合结果，而不直接看到每个参与者的单独更新；差分隐私则对相邻数据集改变输出分布的程度提供形式化约束。它们解决的范围不同，可以组合，但不能互相等同。

裁剪控制单个更新的影响

代码示例 把一份客户更新限制在给定范数内

```
def clip_client_update(update, limit):
    # 单独限制每个客户的更新，零向量无需除以范数。
    norm = np.linalg.norm(update)
    return update * min(1.0, limit/norm) if norm else update.copy()
```

更新 $[3, 4]$ 的范数为 5，阈值 1 时得到 $[0.6, 0.8]$ 。裁剪后的更新再参与聚合，可以限制相应贡献幅度。

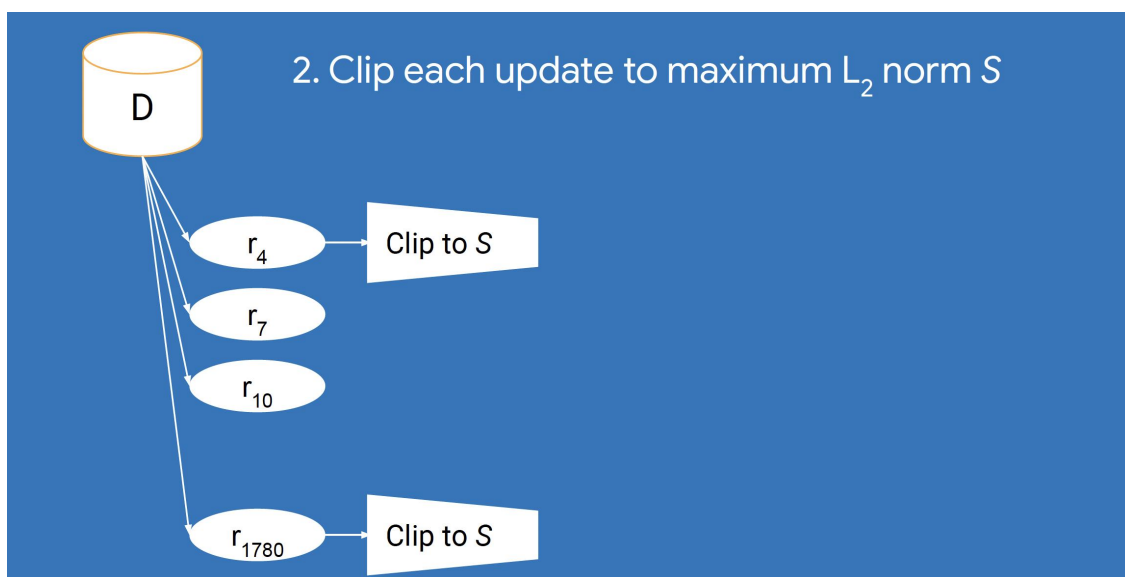


图 18.2 先限制每份更新的范数，再根据选定机制组织聚合与噪声。（课程第 20 讲，第 78 页）

仅仅加上随机噪声还不足以说明满足某个隐私参数。需要定义相邻关系是差一个样本还是差一个客户，说明抽样过程、噪声尺度、参与轮数和隐私会计。通信与训练协议的其他输出也必须与相应假设一致。

18.6 失败、偏差与状态管理

有的客户没有按时返回，不同轮次参与者也可能不同。系统需要决定是否等待、丢弃、使用部分结果或接受陈旧更新，并分析这些选择如何改变目标和更新过程。

如果总是只有较快设备参加，参与分布可能与原数据分布不同。模型版本、压缩方式、随机状态和局部优化器状态也要保持清楚。单纯把客户端代码运行成功，并不能证明多轮协作仍按预期目标训练。

联邦学习把整门课的几个主题再次连在一起：本地算子要正确，梯度和状态要可解释，通信要表达清楚的数学聚合，数值与压缩要守住相应误差约定。

附录 A 完整起点与代码运行

正文把程序拆开讲解，是为了让每一部分对应清楚的概念。这里把几个基础定义放回完整上下文，便于独立编译、调用和修改。

A.1 CUDA 的最小完整程序

保存下面的主程序为 01_relu.cu，同目录放置下一节的 common.cuh。它先建立输入、分配设备存储，再执行 kernel，最后取回并检查每一个结果。

```
#include "common.cuh"
// BEGIN kernel
__global__ void relu(const float* x, float* y, int n) {
    // 把块编号和块内线程编号合成全局元素编号。
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    // 最后一块可能多启动线程，只允许合法位置写出。
    if (i < n) y[i] = x[i] > 0.0f ? x[i] : 0.0f;
}
// END kernel
int main() {
    const int n = 1003;
    const size_t bytes = n * sizeof(float);
    std::vector<float> h_x(n), h_y(n);
    for (int i = 0; i < n; ++i) h_x[i] = (i - 501) * 0.125f;
    // BEGIN lifecycle
    float* d_x = nullptr;
    float* d_y = nullptr;
    // 把新设备地址写入 d_x; bytes 是字节数，不是元素个数。
    CUDA_CHECK(cudaMalloc(&d_x, bytes));
    CUDA_CHECK(cudaMalloc(&d_y, bytes));
    CUDA_CHECK(cudaMemcpy(d_x, h_x.data(), bytes, cudaMemcpyHostToDevice));
    const int threads = 256;
    const int blocks = (n + threads - 1) / threads;
    // 尖括号是启动配置；圆括号里的参数才进入核函数。
    relu<<<blocks, threads>>>(d_x, d_y, n);
    CUDA_CHECK(cudaGetLastError());
    // 等设备工作完成后再继续检查与复制结果。
    CUDA_CHECK(cudaDeviceSynchronize());
    CUDA_CHECK(cudaMemcpy(h_y.data(), d_y, bytes, cudaMemcpyDeviceToHost));
    CUDA_CHECK(cudaFree(d_y));
    CUDA_CHECK(cudaFree(d_x));
    // END lifecycle
    for (int i = 0; i < n; ++i)
        require(h_y[i] == std::max(h_x[i], 0.0f), "ReLU mismatch");
}
```

```
    std::puts("ReLU: 1003 elements passed");
}
```

A.2 设备内存辅助类

后续 CUDA 程序使用的 DeviceBuffer 只包装设备存储与复制操作。对象不允许复制，避免两个所有者重复释放同一块资源。读算法时，可以把 `buffer.data()` 视为前面手工管理的设备指针。

```
#pragma once
#include <cuda_runtime.h>
#include <cstdio>
#include <cstdlib>
#include <vector>
#include <utility>
#include <cstddef>
#include <cmath>
#include <algorithm>
#include <stdexcept>
inline void cuda_check(cudaError_t error, const char* expr, int line) {
    // 运行时返回错误时立即报告，避免带着无效指针继续计算。
    if (error != cudaSuccess) {
        std::fprintf(stderr, "line %d: %s: %s\n", line, expr,
            cudaGetErrorString(error));
        std::exit(EXIT_FAILURE);
    }
}
#define CUDA_CHECK(expr) cuda_check((expr), #expr, __LINE__)
inline void finish_kernel() {
    // 先检查当前错误状态，再等待设备完成，以暴露异步执行问题。
    CUDA_CHECK(cudaGetLastError());
    CUDA_CHECK(cudaDeviceSynchronize());
}
template<class T> class DeviceBuffer {
    T* ptr_ = nullptr;
    size_t size_;
public:
    explicit DeviceBuffer(size_t n) : size_(n) {
        // 长度为0时保留空指针，不执行零大小分配。
        if (n) CUDA_CHECK(cudaMalloc(&ptr_, n * sizeof(T)));
    }
    // 对象负责归还自己拥有的设备数组，不允许复制出第二个释放责任。
    ~DeviceBuffer() { if (ptr_) cudaFree(ptr_); }
    DeviceBuffer(const DeviceBuffer&) = delete;
    DeviceBuffer& operator=(const DeviceBuffer&) = delete;
    T* data() { return ptr_; }
    const T* data() const { return ptr_; }
    size_t size() const { return size_; }
```

```

void upload(const std::vector<T>& host) {
    if (host.size() != size_) throw std::invalid_argument("size mismatch");
    // upload 把主机数组复制到设备，长度必须与设备缓冲区一致。
    if (size_) CUDA_CHECK(cudaMemcpy(ptr_, host.data(), size_ * sizeof(T),
                                     cudaMemcpyHostToDevice));
}

std::vector<T> download() const {
    // 先建立足够大的主机数组，再把设备结果复制回来。
    std::vector<T> host(size_);
    if (size_) CUDA_CHECK(cudaMemcpy(host.data(), ptr_, size_ * sizeof(T),
                                     cudaMemcpyDeviceToHost));

    return host;
}
};

inline void require(bool ok, const char* why) {
    if (!ok) throw std::runtime_error(why);
}

inline bool near(float a, float b, float atol=1e-5f, float rtol=1e-4f) {
    return std::abs(a-b) <= atol + rtol * std::abs(b);
}

```

A.3 在 Linux 与 NVIDIA GPU 环境运行

需要支持所用代码的 NVIDIA GPU、驱动、CUDA Toolkit 及主机 C++ 编译器。先编译一个独立程序，再运行批量检查：

```

nvcc -std=c++17 -O2 examples/cuda/01_relu.cu -o relu
./relu
python3 examples/check_cuda.py
python3 examples/check_cuda.py --sanitize

```

脚本默认按本机 GPU 选择编译架构。交叉编译时，用 `--arch` 指定实际目标；不支持相应 WMMA 操作的设备，可用 `--skip-wmma` 选择基础 kernel 集合。`--sanitize` 需要 Compute Sanitizer，并分别检查内存、共享内存竞争和同步使用。

预期输出给出各示例的结果检查是否满足。性能测量应另行指定输入、预热、重复次数和计时区间，不要把检查程序本身的运行时间直接当作 kernel 基准。

A.4 Python 参考与绑定工程

NumPy 参考程序用小规模输入展示数学关系和执行步骤，可以逐个运行：

```

python3 examples/python/06_autograd_check.py
python3 examples/python/08_train_framework.py
python3 examples/python/17_serving.py

```

安装 NumPy 与 pybind11 后，绑定工程可通过 CMake 构建：

```

cmake -S examples/bindings -B build/bindings \
    -Dpybind11_DIR="$(python3 -m pybind11 --cmakedir)"

```

```
cmake --build build/bindings
PYTHONPATH=build/bindings python3 examples/bindings/check.py
```

构建和运行使用同一个 Python 环境。多配置生成器的输出目录可能还包含配置名，应按实际模块位置调整导入路径。

PyTorch 例子使用相应框架环境。自定义 CUDA 扩展的源码、构建配置与调用示例位于 `examples/torch_extension`。DDP 例子还要求匹配的多 GPU 进程配置。每份程序先说明其输入与支持范围，再根据预期结果理解其职责。

附录 B 一阶自动微分引擎的完整定义

下面是正文使用的 NumPy 后端 Tensor。它支持基本逐元素运算、二维矩阵乘法、归约、重排和一阶反向。数据采用 float64 以便数值检查；这不是要求实际深度学习训练始终使用双精度。

B.1 节点、局部规则与反向遍历

把代码保存为 autograd.py, 即可与正文的小型 MLP 配合使用。数组值按只读约定保存, 参数更新建立新的叶节点。局部 VJP 返回与输入节点顺序一致的梯度贡献。

```
"""Small first-order reverse-mode engine; immutable float64 values, NumPy backend."""
import numpy as np

# BEGIN unbroadcast
def sum_to_shape(grad, shape):
    # 先消去广播添加的前导维, 再处理原来长度为1的轴。
    while grad.ndim > len(shape):
        grad = grad.sum(axis=0)
    axes = tuple(i for i, size in enumerate(shape)
                  if size == 1 and grad.shape[i] != 1)
    if axes:
        grad = grad.sum(axis=axes, keepdims=True)
    return grad.reshape(shape)
# END unbroadcast

class Tensor:
    def __init__(self, data, requires_grad=True, parents=(), vjp=None):
        self.data = np.array(data, dtype=np.float64, copy=True)
        # 保存前向值供反向使用, 避免无意的原地修改破坏导数。
        self.data.flags.writeable = False
        self.requires_grad = requires_grad
        self.parents = tuple(parents) if requires_grad else ()
        self.vjp = vjp
        self.grad = None

    @staticmethod
    def coerce(value):
        return value if isinstance(value, Tensor) else Tensor(value, requires_grad=False)

    def __add__(self, other):
        other = Tensor.coerce(other)
        shapes = self.data.shape, other.data.shape
        return Tensor(self.data + other.data,
```

```

        self.requires_grad or other.requires_grad, (self, other),
        lambda g: (sum_to_shape(g, shapes[0]), sum_to_shape(g, shapes[1]))
    ))
    __radd__ = __add__

# BEGIN multiply
def __mul__(self, other):
    other = Tensor.coerce(other)
    # 局部反向使用前向时对应的两个操作数值。
    a, b = self.data, other.data
    def vjp(g):
        return (sum_to_shape(g * b, a.shape),
                sum_to_shape(g * a, b.shape))
    return Tensor(a * b, self.requires_grad or other.requires_grad,
                  (self, other), vjp)

# END multiply
__rmul__ = __mul__

def __neg__(self):
    return self * -1
def __sub__(self, other):
    return self + (-Tensor.coerce(other))
def __rsub__(self, other):
    return Tensor.coerce(other) + (-self)
def __truediv__(self, other):
    return self * (Tensor.coerce(other) ** -1)
def __rtruediv__(self, other):
    return Tensor.coerce(other) * (self ** -1)

def __pow__(self, power):
    a = self.data
    local = np.zeros_like(a) if power == 0 else power * a ** (power - 1)
    return Tensor(a ** power, self.requires_grad, (self,), lambda g: (g * local,))

def exp(self):
    value = np.exp(self.data)
    return Tensor(value, self.requires_grad, (self,), lambda g: (g * value,))
def log(self):
    a = self.data
    if np.any(a <= 0):
        raise ValueError("log requires positive input")
    return Tensor(np.log(a), self.requires_grad, (self,), lambda g: (g / a,))
def sin(self):
    a = self.data
    return Tensor(np.sin(a), self.requires_grad, (self,), lambda g: (g * np.cos(a)
,))
def relu(self):
    a = self.data

```

```

    return Tensor(np.maximum(a, 0), self.requires_grad, (self,), lambda g: (g * (a
> 0),))

def __matmul__(self, other):
    other = Tensor.coerce(other)
    a, b = self.data, other.data
    if a.ndim != 2 or b.ndim != 2:
        raise ValueError("this teaching implementation supports 2D matmul")
    return Tensor(a @ b, self.requires_grad or other.requires_grad, (self, other),
                  lambda g: (g @ b.T, a.T @ g))

def sum(self, axis=None, keepdims=False):
    shape = self.data.shape
    if axis is None:
        axes = tuple(range(len(shape)))
    else:
        raw = (axis,) if isinstance(axis, int) else tuple(axis)
        axes = tuple(a + len(shape) if a < 0 else a for a in raw)
    value = self.data.sum(axis=axes, keepdims=keepdims)
    def vjp(g):
        if not keepdims and axes:
            g = np.expand_dims(g, axis=tuple(sorted(axes)))
        return (np.broadcast_to(g, shape),)
    return Tensor(value, self.requires_grad, (self,), vjp)

def mean(self, axis=None, keepdims=False):
    if axis is None:
        count = self.data.size
    else:
        axes = (axis,) if isinstance(axis, int) else tuple(axis)
        count = int(np.prod([self.data.shape[i] for i in axes]))
    if count == 0:
        raise ValueError("mean of an empty reduction is undefined here")
    return self.sum(axis, keepdims) / count

def reshape(self, shape):
    old = self.data.shape
    return Tensor(self.data.reshape(shape), self.requires_grad, (self,),
                  lambda g: (g.reshape(old),))

def transpose(self, axes=None):
    if axes is None:
        axes = tuple(reversed(range(self.data.ndim)))
    axes = tuple(a % self.data.ndim for a in axes)
    inverse = tuple(np.argsort(axes))
    return Tensor(self.data.transpose(axes), self.requires_grad, (self,),
                  lambda g: (g.transpose(inverse),))

@property
def T(self):

```

```

        return self.transpose()

# BEGIN backward
def backward(self, seed=None, accumulate=False):
    if seed is None:
        if self.data.size != 1:
            raise ValueError("a non-scalar output needs an explicit seed")
        seed = np.ones_like(self.data)
    seed = np.asarray(seed, dtype=np.float64)
    if seed.shape != self.data.shape:
        raise ValueError("seed shape must equal output shape")
    order, seen = [], set()
    # 先访问输入再记录当前节点，得到正向拓扑顺序。
    def visit(node):
        if node in seen:
            return
        seen.add(node)
        for parent in node.parents:
            visit(parent)
        order.append(node)
    visit(self)
    grads = {self: seed}
    # 反向拓扑序保证下游使用者先传播贡献。
    for node in reversed(order):
        if node not in grads or not node.requires_grad:
            continue
        g = grads[node]
        if accumulate and not node.parents and node.grad is not None:
            node.grad = node.grad + g
        else:
            node.grad = g.copy()
        if node.vjp is None:
            continue
        for parent, contribution in zip(node.parents, node.vjp(g)):
            if parent.requires_grad:
                # 同一输入可能有多条路径或重复边，每份贡献都要相加。
                grads[parent] = grads.get(parent, 0) + contribution
    # END backward

# BEGIN logsumexp
def logsumexp(x, axis=-1, keepdims=False):
    # 稳定移位不求导；完整 log-sum-exp 中对移位的依赖会抵消。
    maximum = Tensor(x.data.max(axis=axis, keepdims=True), requires_grad=False)
    result = (x - maximum).exp().sum(axis=axis, keepdims=True).log() + maximum
    if not keepdims:
        shape = list(result.data.shape)
        del shape[axis]
        result = result.reshape(tuple(shape))

```

```
    return result

def cross_entropy(logits, labels):
    n, classes = logits.data.shape
    target = np.eye(classes)[labels]
    correct_score = (logits * target).sum(axis=1)
    return (logsumexp(logits, axis=1) - correct_score).mean()
# END logsumexp
```

B.2 从最小图开始核对

```
from autograd import Tensor
x = Tensor(3.0)
shared = x * x
loss = shared + shared + x # 两次使用 shared, 另有一条直接路径
loss.backward()
print(float(loss.data), float(x.grad)) # 预期输出 21.0 13.0
```

接着可以改变输入形状，观察广播梯度；换成矩阵乘法，检查梯度中的转置；最后把它们组合成训练循环。每次只增加一项机制，更容易定位形状、求导或状态管理中的错误。

附录 C 贯通几个难点的自检

C.1 线程和归约

若块大小为 8，线程位于块 2 的块内位置 3，其全局编号为 $2 \times 8 + 3 = 19$ 。数据只有 17 个元素时，这条线程不能写出元素 19；但若它属于需要全块协作的算法，仍应按该算法的阶段约定处理加载与屏障。

把减半归约直接用于 6 个元素时，步长先为 3 再为 1，会丢下一个部分和。对 $[1, 2, 3, 4, 5, 6]$ ，第一轮得到前部 $[5, 7, 9]$ ，下一轮只合并 5 与 7。这个例子解释了为什么正文的简化 kernel 要求块大小为 2 的幂，而不是把这个条件当作任意惯例。

C.2 广播和梯度

形状 $(1, 3, 1)$ 的参数广播到 $(2, 3, 4)$ 输出，若输出梯度全为 1，则每个参数位置收到 $2 \times 4 = 8$ 份贡献，返回形状仍应为 $(1, 3, 1)$ 。少做了求和或误删了轴，都会违反这个可手算的结果。

共享节点与重复输入边也应分别检查。 $u = x * x$ 中， x 作为两个输入位置出现； $L = u + u$ 中， u 被两次使用。对节点做拓扑去重，不允许顺手丢掉这些数学上的贡献。

C.3 并行拆分与归一化

列分片的线性层产生不同输出特征，结果按特征拼接；行分片产生同一输出的部分和，结果相加。若第二层的输出偏置在每个 rank 都加入再求和，就多加了 $(p-1)b$ 。

本地批大小不同、损失采用本地 mean 时，全局梯度应按样本数量加权。微批次累积、数据并行平均和损失内部归一化需要放在同一条公式里核对，避免重复缩放。

C.4 在线注意力为什么要重缩放旧状态

第一块分数为 $[0, 0]$ ，对应值为 $[1, 3]$ ，于是 $m = 0, \ell = 2, u = 4$ 。第二块只有分数 $\log 2$ 、值 10。新最大值为 $\log 2$ ，旧状态缩放系数为 $1/2$ ：

$$\ell' = 2/2 + 1 = 2, \quad u' = 4/2 + 10 = 12, \quad O = 12/2 = 6.$$

直接一次性计算，权重比例为 $[1, 1, 2]$ ，输出同样为 6。若不重缩放旧状态，两种计算就不会一致。

参考文献

- [1] 王鹏帅、楚梦渝、梁一韬. 人工智能中的编程. 北京大学, 2025 年秋季课程课件, 01–20.
- [2] NVIDIA. CUDA Programming Guide. <https://docs.nvidia.com/cuda/cuda-programming-guide/>
- [3] NVIDIA. C/C++ Language Extensions. <https://docs.nvidia.com/cuda/cuda-programming-guide/05-appendices/cpp-language-extensions.html>
- [4] NVIDIA. CUDA C++ Best Practices Guide. <https://docs.nvidia.com/cuda/cuda-c-best-practices-guide/>
- [5] NVIDIA. cuBLAS Library. <https://docs.nvidia.com/cuda/cublas/>
- [6] pybind11. NumPy and buffer protocol. <https://pybind11.readthedocs.io/en/stable/advanced/pycpp/numpy.html>
- [7] PyTorch. Extending PyTorch. <https://docs.pytorch.org/docs/stable/notes/extending.html>
- [8] PyTorch. Custom C++ and CUDA Operators. https://docs.pytorch.org/tutorials/advanced/cpp_custom_ops.html
- [9] NVIDIA. NCCL Collective Operations. <https://docs.nvidia.com/deeplearning/nccl/user-guide/docs/usage/collectives.html>
- [10] PyTorch. DistributedDataParallel. <https://docs.pytorch.org/docs/stable/generated/torch.nn.parallel.DistributedDataParallel.html>
- [11] Tri Dao et al. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. 2022. <https://arxiv.org/abs/2205.14135>
- [12] Woosuk Kwon et al. Efficient Memory Management for Large Language Model Serving with Page-dAttention. 2023. <https://arxiv.org/abs/2309.06180>
- [13] Yaniv Leviathan, Matan Kalman, Yossi Matias. Fast Inference from Transformers via Speculative Decoding. 2022. <https://arxiv.org/abs/2211.17192>
- [14] Brendan McMahan et al. Communication-Efficient Learning of Deep Networks from Decentralized Data. AISTATS, 2017. <https://proceedings.mlr.press/v54/mcmahan17a.html>
- [15] Tian Li et al. Federated Optimization in Heterogeneous Networks. MLSys, 2020. <https://arxiv.org/abs/1812.06127>
- [16] Tian Li et al. Ditto: Fair and Robust Federated Learning Through Personalization. ICML, 2021. <https://proceedings.mlr.press/v139/li21h.html>